

版权所有，盗版必究

- 小时达司机端设计图：
<https://codesign.qq.com/s/P4VIZMyVoRZq6wL/ALwE9V4JwNRZX1D/inspect>
- 小时达接口地址：<https://apifox.com/apidoc/shared-4b036830-59b1-4526-b00a-61df2b3d4ae1>
- 小时达司机端账号注册地址：<https://fe-slwl-manager.itheima.net/#/driver-register>
- 上课代码仓库地址- <https://gitee.com/luckybo0027/xsd.git>
- 完整代码仓库-<https://gitee.com/shuiruohanyu/quick-delivery>

分层架构逻辑模型

HarmonyOS应用的分层架构主要包括三个层次：产品定制层products、基础特性层features和公共能力层commons，为开发者构建了一个清晰、高效、可扩展的设计架构。

- 不同设备意味着不同的入口。products是个入口可能包含手机 + 平板 + 2in1 + 手表 + Car

三层: 三个大的文件夹. UI界面适配的. 可复用的业务逻辑、组合各个模块

- **产品定制层**产品定制层专注于满足不同设备或使用场景（如应用）的个性化需求，包括UI设计、资源和配置，以及针对特定场景的交互逻辑和功能特性。产品定制层的功能模块独立运作，同时依赖基础特性层和公共能力层来实现具体功能。作为应用的入口，产品定制层是用户直接互动的界面。为满足特定产品需求，产品定制层可灵活地调整和扩展，从而满足各种使用场景。
- **基础特性层**基础特性层位于公共能力层之上，用于存放基础特性集合，例如相对独立的功能UI和业务逻辑实现。该层的每个功能模块都具有高内聚、低耦合、可定制的特点，以支持产品的灵活部署。基础特性层为上层的产品定制层提供稳健且丰富的基础功能支持，包括UI组件、基础服务等。同时依赖于下层的公共能力层为其提供通用功能和服务。为了增强系统的可扩展性和维护性，基础特性层将功能进行模块化处理。例如，一个应用的底部导航栏中的每个选项都可能是一个独立的业务模块。
- **公共能力层**公共功能层用于存放公共基础能力，集中了例如公共UI组件、数据管理、外部交互以及工具库等的共享功能。应用可以共享和调用这些公共能力。公共能力层为上层的基础特性层和产品定制层提供稳定可靠的功能支持，确保整个应用的稳定性和可维护性。公共能力层包括但不限于以下组成：
 - 公共UI组件：这些组件被设计成通用且高度可复用的，确保在不同的应用程序模块间保持一致的用户体验。公共UI组件提供了标准化且友好的界面，帮助开发者快速实现常见的用户交互需求，例如提示、警告、加载状态显示等，从而提高开发效率和用户满意度。
 - 数据管理：负责应用程序中数据的存储和访问，包括应用数据、系统数据等，提供了统一的数据管理接口，简化数据的读写操作。通过集中式的数据管理方式不仅使得数据的维护更为简单，而且能够保证数据的一致性和安全性。
 - 外部交互：负责应用程序与外部系统的交互，包括网络请求、文件I/O、设备I/O等，提供统一的外部交互接口，简化应用程序与外部系统的交互。开发者可以更为方便地实现应用程序的网络通信、数据存储和硬件接入等功能，从而加速开发流程并保证程序的稳定性和性能。
 - 工具库：提供一系列常用工具函数和类，例如字符串处理、日期时间处理、加密解密、数据压缩解压等，帮助开发者提高效率和代码质量。

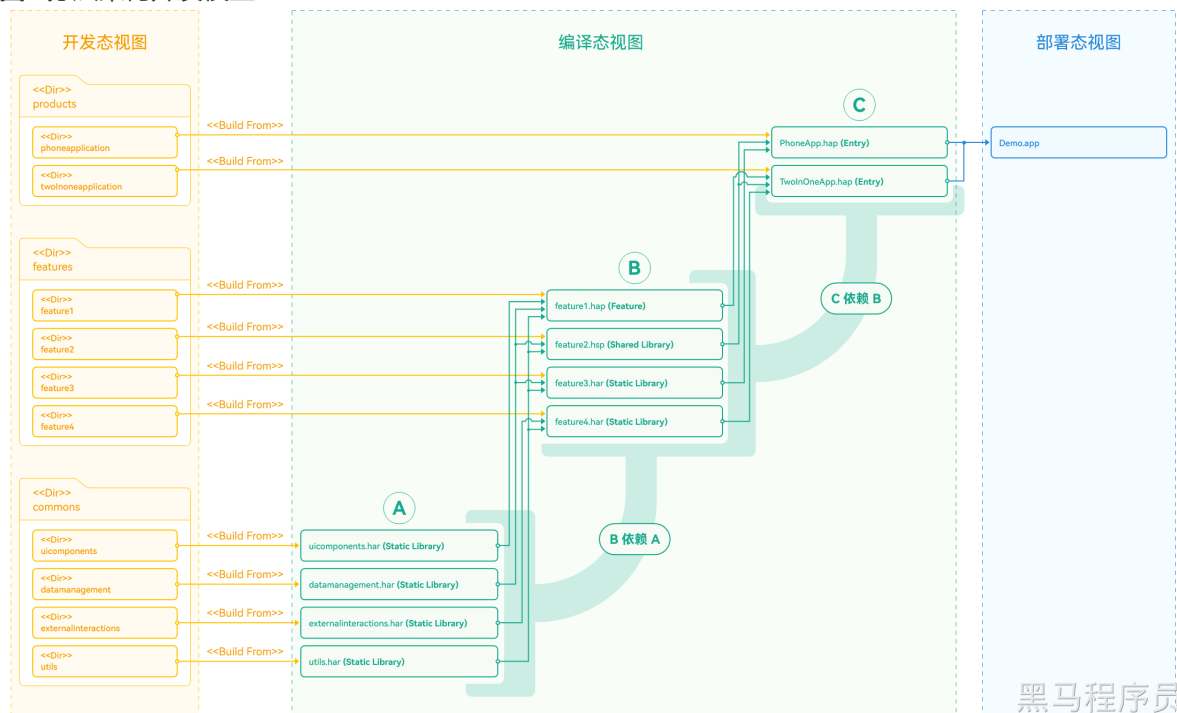
Hap-entry

Hsp-共享包

Har包-静态共享包

1. 开发模型

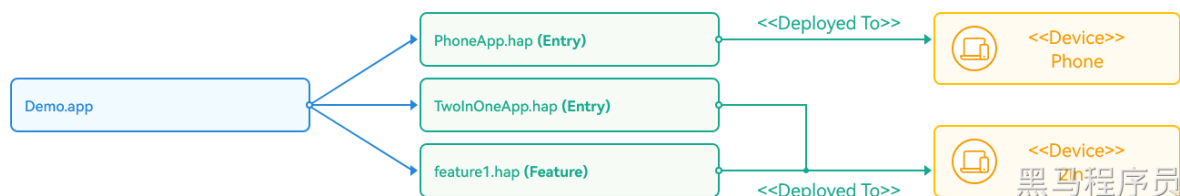
图2 分层架构开发模型



- 产品定制层**产品定制层的各个子目录会被编译成一个Entry类型的HAP，作为应用的主入口。该层主要针对跨多种设备，为各种设备形态集成相应的功能和特性。产品定制层被划分为多个功能模块，每个功能模块都针对特定的设备或使用场景设计，并根据具体的产品需求进行功能及交互的定制开发。**说明**
 - 在产品定制层，开发者可以从不同设备对应应用的UX设计和功能两个维度，结合具体的业务场景，选择一次编译生成相同或者不同的HAP（或其组合）。
 - 通过使用定制多目标构建产物的定制功能，可以将应用所对应的HAP编译成各自的.app文件，用于上架到应用市场。
- 基础特性层**在基础特性层中，功能模块根据部署需求被分为两类。对于需要通过Ability承载的功能，可以设计为Feature类型的HAP，而对于不需要通过Ability承载的功能，根据是否需要实现按需加载，可以选择设计为HAR模块或者HSP模块，编译后对应HAR包或者HSP包。
- 公共能力层**公共能力层的各子目录将被编译成HAR包，而他们只能被产品定制层和基础特性层所依赖，不允许存在反向依赖。该层旨在提取模块化公共基础能力，为上层提供标准接口和协议，从而提高整体的复用率和开发效率。

2. 部署模型

图3 分层架构部署模型（不同设备的定制）



应用程序（.app文件）在流水线或应用市场上被解包为 $n \times \text{Entry}$ 类型的HAP + $n \times \text{Feature}$ 类型的HAP，根据设备类型和使用场景将应用部署到不同类型的设备上，实现多端的统一用户体验。

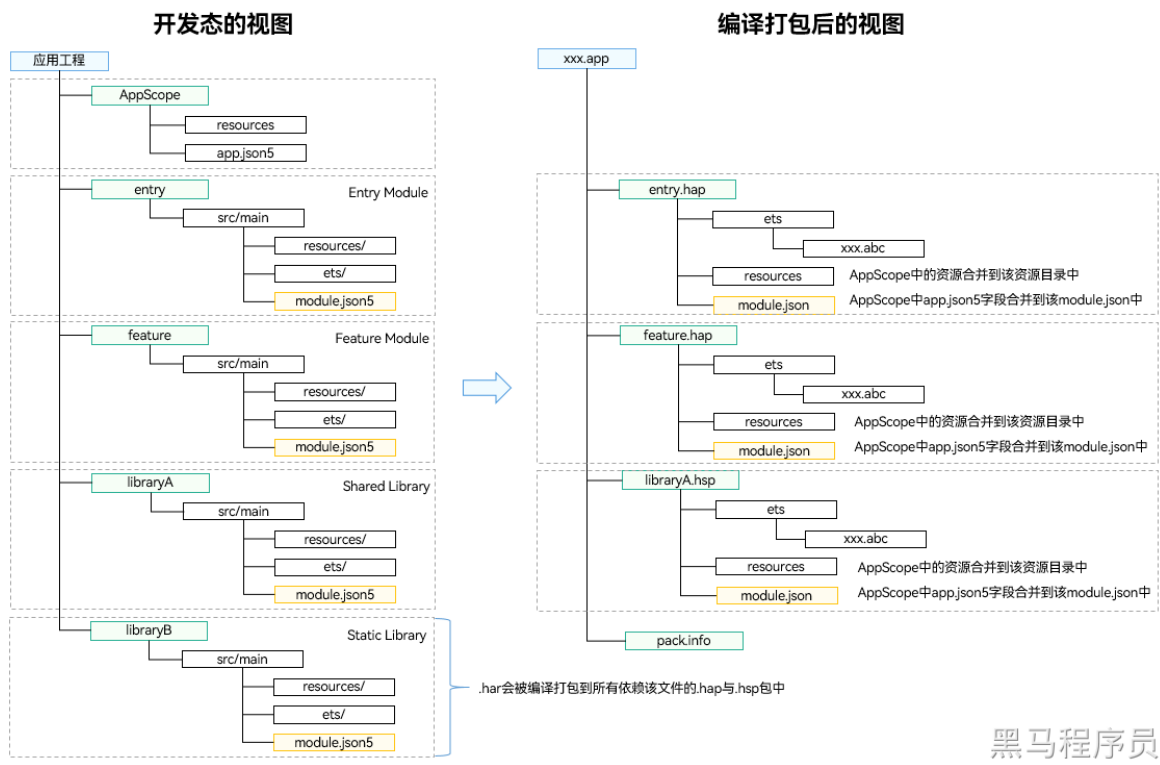
说明

当Entry类型的HAP和Feature类型的HAP被分发并部署到相应的设备上时，他们所依赖的HSP也会一同被分发并部署到相应的设备上。

在部署模型中，每个Entry类型的HAP代表了应用的入口点，而Feature类型的HAP则包含了应用的特定功能模块。允许应用能够以模块化的方式适配和部署，从而满足不同设备和场景的需求。

该部署模型不仅优化了应用的组织结构，也为保持应用在各种设备和场景中的一致性提供了支持。通过

按照设备类型和使用场景来区分和部署不同的HAP，能确保无论在何种设备或场景中，用户都能获得统一且高质量



选择合适的包类型

HAP、HAR、HSP三者的功能和使用场景总结对比如下：

Module类型	包类型	说明
Ability	HAP	应用的功能模块，可以独立安装和运行，必须包含一个entry类型的HAP，可选包含一个或多个feature类型的HAP。
Static Library	HAR	静态共享包，编译态复用。 - 支持应用内共享，也可以发布后供其他应用使用。 - 作为三方库，发布到OHPM私仓，供公司内部其他应用使用。 - 作为三方库，发布到OHPM中心仓，供其他应用使用。 - 多包（HAP/HSP）引用相同的HAR时，会造成多包间代码和资源的重复拷贝，从而导致应用包膨大。
Shared Library	HSP	动态共享包，运行时复用。 - 当前仅支持应用内共享。 - 当多包（HAP/HSP）同时引用同一个共享包时，采用HSP替代HAR，可以避免HAR造成的多包间代码和资源的重复拷贝，从而减小应用包大小。

开发者可以根据实际场景所需的能力，选择相应类型的包进行开发。在后续的章节中还会针对如何使用HAP、HAR、HSP分别展开详细介绍。

规格	HAP	HAR	HSP
支持在配置文件中声明UIAbility组件与ExtensionAbility组件	√	×	×
支持在配置文件中声明pages页面	√	×	√
支持包含资源文件与.so文件	√	√	√
支持依赖其他HAR文件	√	√	√
支持依赖其他HSP文件	√	√	√
支持在设备上独立安装运行	√	×	×

黑马程序员

说明

- HAR虽然不支持在配置文件中声明pages页面，但是可以包含pages页面，并通过命名路由的方式进行跳转。
- 由于HSP仅支持应用内共享，如果HAR依赖了HSP，则该HAR文件仅支持应用内共享，不支持发布到二方仓或三方仓供其他应用使用，否则会导致编译失败。
- HAR和HSP均不支持循环依赖，也不支持依赖传递。

黑马程序员

3. 单层项目架构

```
1  ets
2  |— common                // 通用模块
3  |   |— builders          // - 通用组件 @Builder
4  |   |— components        // - 通用组件 @Component
5  |   |— constants         // - 常量数据
6  |   |— images            // - 图片资源
7  |   |— utils             // - 工具类
8  |— entryability          // 入口UIAbility
9  |   |— EntryAbility.ts
10 |— models                // - 数据模型
11 |— pages                 // - 页面组件
12 |   |— HomePage.ets
13 |   |— Index.ets
14 |— views                 // - 页面对应自定义组件
15 |   |— HomePage
16 |   |— Index
```

Fence 1

三层项目架构代码实例
[优秀实践-HMOS世界 \(ArkTS\) .zip](#)

4. 三层架构

```

1 project-root
2 |— commons // 基础层（通用组件、工具类、资源）
3 |   |— basic // - 通用模块
4 |— features // 需求层（业务组件）
5 |   |— home // - 需求模块
6 |   |— mine // - 需求模块
7 |— products // 入口层（页面）
8 |   |— phone // - 设备模块
9

```

Fence 2

三层架构模式

HMOS世界应用采用三层架构模式，整体架构如图所示：

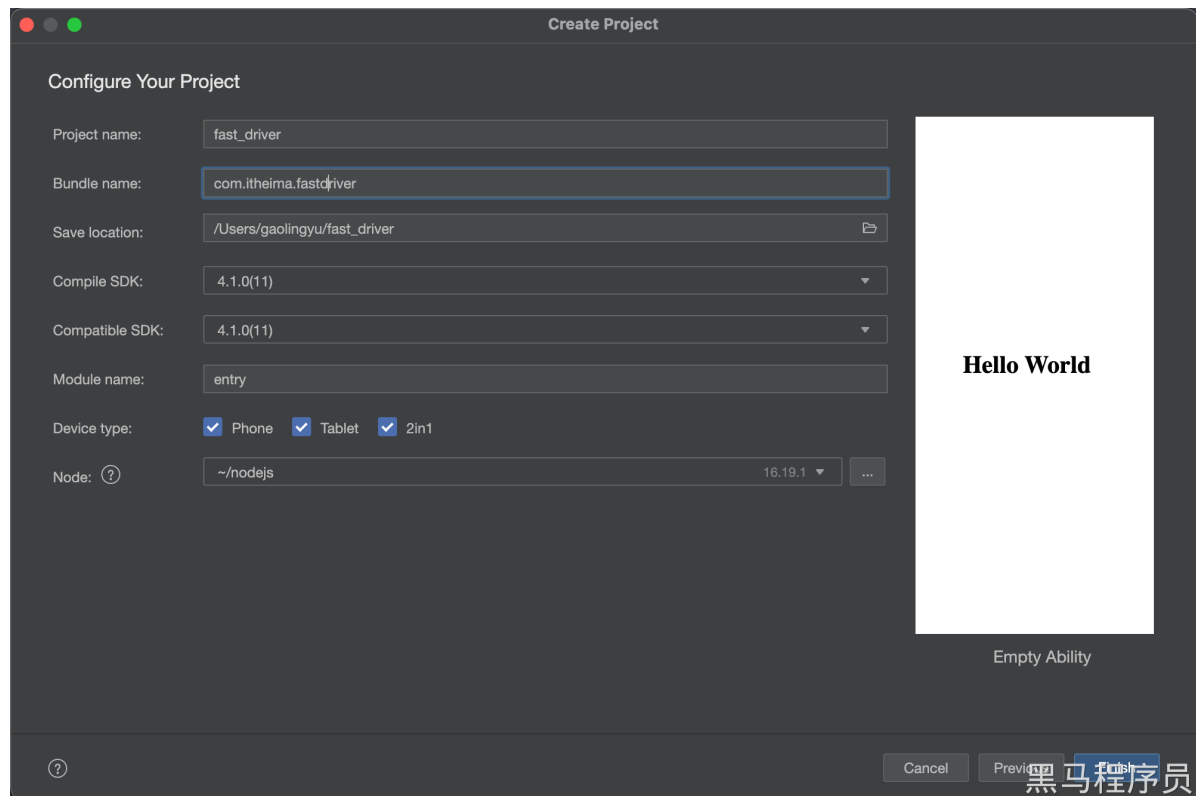
- products层（产品定制层）：用于针对不同设备形态进行功能和特性集成。products层各个子目录分别编译为一个Entry类型的HAP包，作为应用的主入口，products层不可以横向调用。目前仅支持phone设备，后期会支持更多设备类型。
- features层（基础特性层）：用于放置不同的业务模块，例如探索、学习、活动、我的、登录等，供产品灵活部署。分别编译成HSP包，features层可以横向调用及依赖common层，同时可以被products层不同设备形态的HAP所依赖，但是不能反向依赖products层。
- common层（公共能力层）：用于存放项目的工具类、公共常量等，例如公共UI组件、日志管理、通用工具方法等。编译成HAR包，只可以被products和features依赖，不可以反向依赖。

通过上述分层的架构设计，将不同业务拆分为HSP/HAR共享包进行模块化开发，可以更高效地管理工程、多人协作和特性集成。同时架构也具备更好的扩展性，当需要新增业务模块或设备类型时，新增features和products模块即可，并充分复用代码。

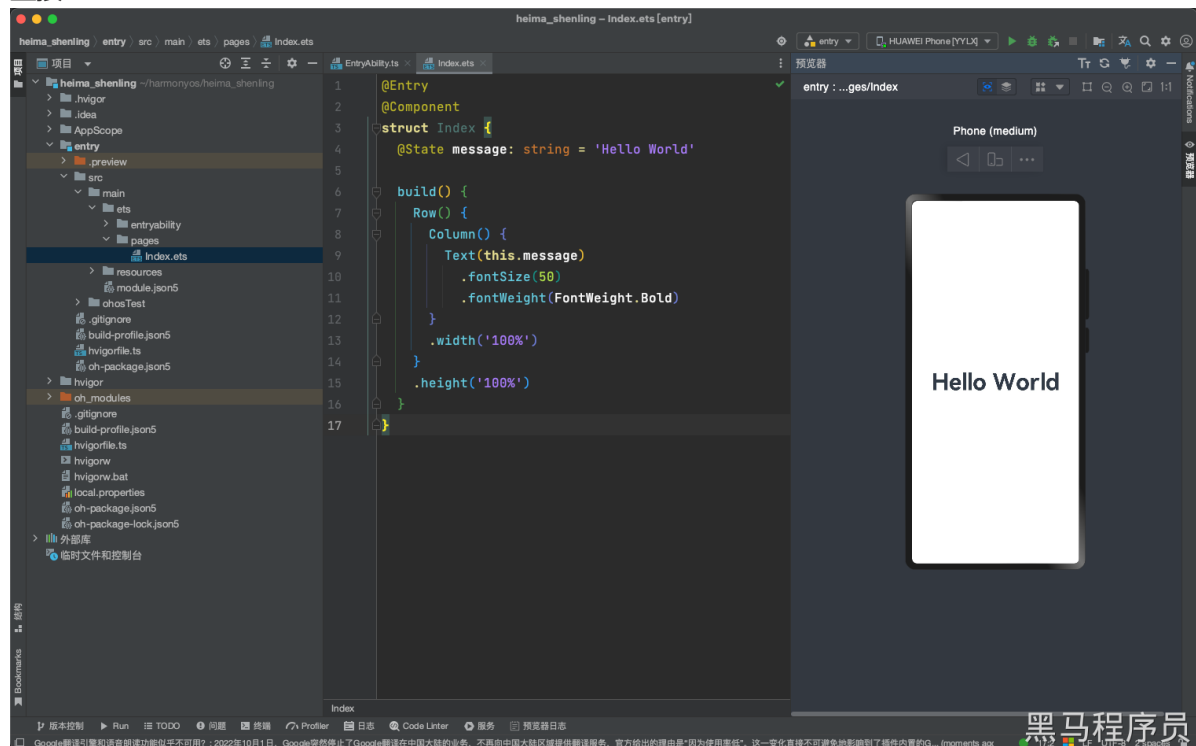
黑马程序员

创建项目-使用git管理

使用DevEcoStudio创建一个空项目



直接finish



- 建立码云远程仓库

仓库名称 * ✓
heima-shening

归属 路径 * ✓
shuirohanyu / heima-shening

仓库地址: <https://gitee.com/shuirohanyu/heima-shening>

仓库介绍 0/100
用简短的语言来描述一下吧

☐ 开源 (所有人可见) ②
☒ 私有 (仅仓库成员可见)

☐ 初始化仓库 (设置语言、.gitignore、开源许可证)
☐ 设置模板 (添加 Readme、Issue、Pull Request 模板文件)
☐ 选择分支模型 (仓库创建后将根据所选模型创建分支)

创建

黑马程序员

- 拷贝远程仓库链接

快速设置— 如果你知道该怎么操作, 直接使用下面的地址

HTTPS SSH <https://gitee.com/shuirohanyu/heima-shening>

我们强烈建议所有的git仓库都有一个 README, LICENSE, .gitignore 文件

初始化 readme 文件

Git入门? 查看 [帮助](#), [Visual Studio](#) / [TortoiseGit](#) / [Eclipse](#) / [Xcode](#) 下如何连接本站, [如何导入仓库](#)

简易的命令行入门教程:

黑马程序员

- 找到创建项目的目录下初始化仓库 并添加远程仓库 推送

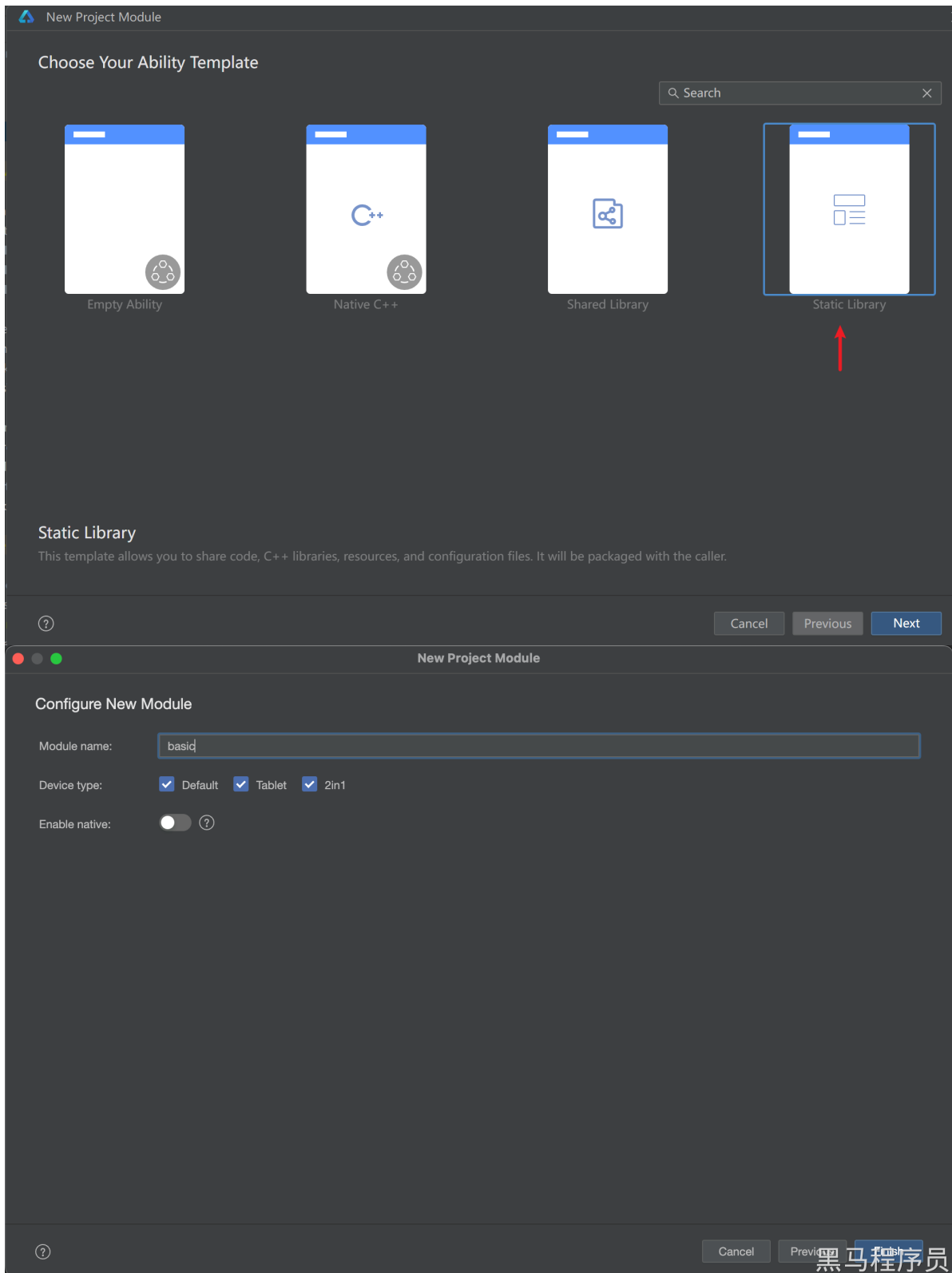
```
1 $ git init
2 $ git add .
3 $ git commit -m "初始化神领物流项目"
4 $ git remote add origin <远程仓库地址>
5 $ git push -u origin master
```

Fence 3

创建一个静态har包， 导入素材和资源

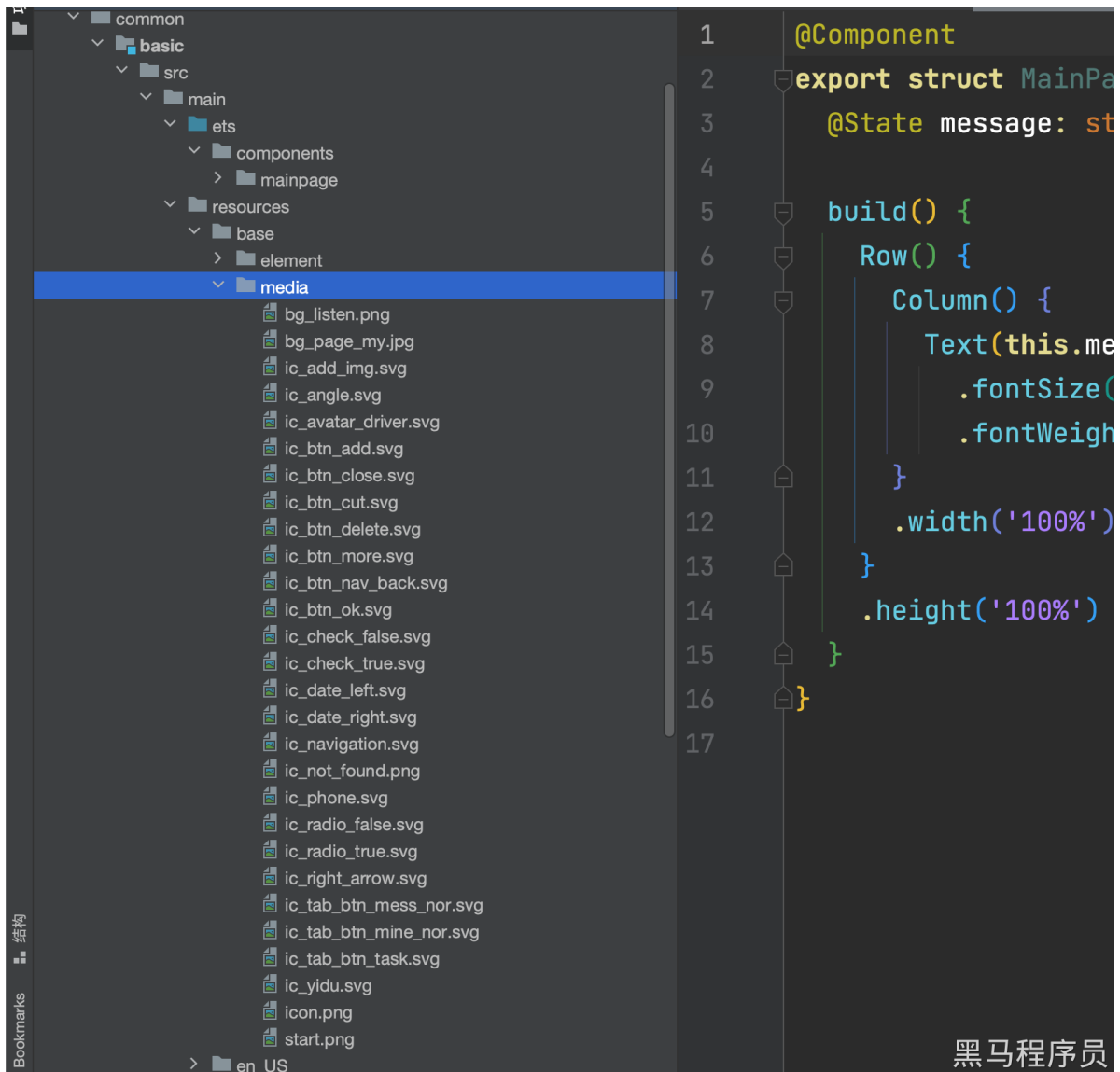
本次项目采用单层架构 + har公共包的模式进行开发

- 新建一个common目录, 在该目录下新建一个静态模块 Static Library



[media.zip](#)

1. 下载该media文件夹， 将所有的图标素材拖动到静态资源包下的 src/main/resources/base/media 下



- 导入色值

```
1 {  
2   "color": [  
3     {  
4       "name": "start_window_background",  
5       "value": "#FFFFFF"  
6     },  
7     {  
8       "name": "primary",  
9       "value": "#FE6A3D"  
10    },  
11    {  
12      "name": "primary_disabled",  
13      "value": "#FADCD9"  
14    },  
15    {  
16      "name": "success",  
17      "value": "#27BA9B"  
18    },  
19    {  
20      "name": "warning",  
21      "value": "#FFAB2E"  
22    },  
23  ],  
24 }
```

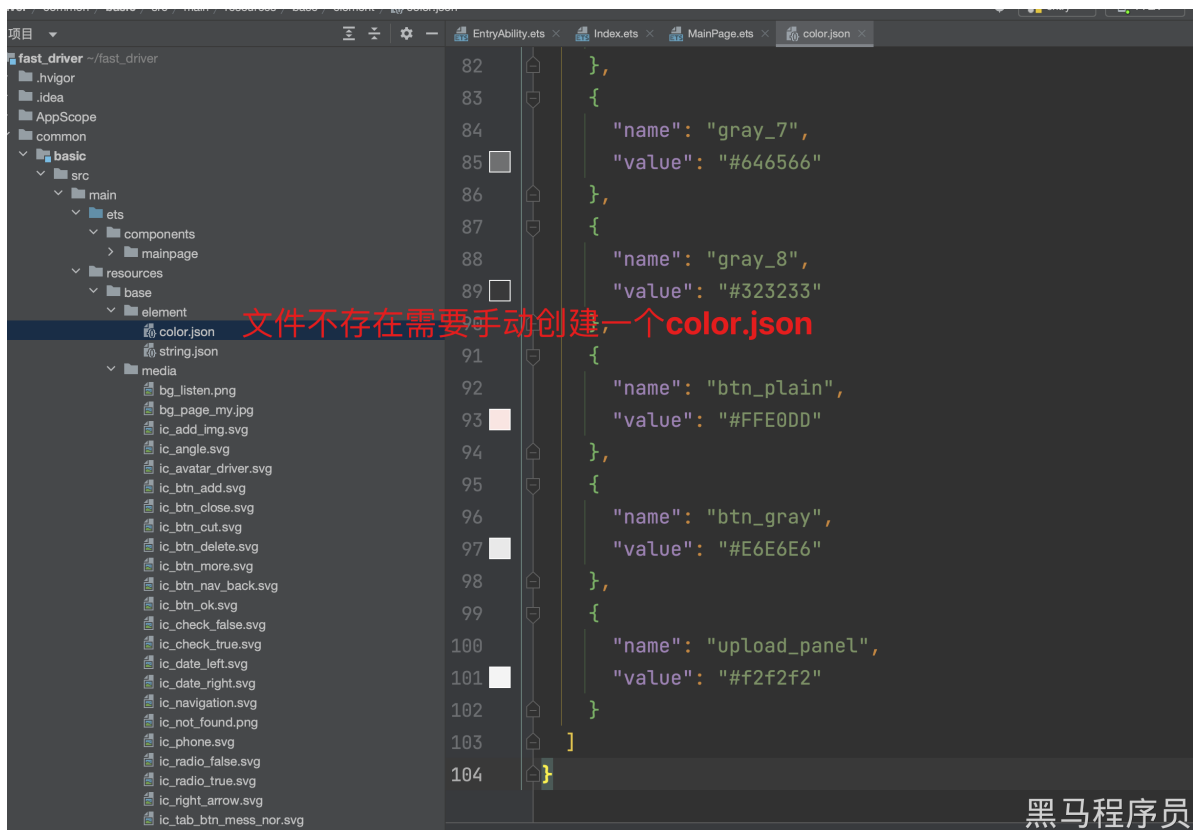
```
23   {
24     "name": "danger",
25     "value": "#FF4C4C"
26   },
27   {
28     "name": "text_primary",
29     "value": "#2A2929"
30   },
31   {
32     "name": "text_secondary",
33     "value": "#818181"
34   },
35   {
36     "name": "text_placeholder",
37     "value": "#C2C1C1"
38   },
39   {
40     "name": "border",
41     "value": "#D9D9D9"
42   },
43   {
44     "name": "background_divider",
45     "value": "#EEEEEE"
46   },
47   {
48     "name": "background_page",
49     "value": "#F4F4F4"
50   },
51   {
52     "name": "black",
53     "value": "#000000"
54   },
55   {
56     "name": "white",
57     "value": "#FFFFFF"
58   },
59   {
60     "name": "gray_1",
61     "value": "#F7F8FA"
62   },
63   {
64     "name": "gray_2",
65     "value": "#F2F3F5"
66   },
67   {
68     "name": "gray_3",
69     "value": "#EBEDF0"
70   },
71   {
72     "name": "gray_4",
73     "value": "#DEDEE0"
74   },
75   {
76     "name": "gray_5",
77     "value": "#C8C9CC"
```

```

78     },
79     {
80         "name": "gray_6",
81         "value": "#969799"
82     },
83     {
84         "name": "gray_7",
85         "value": "#646566"
86     },
87     {
88         "name": "gray_8",
89         "value": "#323233"
90     },
91     {
92         "name": "btn_plain",
93         "value": "#FFE0DD"
94     },
95     {
96         "name": "btn_gray",
97         "value": "#E6E6E6"
98     },
99     {
100        "name": "upload_panel",
101        "value": "#f2f2f2"
102    }
103 ]
104 }

```

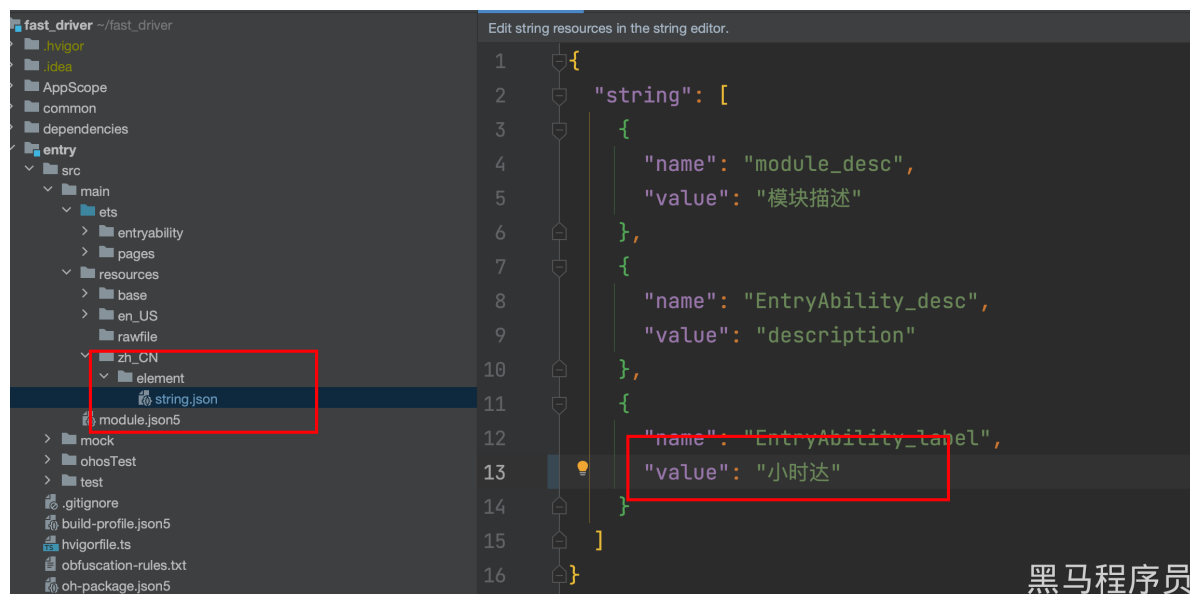
Fence 4



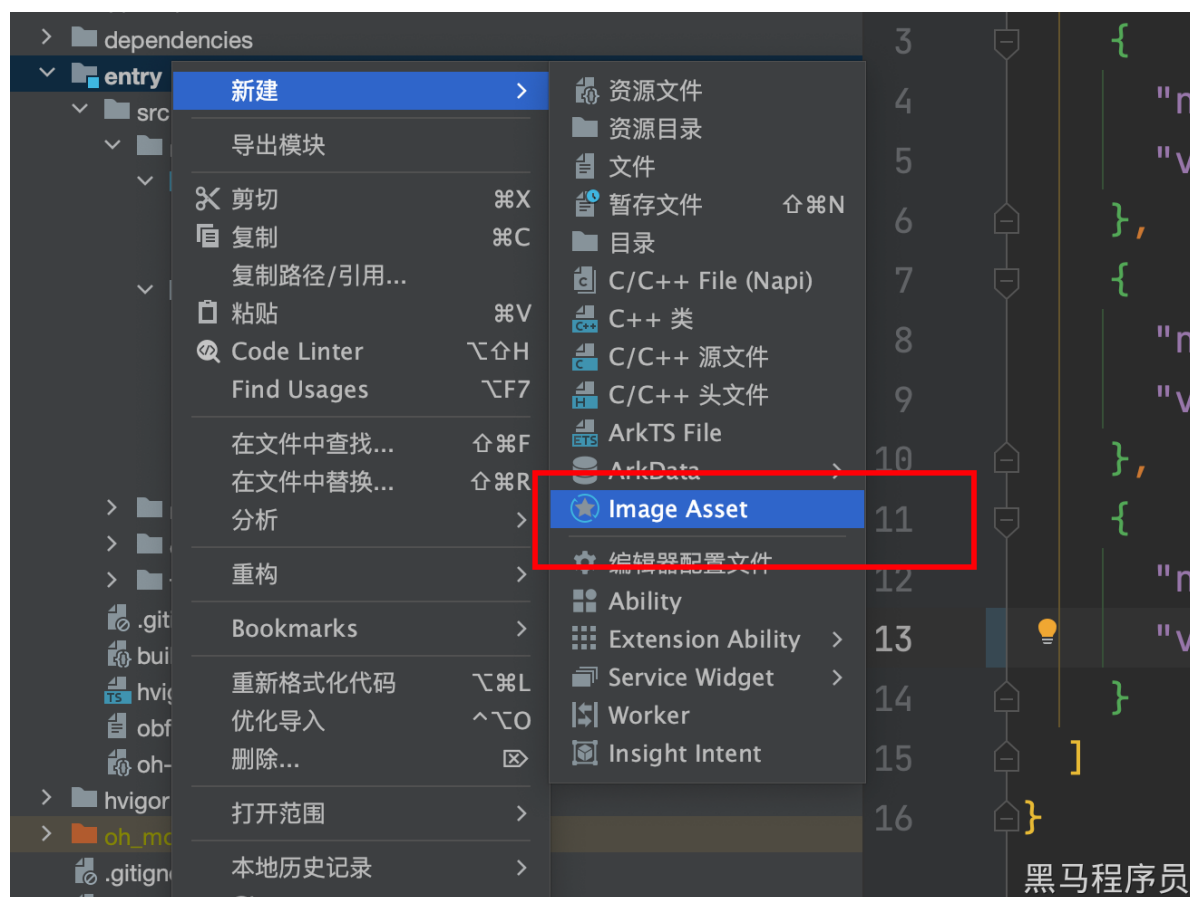
提交代码

修改项目名称和图标

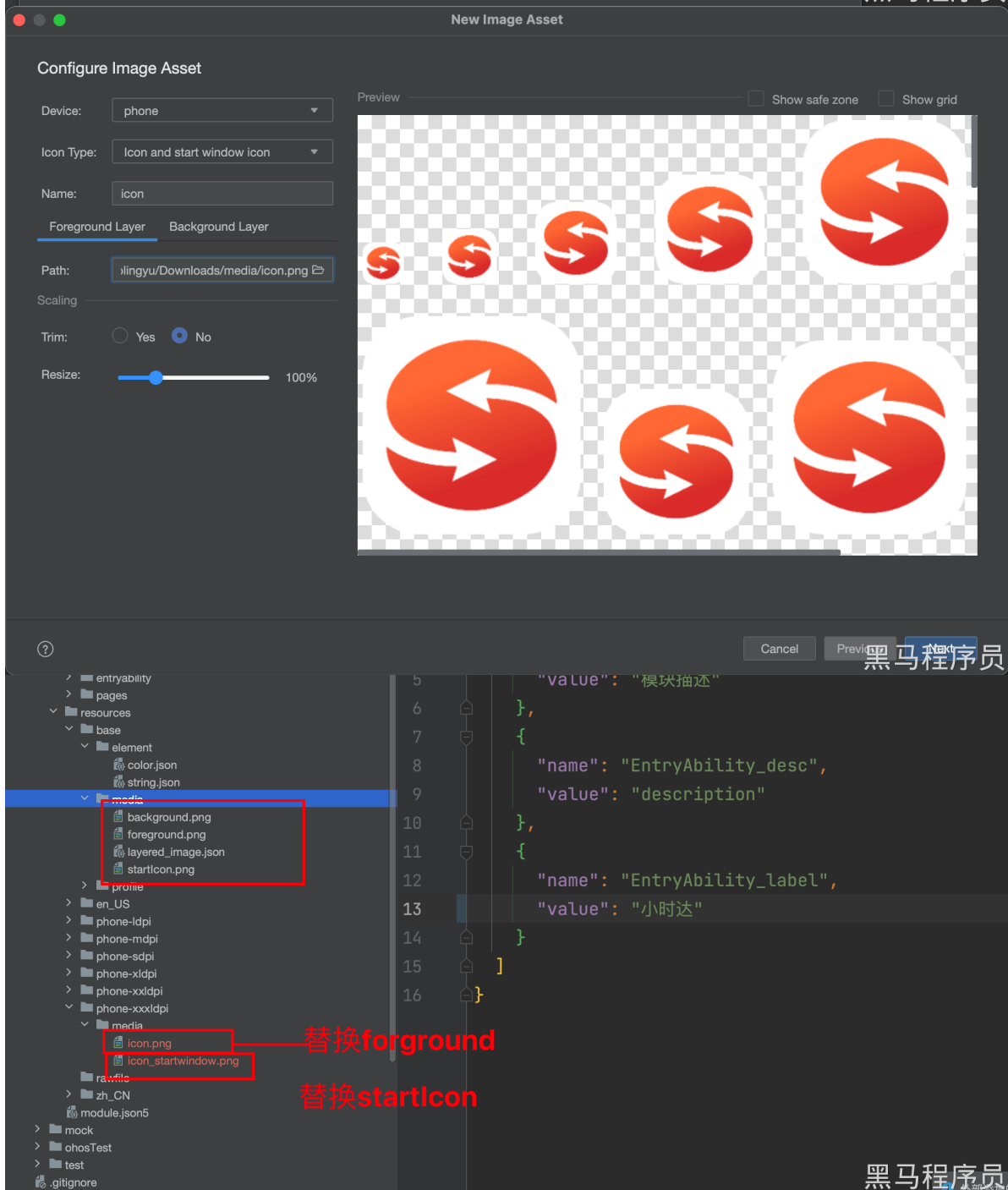
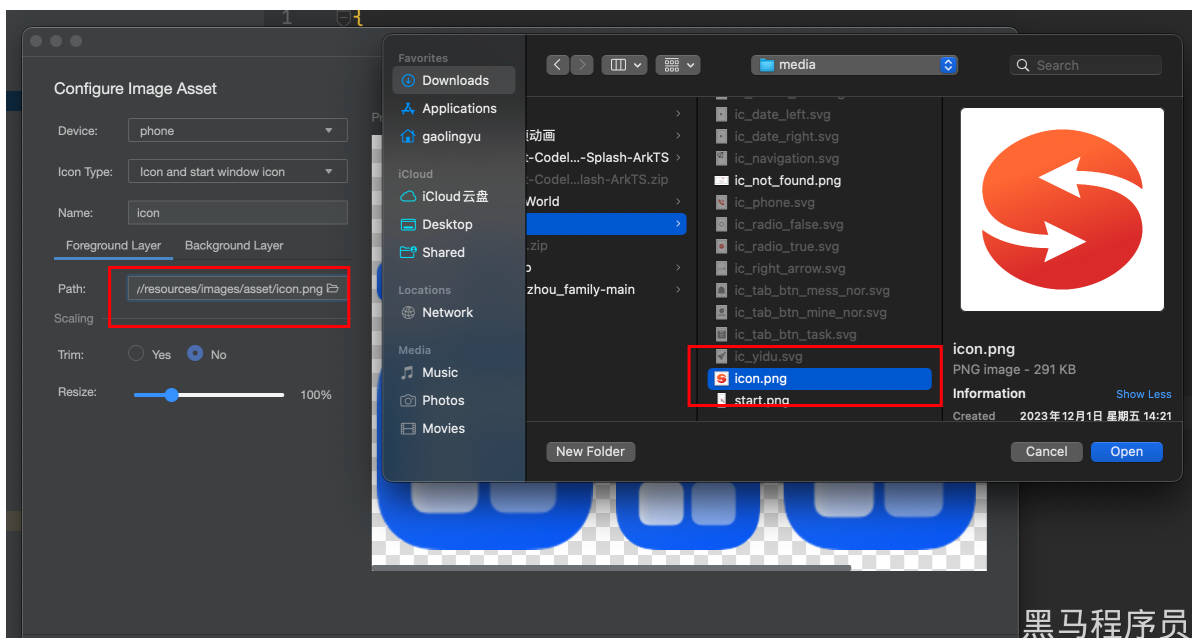
- 修改项目名称

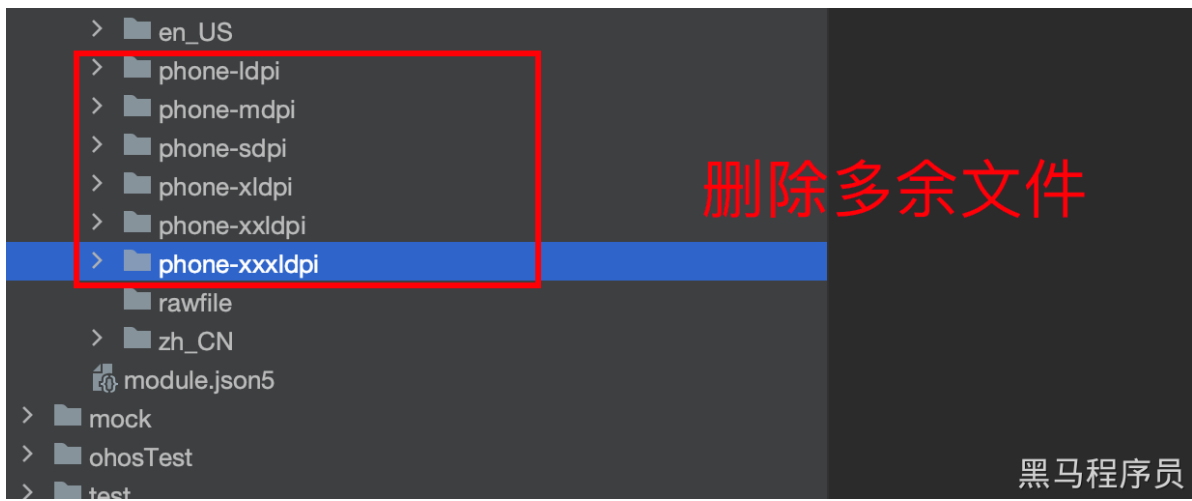


- 通过生成图标工具创建app图标



- 选择图标





提交代码

公共静态包(HAR)基础目录搭建

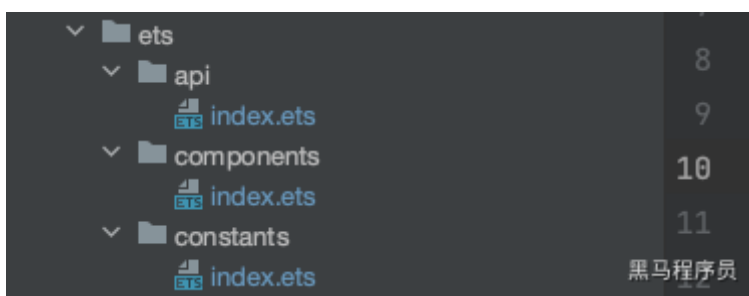
在har包中建立如下目录

- 按照客户端开发的需要我们要在项目中建立如下结构

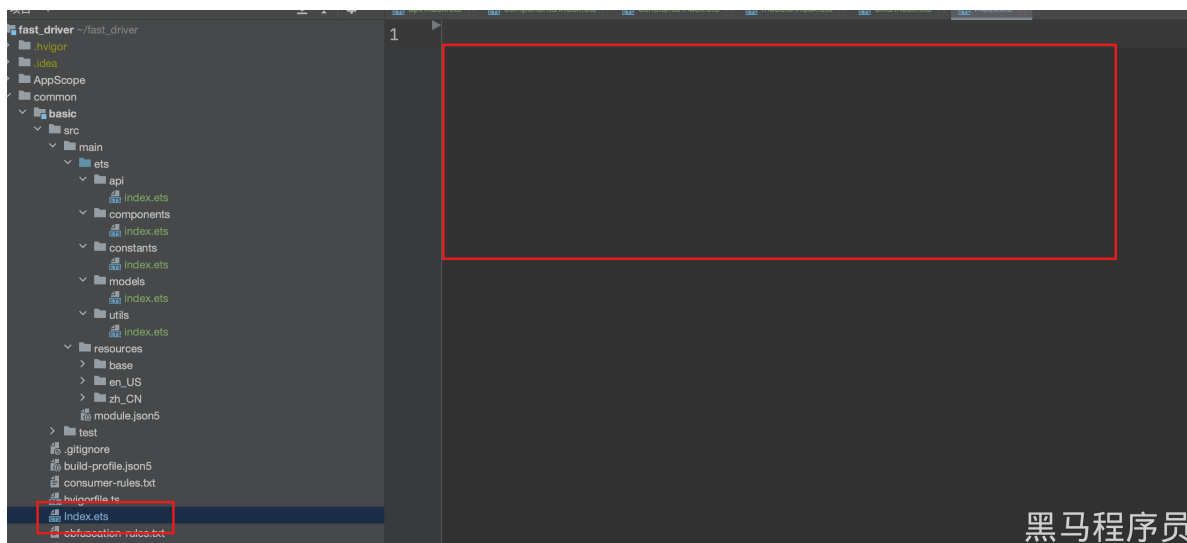
1	-ets	
2	-api	负责存放所有的请求
3	-components	负责放置所有的公共组件
4	-constants	负责存放所有的常量
5	-models	负责存放所有的数据类型结构
6	-utils	负责存放所有的工具

Fence 5

- 在项目中添加上述的目录，并统一新建一个index.ets文件



- 清空har包下index.ets文件内容



提交代码

搭建广告展示页Start

广告页的思路

-华为有广告业务，但是我们不用-ad模块

想自定义广告-

场景：app启动-有广告需求，就打开广告页，没有的话就去登录或者主页

腾讯体育的广告-启动有广告页，退到后台的情况下，再次进入前台也会有广告



分析- 广告页作为一个app启动的首页，应该是在我们应用启动就进去的。

- 有的app有的需要广告页，有的不需要，搞个配置呗!!!
1. 通过首选项配置存储我们的一些常用配置，比如要不要广告页，还有广告页的路由地址，点击广告页跳转的链接，广告页倒计时的秒数
 2. 在入口处进行判断是否需要广告页，需要的话，跳转广告页-广告页根据设置的参数进行渲染
 3. 有的同学可能会问，广告页能不能设置-因为运营人员肯定不能每次都去改我们底层的代码-这里我还可以设置成动态的-就是初始化的时候通过请求去读一下云端请求，然后把我们的图片和一些参数配置下来，这样每次你启动app就是运营人员给你配置的广告和设置了

- 新建一个关于广告类的数据模型-basic/models/advert.ets

```
1
2 export class AdvertClass {
3     showAd: boolean = false // 显示广告
4     adTime: number = 5 // 广告时长
5     adUrl?: string = '' // 广告链接
6     adImg?: ResourceStr = '' // 广告图片
7 }
```

Fence 6

- 在model/index.ets中进行统一导出

```
1 export * from './advert'
```

Fence 7

- 在utils中新建一个关于读取首选项的类，用来读取和设置首选项的广告设置

```
1 import { AdvertClass } from '../models'
2 import preferences from '@ohos.data.preferences'
3 import { USER_SETTING, USER_SETTING_AD } from '../constants'
4 // 默认广告选项
5 export const defaultAd: AdvertClass = {
6     showAd: true,
7     adTime: 5,
8     adImg: $r('app.media.start')
9 }
10 export class UserSettingClass {
11     context: Context
12     constructor(context: Context) {
13         this.context = context
14     }
15     // 获取存储用户信息的首选项仓库
16     getStore () {
17         return preferences.getPreferencesSync(this.context, USER_SETTING)
18     }
19     // 设置用户广告设置
20     setUserAd (ad: AdvertClass) {
21         const adStore = this.getStore()
22         adStore.putSync(USER_SETTING_AD, JSON.stringify(ad))
23         await adStore.flush()
24     }
25     // 获取广告配置
26     getUserAd (): AdvertClass {
27         const adStore = this.getStore()
28         return JSON.parse( adStore.getSync(USER_SETTING_AD,
29             JSON.stringify(defaultAd)) as string) as AdvertClass
30     }
31 }
```

Fence 8

在上面代码中，我们设计了读取和设置广告的两个方法，并且提供了一个默认广告的设置

- 在utils中统一导出

```
1 | export * from './setting'
```

Fence 9

- 上面还用到了两个常量，我们同样需要在constants目录下定义一个文件专门用来记录-setting

```
1 | export const USER_SETTING = 'fast_driver_setting' // 用来存储用户设置的首选项的key
2 | export const USER_SETTING_AD = 'fast_driver_setting_ad' // 用来存储用户设置广告首
   | 选项的key
```

Fence 10

- 同样在constants/index.ets文件中导出

```
1 | export * from './setting'
```

Fence 11

- 在basic/Index.ets统一导出

```
1 | export * from './src/main/ets/models'
2 | export * from './src/main/ets/constants'
3 | export * from './src/main/ets/utils'
```

Fence 12

entry模块-oh-package.json5

- 在ability中引入该har包依赖

```
1 | {
2 |   "name": "entry",
3 |   "version": "1.0.0",
4 |   "description": "Please describe the basic information.",
5 |   "main": "",
6 |   "author": "",
7 |   "license": "",
8 |   "dependencies": {
9 |     "@hm/basic": "file:../common/basic"
10 |   }
11 | }
12 |
13 |
```

Fence 13

ability中判断

- 导入包

```
1 | import { AdvertClass, UserSettingClass, defaultAd } from '@hm/basic'
2 |
```

Fence 14

- 判断

```
1 | async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
2 |   // Main window is created, set main page for this ability
3 | }
```

```

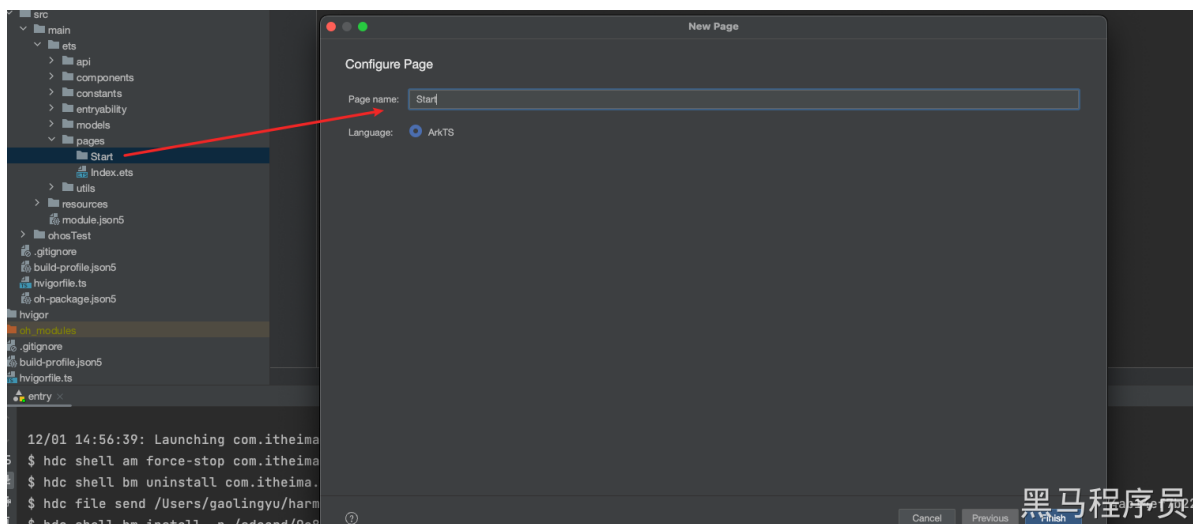
4      // 通过模拟请求拿到广告信息
5      const ad = await new Promise<AdvertClass>((resolve, reject) => {
6          setTimeout(() => {
7              resolve(defaultAd) // 广告信息
8          }, 500) // 0.5后秒请求成功
9      })
10
11
12      const setting = new UserSettingClass(this.context) // getContext拿到的是
13      undefined
14      setting.setUserAd(ad) // 写入首选项
15      // 检查是否有广告
16      if(ad.showAd) {
17          // 展示广告的情况 展示广告页 有时限
18          // 创建一个子窗口 子窗口加载广告 广告播完 窗口销毁
19      }
20      windowStage.loadContent('pages/Index'); // 正常加载页面

```

Fence 15

这里我们模拟了一个请求，给了一个默认广告，写入首选项-正常加载主页

1. 在pages下新建Start目录，下面新建Start的page页面



- 实现Start页的页面结构及倒计时逻辑

```

1  import { UserSettingClass, AdvertClass } from '@hm/basic'
2
3  @Entry
4  @Component
5  struct Start {
6      userSetting: UserSettingClass = new UserSettingClass(getContext(this))
7      @State
8      adObj: AdvertClass = {
9          showAd: false,
10         adTime: 0
11     }
12     timer: number = -1
13
14     async aboutToAppear() {
15         this.adObj = await this.userSetting.getUserAd()

```



```

16     this.timer = setInterval(() => {
17         if(this.adObj.adTime === 0) {
18             clearInterval(this.timer)
19             return
20         }
21         this.adObj.adTime--
22     }, 1000)
23 }
24 build() {
25     Stack({ alignContent: Alignment.TopEnd }) {
26         Image(this.adObj.adImg).objectFit(ImageFit.Cover)
27         Text(`${this.adObj.adTime}秒后跳过`)
28             .padding({ left: 10, right: 10 })
29             .margin({ right: 20, top: 20 })
30             .height(30)
31             .fontSize(14)
32             .borderRadius(15)
33             .backgroundColor($r("app.color.background_page"))
34             .textAlign(TextAlign.Center)
35     }
36     }.height('100%').width('100%')
37 }
38 }
39

```

Fence 16

- 使用子窗口模式加载广告

我们可以使用windowStage的createSubWindow来实现在当前页面上创建一个窗口

```

1  if (result.showAd) {
2      const win = await windowStage.createSubWindow("ad_window")
3      await win.showWindow()
4      win.setUIContent("pages/Start/Start")
5  }

```

Fence 17

- 广告页在广告结束或者点击跳过广告时，关闭广告

Start/Start页面

```

1  closewin () {
2      window.findwindow("ad_window").destroywindow()
3  }

```

Fence 18

- 完成Start页面代码

```

1  import { UserSettingClass, AdvertClass } from '@hm/basic'
2  import { window } from '@kit.ArkUI'
3
4  @Entry
5  @Component
6  struct Start {
7      userSetting: UserSettingClass = new UserSettingClass(getContext(this))

```

```

8   @State
9   adObj: AdvertClass = {
10     showAd: false,
11     adTime: 0
12   }
13   timer: number = -1
14   closewin () {
15     window.findwindow("ad_window").destroywindow()
16   }
17   async aboutToAppear() {
18     this.adObj = await this.userSetting.getUserAd()
19     this.timer = setInterval(() => {
20       if(this.adObj.adTime === 0) {
21         clearInterval(this.timer)
22         this.closewin()
23         return
24       }
25       this.adObj.adTime--
26     }, 1000)
27   }
28   aboutToDisappear(): void {
29     clearInterval(this.timer)
30   }
31   build() {
32     Stack({ alignContent: Alignment.TopEnd }) {
33       Image(this.adObj.adImg).objectFit(ImageFit.Cover)
34       Text(`${this.adObj.adTime}秒后跳过`)
35         .padding({ left: 10, right: 10 })
36         .margin({ right: 20, top: 20 })
37         .height(30)
38         .fontSize(14)
39         .borderRadius(15)
40         .backgroundColor($r("app.color.background_page"))
41         .textAlign(TextAlign.Center)
42         .onClick(() => {
43           this.closewin()
44         })
45     }
46     }.height('100%').width('100%')
47   }
48 }
49

```

Fence 19

这里请注意-如果需要使用线上的广告图片，需要开启网络权限

完整代码-EntryAbility

```

1  import { AbilityConstant, UIAbility, Want } from '@kit.AbilityKit';
2  import { hilog } from '@kit.PerformanceAnalysisKit';
3  import { window } from '@kit.ArkUI';
4  import { AdvertClass, UserSettingClass } from '@hm/basic'
5
6  export default class EntryAbility extends UIAbility {
7    onCreate(want: Want, launchParam: AbilityConstant.LaunchParam): void {
8      hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
9

```

```

9      }
10
11      onDestroy(): void {
12          hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onDestroy');
13      }
14
15      async onWindowStageCreate(windowStage: window.WindowStage): Promise<void>
16      {
17          // Main window is created, set main page for this ability
18          hilog.info(0x0000, 'testTag', '%{public}s', 'Ability
19      onWindowStageCreate');
20          // 这里要尝试去读一下运营的配置-我们现在还没有实现接口, 直接模拟一下
21          const userSetting = new UserSettingClass(this.context)
22          const result = await new Promise<AdvertClass>((resolve, reject) => {
23              setTimeout(() => {
24                  resolve({
25                      showAd: true,
26                      adTime: 5,
27                      adImg: $r("app.media.start")
28                  } as AdvertClass)
29              }, 500)
30          })
31          await userSetting.setUserAd(result) // 写入首选项
32          if (result.showAd) {
33              const win = await windowStage.createSubwindow("ad_window")
34              await win.showWindow()
35              win.setUIContent("pages/Start/Start")
36          }
37          windowStage.loadContent('pages/Index');
38      }
39
40      onWindowStageDestroy(): void {
41          // Main window is destroyed, release UI related resources
42          hilog.info(0x0000, 'testTag', '%{public}s', 'Ability
43      onWindowStageDestroy');
44      }
45
46      onForeground(): void {
47          // Ability has brought to foreground
48          hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onForeground');
49      }
50
51      onBackground(): void {
52          // Ability has back to background
53          hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onBackground');
54      }
55  }
56

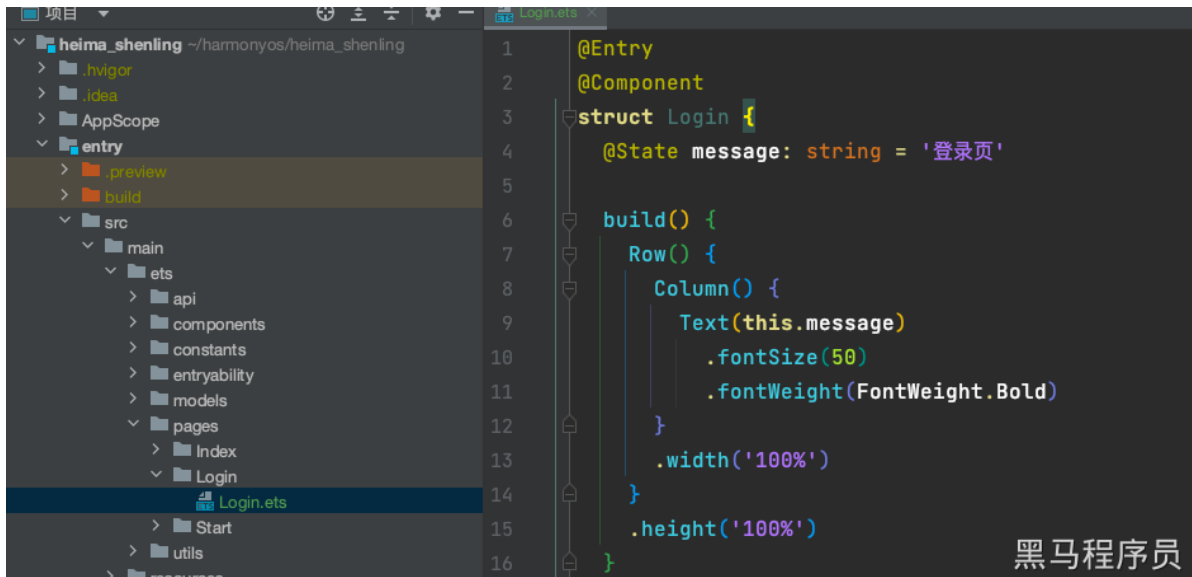
```

初始化App检查token

由于手机端是必须要求用户登录的，所以我们需要在跳转到主页前检查是否有token，有token的话跳转到主页，没有token的话跳转到登录

Entry模块

1. 新建登录页- pages/Login/Login



Har包basic

2. 在广告页跳转前检查token，有的话跳转到主页 Index，没有的话跳转到Login

- 声明一个token_key的常量-constants/user.ets

```
1 export const TOKEN_KEY = 'user_token'
```

Fence 21

- 在constant/index.ts中导出

```
1 export * from './user'
```

Fence 22

这里需要在首选项中在声明两个方法

```
1 /** 持久化存token */
2 async setToken(token: string) {
3     const store = this.getStore()
4     store.putSync(TOKEN_KEY, token)
5     await store.flush() // flush是异步的
6 }
7
8 /** 持久化获取token */
9 getToken() {
10     const store = this.getStore()
11     return store.getSync(TOKEN_KEY, '') as string
12 }
```

Fence 23

```

1  async onWindowStageCreate(windowStage: window.WindowStage): Promise<void> {
2      // Main window is created, set main page for this ability
3      hiLog.info(0x0000, 'testTag', '%{public}s', 'Ability
onWindowStageCreate');
4
5
6      // 模拟发请求，查询广告配置
7      const ad = await new Promise<AdvertClass>((resolve) => {
8          setTimeout(() => {
9              resolve(defaultAd) // artTS进阶  resolve 表示成功的返回值
10             }, 500)
11         })
12
13     // 存到用户的持久化存储中
14     const userSetting = new UserSettingClass(this.context)
15     userSetting.setAdvert(ad)
16
17     if(ad.showAd){
18         // 创建子窗口
19         const win = await windowStage.createSubwindow('ad_window')
20         await win.showWindow() // 打开子窗口
21         win.setUIContent('pages/Start/Start') // 显示那个页面的内容做为子窗口的页面
22     }
23
24     windowStage.loadContent('pages/Index');
25
26     const token = await userSetting.getUserToken()
27     if(token) {
28         AppStorage.setOrCreate(TOKEN_KEY, token)
29         windowStage.loadContent('pages/Index');
30     } else {
31         windowStage.loadContent("pages/Login/Login")
32     }
33
34 }

```

Fence 24

:::info

- 提交代码

:::

登录页基本功能实现

Entry模块：登录信息属于业务的数据，所以内容留存在当前的entry模块中



- 拷贝结构

```
1 @Entry
2 @Component
3 struct Login {
4
5     @State showLoading: boolean = false
6
7     @Styles
8     loginStyle() {
9         .backgroundColor('#fff')
```

```

10     .border({ color: $r('app.color.background_divider'), width: { bottom: 1
    } })
11     .width('100%')
12     .height(58)
13     .borderRadius(0)
14 }
15
16 build() {
17     column() {
18         // 顶部标题
19         Text("小时
    达").fontColor($r('app.color.text_primary')).fontSize(18).height(25)
20         // 账号登录
21         Row() {
22             Text('账号登
    录').fontColor($r('app.color.text_primary')).fontSize(24).fontWeight(Fontwei
    ght.Bold)
23             Row() {
24                 Text("手机号登
    录").fontColor($r('app.color.primary')).fontSize(16).fontWeight(FontWeight.B
    old)
25                 Image($r("app.media.ic_angle")).width(10).height(10).margin({
    left: 5 })
26             }
27         }
28         .width('100%')
29         .justifyContent(FlexAlign.SpaceBetween)
30         .margin({ top: 50, bottom: 50 })
31
32         // 用户名输入框
33         TextInput({ placeholder: '请输入账号' })
34             .loginStyle()
35
36         // 密码框
37         TextInput({ placeholder: '请输入密码' })
38             .loginStyle()
39             .type(InputType.Password) // 密码框
40             .showPasswordIcon(true) // 显示密码按钮
41
42         // 登录按钮
43         Button({ type: ButtonType.Capsule }) {
44             Row() {
45                 if (this.showLoading) {
46                     LoadingProgress().width(20).height(20).margin({ right: 12
    }).color($r('app.color.white'))
47                 }
48                 Text('登录').fontColor($r('app.color.white'))
49             }
50         }
51         .backgroundColor($r('app.color.primary_disabled'))
52         .width('100%')
53         .height(50)
54         .margin({ top: 50 })
55     }
56     .padding({ left: 32, right: 32 })

```

```

57 |     .margin({ top: 40 })
58 |   }
59 | }

```

Fence 25

:::info

- 接下来我们来把表单数据进行一下双向绑定-和类型定义
- ...
- 首先在models下新建user.ts文件

这里我们拷贝接口文档中的interface接口声明-要去除默认自带的

```

1 | export interface LoginFormModel {
2 |     /**
3 |      * 登录账号
4 |      */
5 |     account: string;
6 |     /**
7 |      * 登录密码
8 |      */
9 |     password: string;
10 | }

```

Fence 26

- 然后在models/index.ets中统一导出

```

1 | export * from './user'

```

Fence 27

接下来就可以通过 引用models来使用所有模块的类型了

- 在Login页面中导入类型

```

1 |
2 |

```

Fence 28

- 在Login组件中声明State数据

```

1 | @State accountForm: LoginFormModel = {
2 |     account: '',
3 |     password: ''
4 | }

```

Fence 29

- 完成账户和密码的双向绑定

```

1 | // 用户名输入框
2 |     TextInput({ placeholder: '请输入账号', text: this.accountForm.account })
3 |         .loginStyle().onChange(value => {
4 |             this.accountForm.account = value
5 |         })
6 |
7 | // 密码框

```



```

8      TextInput({ placeholder: '请输入密码', text: this.accountForm.password
    })
9      .loginStyle()
10     .type(InputType.Password) // 密码框
11     .showPasswordIcon(true)
12     .onChange((value) => {
13         this.accountForm.password = value
14     }) // 显示密码按钮

```

Fence 30

注意：这里不能使用因为我们只支持单层数据的绑定，出现嵌套的情况还得使用监听方法

- 如果账户名和密码有值，则使按钮可用

```

1 // 通过校验
2 getFormValidate() {
3     if (this.accountForm.password && this.accountForm.account) {
4         return true
5     }
6     return false
7 }

```

Fence 31

- 条件判断使用何种颜色和点击反馈

```

1 // 登录按钮
2 Button({ type: ButtonType.Capsule, stateEffect: this.getFormValidate()
    }) {
3     Row() {
4         if (this.showLoading) {
5             // 显示进度条再显示
6             LoadingProgress()
7                 .width(20)
8                 .height(20)
9                 .margin({ right: 12 })
10                .color($r('app.color.white'))
11        }
12        Text('登录').fontColor($r('app.color.white'))
13    }
14 }
15 .backgroundColor(this.getFormValidate() ? $r('app.color.primary') :
    $r('app.color.primary_disabled'))
16 .width('100%')
17 .height(50)
18 .margin({ top: 50 }).enabled(this.getFormValidate())

```

Fence 32

提交代码

封装统一请求工具request

- 申请基础网络权限-在module.json5中配置

```
1  "requestPermissions": [  
2      {  
3          "name": "ohos.permission.INTERNET",  
4      }  
5  ]
```

Fence 33

Basic模块

- 在constants/index.ets中设置baseUrl基础地址常量

```
1  /** 请求基础地址 */  
2  export const BASE_URL = 'https://slwl-api.itheima.net'
```

Fence 34

- 封装泛型工具-models/index.ets

工具的目的-让返回的结构完成统一

```
1  export interface ResponseData<T> {  
2      code: number  
3      msg: string  
4      data: T  
5  }
```

Fence 35

- 封装一个公共的request方法来支持get/post/put/delete方法

[http官网链接](#)

```
1  import http from '@ohos.net.http'  
2  import { UserSettingClass } from '.'  
3  import { BASE_URL, TOKEN_KEY } from '../constants'  
4  import { promptAction, router } from '@kit.ArkUI'  
5  import { ResponseData } from '../models'  
6  
7  // 任何对象都可以as object  
8  export async function requestHttp<T>(url: string, method: http.RequestMethod  
9  = http.RequestMethod.GET,  
10     data?: object) {  
11     const req = http.createHttp()  
12     let urlStr = BASE_URL + url // 拼接url  
13  
14     // 处理get请求的参数  
15     if (method === http.RequestMethod.GET) {  
16         if (data) {  
17             const xxx = Object.keys(data).map(key =>  
18                 `${key}=${data[key]}`).join('&')  
19             urlStr += `?${xxx}` // get请求时拼接参数到查询参数上 ?a=1&b=2&c=3  
20         }  
21     }
```

```

19     }
20
21     // 组装参数
22     let config: http.HttpRequestOptions = {
23         method: method,
24         extraData: method !== http.RequestMethod.GET ? data : '', // 非get请求不
传data
25         header: {
26             ContentType: 'application/json',
27             Authorization: AppStorage.get(TOKEN_KEY) || '',
28         },
29         readTimeout: 10000, // 如果多少秒没响应就断开
30         expectDataType: http.HttpDataType.OBJECT // 自动将res.result转换为对象
31     }
32
33     try {
34         const res = await req.request(urlStr, config)
35         if (res.responseCode === 401) {
36             promptAction.showToast({ message: '登录失效' })
37             // 删除token - 删两个地址 首选项- 全局状态
38             new UserSettingClass(getContext()).setToken("") // 清空首选项token
39             AppStorage.set(TOKEN_KEY, "") // 只能设置 不能删除
40             router.replaceUrl({
41                 url: 'pages/Login/Login' // 跳转到登录页
42             })
43             return Promise.reject(new Error("登录失效"))
44         }
45
46         if (res.responseCode === 404) {
47             promptAction.showToast({ message: '请求地址错误' })
48             return Promise.reject(new Error("请求地址错误"))
49         }
50
51         const result = res.result as ResponseData<T>
52         if (result.code === 200) {
53             return result.data
54         } else {
55             promptAction.showToast({ message: "接口请求错误" }) // res.msg没处理好 有
可能出来一堆服务器语言
56             console.error(' 接口请求错误-----> ', JSON.stringify(result))
57             return Promise.reject(JSON.stringify(result)) // 业务错误 请求终止
58         }
59     } catch (err) {
60         console.error('err -----> ', err)
61         return Promise.reject(err)
62     }
63 }
64
65 }
66
67
68 export class Request {
69     static get<T = null>(url: string, data?: object): Promise<T> {
70         return requestHttp<T>(url, http.RequestMethod.GET, data)
71     }

```

```

72
73     static post<T = null>(url: string, data?: object): Promise<T> {
74         return requestHttp<T>(url, http.RequestMethod.POST, data)
75     }
76
77     static delete<T = null>(url: string, data?: object): Promise<T> {
78         return requestHttp<T>(url, http.RequestMethod.DELETE, data)
79     }
80
81     static put<T = null>(url: string, data?: object): Promise<T> {
82         return requestHttp<T>(url, http.RequestMethod.PUT, data)
83     }
84 }
85

```

Fence 36

- 在utils中导出

```

1 | export * from './request'

```

Fence 37

:::info

总结：上面请求工具总共处理了这么几件事

- 封装请求-拼接基础地址-传入参数
 - 解构返回的数据-判断状态
 - 200 认为成功直接返回data数据
 - 200以外 401 认为是token超时
 - 500认为是服务器任务
 - 最后暴露一个类的四个静态方法 可以方便的调用 get/put/delete/post
 - 提交代码
- ...

封装请求登录的api

Entry模块

- 在api下新建user.ets，并在index.ets导出

```
1 import { Request } from '@hm/basic'
2 import { LoginFormModel } from '../models'
3
4 export function loginAPI(data: LoginFormModel) {
5     return Request.post<string>('/driver/login/account', data)
6 }
```

Fence 38

- index.ets中导出

```
1 export * from './user'
```

Fence 39

:::info

封装api是为后续更灵活的使用

- Request.post这里的string相当于传入的泛型数据，指代返回的data是个string
 - 提交代码
- ...

登录存储token跳转主页

- 首先大家先通过这个网址

[神领TMS管理系统](#)

- 去注册一个属于自己的账号
- 在登录页引入登录api-存入token-跳转主页
- 导入api

```
1 import router from '@ohos.router'
2 import { login } from '../api'
3 import { TOKEN_KEY, UserSetting } from '@hm/basic'
```

Fence 40

```
1 // 调用登录接口
2 async login() {
3   this.showLoading = true
4   const token = await login(this.accountForm)
5   AppStorage.setOrCreate(TOKEN_KEY, token) // 写入全局状态
6   new UserSettingClass(getContext(this)).setUserToken(token) // 存入首选项
7   this.showLoading = false
8   router.replaceUrl({
9     url: 'pages/Index'
10  })
11 }
```

Fence 41

- 注册事件位置

```
1 Button("登录")...
2   .onClick(() => {
3     this.login()
4   })
```

Fence 42

- 当我们使用键盘回车之后，账号和密码也可以直接提交

```
1 TextInput()
2   ...
3   .onSubmit(() => {
4     if(this.getFormValidate()) {
5       this.login()
6     }
7   })
```

Fence 43

:::info

总结

- 登录需要自己去网站上注册账号-移动端暂时没有注册接口
- 请求的时候需要准确的准备返回的响应体
- 登录成功之后-存入token- 跳转页面

- 这里登录失败为啥没处理，因为在请求模块的位置，我们判断了200，只要是200 我们就终止了请求，不会再往下走啦
 - 最后不要忘记提交代码
- ...

布局首页结构



basic模块

```
1  /** 首页Tab类型 */
2  export interface TabClass {
3      title: string
4      name: string
5      icon?: ResourceStr
6  }
```

Fence 44

entry模块

:::info

- 上图得知，首页底部分为三个区域，任务，消息，我的
- 任务里面有 待提货，在途，已完成
- ...
- 底部为第一级别，上部分内容为第二级别，我们先完成第一级别

首先把我们的首页设置成Index/Index，改成目录结构的嵌套

- 另外需要调整原来路由跳转的地方- Login/EntrtAbility

采用Tabs组件就可以轻松实现

```

1  import { TabClass } from '@hm/basic'
2
3  @Entry
4  @Component
5  struct Index {
6      @State
7      currentIndex: number = 0 // 当前激活项
8      @State tabsData: TabClass[] = [{
9          title: '任务',
10         name: 'task',
11         icon: $r("app.media.ic_tab_btn_task")
12     }, {
13         title: '消息',
14         name: 'message',
15         icon: $r("app.media.ic_tab_btn_mess_nor")
16     }, {
17         title: '我的',
18         name: 'my',
19         icon: $r("app.media.ic_tab_btn_mine_nor")
20     }]
21
22     @Builder
23     getTabBar(item: TabClass) {
24         Column() {
25             Image(item.icon).width(22).height(22)
26             .fillColor(item.name === this.tabsData[this.currentIndex].name ?
27                 $r('app.color.primary') : $r('app.color.text_secondary'))
28             Text(item.title)
29             .fontSize(12)
30             .fontWeight(400)
31             .margin({ top: 5 })
32             .fontColor(item.name === this.tabsData[this.currentIndex].name ?
33                 $r('app.color.primary') : $r('app.color.text_secondary'))
34         }.alignItems(HorizontalAlign.Center)
35     }
36     build() {
37         Tabs({ barPosition: BarPosition.End, index: $$this.currentIndex }) {
38             ForEach(this.tabsData, (item: TabClass) => {
39                 TabContent() {
40                     if(item.name === 'task') {
41                         Text("任务组件")
42                     }
43                     else if(item.name === 'message') {
44                         Text("消息组件")
45                     }

```

```
46         else {  
47             Text("我的组件")  
48         }  
49     }.tabBar(this.getTabBar(item))  
50 })  
51 }  
52 }  
53 }
```

Fence 45

提交代码

我的-组件基本布局

Entry模块



- 页面也可以被当成组件来使用- 既可以被引用-也可以被跳转
- 组件不能当成页面来使用

:::info

注意:

我的是组件而不是页面，因为它属于Tabs中的子组件，所以我们在pages/Index 目录下新建一个My 文件夹

里面新建一个My.ets(组件可以是Page,也可以不是，但是如果是Page的话，需要进行导出)

:::

- 新建pages/Index/My/My.ets文件-结构从静态模板中拷贝

```

1  @Preview
2  @Component
3  struct My {
4      build() {
5          column(){
6              // 基本信息
7              column() {
8                  Image($r("app.media.ic_avatar_driver"))
9                      .width(67)
10                     .height(67)
11                     .borderRadius(34.5)
12                     .backgroundColor($r('app.color.white'))
13                 Text("司机")
14                     .fontSize(18)
15                     .fontWeight(600)
16                     .lineHeight(25)
17                     .margin({
18                         top: 9,
19                         bottom: 9
20                     })
21                     .fontColor($r('app.color.white'))
22                 Text(`司机编号:
66666666`) .fontSize(14).fontWeight(400).lineHeight(20).fontColor($r('app.co
lor.white'))
23                 Text(`手机号: 66666666`)
24                     .fontSize(14)
25                     .fontWeight(400)
26                     .lineHeight(20)
27                     .margin({
28                         top: 10
29                     })
30                     .fontColor($r('app.color.white'))
31             }
32             .backgroundImage($r("app.media.bg_page_my"))
33             .backgroundImageSize(ImageSize.Cover)
34             .width('100%')
35             .alignItems(HorizontalAlign.Center)
36             .justifyContent(FlexAlign.Center)
37             .height(292)
38             .margin({
39                 top: -2
40             })
41
42             // 本月任务
43             column() {
44                 Text("- 本月任务 -
") .fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
45                 Row() {
46                     column() {
47
48                         Text("1000").fontSize(18).fontColor($r('app.color.text_primary')).lineHeig
ht(25).margin({
49                             bottom: 17
50                         })

```

```

50         Text("任务总
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
51     }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
52
53     Column() {
54         Text("1000")
55             .fontSize(18)
56             .fontColor($r('app.color.text_primary'))
57             .lineHeight(25)
58             .margin({ bottom: 17 })
59         Text("完成任务
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
60     }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
61
62     Column() {
63         Text("1000")
64             .fontSize(18)
65             .fontColor($r('app.color.text_primary'))
66             .lineHeight(25)
67             .margin({ bottom: 17 })
68         Text("运输里程
(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
69     }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
70
71     }.justifyContent(FlexAlign.SpaceBetween).width('100%').layoutweight(1)
72     }.backgroundColor($r('app.color.white'))
73     }.borderRadius(8)
74     }.margin({ left: 14.5, right: 14.5, top: -55, bottom: 15 })
75     }.height(134)
76     }.padding({ top: 13.5, bottom: 13.5 })
77     }.justifyContent(FlexAlign.SpaceBetween)
78
79     // 信息列表
80     Column() {
81         Row() {
82             Text("车辆信
息").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
83             Image($r("app.media.ic_btn_more")).width(24).height(24)
84         }
85         .justifyContent(FlexAlign.SpaceBetween)
86         .alignItems(VerticalAlign.Center)
87         .border({ width: { bottom: 1 }, color:
$r('app.color.background_page') })
88         .width('100%')
89         .height(60)
90
91         Row() {
92             Text("任务数
据").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
93             Image($r("app.media.ic_btn_more")).width(24).height(24)
94         }
95         .justifyContent(FlexAlign.SpaceBetween)
96         .alignItems(VerticalAlign.Center)

```

```

97         .border({ width: { bottom: 1 }, color:
    $r('app.color.background_page') })
98         .width('100%')
99         .height(60)
100
101     Row() {
102         Text("系统设
    置").fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
103         Image($r("app.media.ic_btn_more")).width(24).height(24)
104     }
105     .justifyContent(FlexAlign.SpaceBetween)
106     .alignItems(VerticalAlign.Center)
107     .width('100%')
108     .height(60)
109 }
110 .padding({ left: 17.5, right: 17.5 })
111 .backgroundColor($r('app.color.white'))
112 .margin({ left: 14.5, right: 14.5, bottom: 15 })
113 .borderRadius(8)
114
115     }.width('100%').height('100%').backgroundColor($r('app.color.background_pag
    e')).borderRadius(8)
116 }
117 }
118
119 export default My

```

Fence 46

这里用了默认导出 `export default My` 为什么不用按需导出啦，按需导出针对的是我们的组件库及一些公用的方法和组件，因为不确定一个组件可能会导出哪些内容，可能一个可能多个，但是这里 `My` 属于业务组件，只用于完成这个业务，所以这里用了默认导出

- 放置在Index组件中的**我的位置**

```

ForEach(this.tabsData, (item: TabClass) => {
    TabContent(){
        if(item.name === 'task') {
            Text("任务组件")
        }
        else if(item.name === 'message') {
            Text("消息组件")
        }
        else {
            My()
        }
    }.tabBar(this.getTabBar(item))
})

```

黑马程序员



好像长得一样

黑马程序员

:::info

- 大家拷贝过程中发现了什么，就是我们车辆信息，任务数据，系统设置好像结构基本都一样，但是这样代码可读性确实有点低，怎么办？
 - 封装组件库是一个好的办法
 - 大家先把代码提交，接下来，我们来抽提两个组件 HmCard和HmCardItem
- :::

抽提HmCard和HmCardItem组件

Basic模块

info

- 抽提组件的原因是为了复用，小时达中有很多类似的需求设计如下图



黑马程序员

黑马程序员



综上，我们需要一个卡片的容器，和一个左右显示内容的组件，并且容器中还可以放置其他组件

- 所以我们首先封装一个卡片HmCard- components/HmCard.ets

```
1  @Component
2  export struct HmCard {
3      @BuilderParam
4      HmCardFn: () => void
5
6      build() {
7          column() {
8              column() {
9                  if(this.HmCardFn) {
10                     this.HmCardFn()
11                 }
12             }
13         }.backgroundColor($r('app.color.white')).borderRadius(10).width('100%').padding({
14             left: 15,
15             right: 15
16         }).backgroundColor($r('app.color.white'))
17     }.width('100%').padding(15)
18 }
19 }
```

Fence 47

- 在compoments/index.ets中导出

```
1  export * from './HmCard'
```

Fence 48

:::info

- HmCard组件中，我们定义了一个BuilderParam的函数（插槽），通过传入的方式传入内容
:::
- 接下来我们封装HmCardItem.ets组件，在HmCard同级新建HmCardItem组件

需求

:::info

1. 能够两头对齐
2. 右侧能够显示向右的图标，也可以不显示
3. 右侧还可以显示内容-并且应该是可变得

4. 右侧的内容可以触发点击事件，并且在使用它时可以监听到

5. 可以控制是否显示下边框

...

HmCardItem.ets代码实现

```
1  @Component
2  struct HmCardItem {
3
4      leftText: string = ''
5      @Prop rightText: string
6
7      showBottomBorder: boolean = true
8      showRightIcon: boolean = true
9
10     onRightClick: () => void = () => {}
11
12
13     build() {
14         Row() {
15
16             Text(this.leftText).fontSize(16).fontWeight(400).fontColor($r('app.color.text_primary'))
17             Row() {
18                 if (this.rightText) {
19
20                     Text(this.rightText).fontColor($r("app.color.text_secondary")).fontWeight(400).fontSize(14)
21                 }
22                 if (this.showRightIcon) {
23                     Image($r("app.media.ic_btn_more")).width(24).height(24)
24                 }
25                 .onClick(() => {
26                     this.onRightClick()
27                 })
28             }
29             .justifyContent(FlexAlign.SpaceBetween)
30             .alignItems(VerticalAlign.Center)
31             .border({
32                 width: {
33                     bottom: this.showBottomBorder ? 1 : 0
34                 },
35                 color: $r('app.color.background_divider')
36             })
37             .width('100%')
38             .height(60)
39         }
40     }
41
42     export { HmCardItem }
```

Fence 49

- 在components/index.ets导出

```
1 export * from './HmCard'
2 export * from './HmCardItem'
```

Fence 50

在basic/Index.ets中导出

```
1 export * from './src/main/ets/components'
```

Fence 51

Entry模块

- 引入组件

```
1 import { HmCard, HmCardItem } from '@hm/basic'
```

Fence 52

- 接下来使用HmCard和HmCardItem去替换我的页面的三个内容吧

```
1 HmCard() {
2     HmCardItem({ leftText: '车辆信息', rightText: '' })
3     HmCardItem({ leftText: '任务设置', rightText: '' })
4     HmCardItem({ leftText: '系统设置', rightText: '', showBottomBorder: false
5 })
6 }
```

Fence 53

一模一样的，Nice！



:::info

总结

- 封装了HmCard和HmCardItem组件，利用插槽技术把CardItem放入Card中
 - CardItem设置了几个属性 如 显示图标 右侧文本(Prop) 显示底边框
 - 提交代码
- :::

获取我的个人信息

之前的遗留问题



info

标准流程

- 定义接口数据结构
- 封装api
- 组件中定义响应式数据
- 封装方法调用api获取数据赋值给响应式数据
- 在aboutToAppear中调用方法
- 将响应式数据更换到视图上换成真是的
- ...

1. 找到[获取个人信息](#)的接口文档

HTTP 状态码: 200		内容格式: JSON	
数据结构		<> 生成代码	示例
<code>code</code> integer	状态编码: 200-成功, 非200 -> 失败	必需	{ "code" "data" "ava" "nam" "num" "pho" }, "msg": }
data	object {4} 响应数据	必需	
avatar	string 头像	必需	
name	string 姓名	必需	
number	string 司机编号	必需	
phone	string 手机号	必需	
msg	string 提示信息	必需	

修改时间 6 个月前

黑马程序员

- 定义个人信息接口

```

1  export interface UserInfoModel {
2      /** 头像 */
3      avatar: string;
4
5      /** 姓名 */
6      name: string;
7
8      /** 司机编号 */
9      number: string;
10
11     /** 手机号 */
12     phone: string;
13 }

```

Fence 54

2. 在api/user.ts中封装获取用户信息的接口

```

1  import { LoginFormModel, UserInfoModel } from '../models'
2
3  // 获取用户信息
4  export const getUserInfoAPI = () => {
5      return Request.get<UserInfoModel>("/drive/users")
6  }

```

Fence 55

3. 在My页面中导入类型并声明State数据

```

1  import { UserInfoModel } from '../../models'
2  struct My {
3      @State
4      userInfo: UserInfoModel = {} as UserInfoModel
5      ...
6  }

```

Fence 56

4. 封装一个方法

```

1  async getUserInfo() {
2      this.userInfo = await getUserInfoAPI()
3  }

```

Fence 57

5. 在父组件通过Provide传入一个当前name

```

1  @State
2  @Watch("updateName")
3  currentIndex: number = 0 // 当前激活项
4
5  @Provide
6  currentName: string = ""
7
8  updateName() {
9      this.currentName = this.tabsData[this.currentIndex].name
10 }

```

6. 在子组件监听数据变化，如果是当前的tab则获取个人信息

```
1  @Consume
2  @Watch("getUserInfo")
3  currentName: string
4  async getUserInfo() {
5    if (this.currentName === 'my') {
6      this.userInfo = await getUserInfoAPI()
7    }
8  }
```

Fence 59

7. 更新数据

```
Column() {
  Image(this.userInfo.avatar || $r("app.media.ic_avatar_driver"))
    .width(67)
    .height(67)
    .borderRadius(34.5)
    .backgroundColor($r("#FFFFFF"))
  Text(this.userInfo.name)
    .fontSize(18)
    .fontWeight(600)
    .lineHeight(25)
    .margin({
      top: 9,
      bottom: 9
    })
    .fontColor($r("#FFFFFF"))
  Text(`司机编号: ${this.userInfo.number}`)
  Text(`手机号: ${this.userInfo.phone}`)
    .fontSize(14)
    .fontWeight(400)
    .lineHeight(20)
    .margin({
      top: 10
    })
    .fontColor($r("#FFFFFF"))
}
```

黑马程序员

:::info

遵循我们一开始的标准流程后续所有的请求都按照此方式来进行

- 提交代码

...

任务数据获取

上一节如果你能够顺利完成的的话，那么接下来，我们来做一个带参数的查询

获取任务数据接口地址

1. 导入数据类型- entry/models/user_task.ets

导入的数据包括

- 请求参数UserTaskInfoParams
- 响应数据UserTaskInfo

```
1  /** 响应数据 */
2  export interface UserTaskInfoModel {
3      /** 完成任务数量,基于实际完成时间统计 */
4      completedAmounts: number;
5      /** 每日里程,基于实际完成时间统计 */
6      dailyMileage: DailyMileageModel[];
7      /** 任务数量,基于计划完成时间统计 */
8      taskAmounts: number;
9      /** 运输里程, 单位: 公里, 基于实际完成时间统计 */
10     transportMileage: number;
11 }
12
13 export interface DailyMileageModel {
14     /** 日期,格式: 2022-07-16 00:00:00 */
15     dateTime: string | null;
16     /** 里程, 单位: 公里;计算公式: 原始数据 (单位米) /1000 四舍五入取整 */
17     mileage: number | null;
18 }
19
20 export interface UserTaskInfoParamsModel {
21     /** 月 */
22     month: string;
23     /** 年 */
24     year: string;
25 }
```

Fence 60

使用i2c插件自动生成实现类class-具体方式参照上一小节

2. 封装api-user.ets

```
1  import { UserTaskInfoModel, UserTaskInfoParamsModel } from '../models'
2
3  // 获取用户任务数据
4  export const getUserTaskInfoAPI = (data: UserTaskInfoParamsModel) => {
5      return Request.get<UserTaskInfoModel>("/driver/users/taskReport", data)
6  }
```

Fence 61

3. 在my组件中定义数据，封装方法调用


```

1  @State
2  userTaskInfo: UserTaskInfoModel = {} as UserTaskInfoModel
3  queryTaskParams: UserTaskInfoParamsModel = {
4      year: new Date().getFullYear().toString(),
5      month: (new Date().getMonth() + 1).toString()
6  }
7  async getUserInfo() {
8      if(this.currentName === 'my') {
9          this.userInfo = await getUserInfo()
10         this.userTaskInfo = await getUserTaskInfoAPI(this.queryTaskParams)
11     }
12 }

```

Fence 62

```

{"taskAmounts":0,"completedAmounts":0,
 "transportMileage":0,"dailyMileage":[]}

```

黑马程序员

:::info

注意：由于接口的问题，任务数据中的年和月是必填的，而且必须是字符串，否则会报错，请注意

:::

4. 最后一步替换数据

```

Column() {
    Text("- 本月任务 -").fontSize(14).fontColor($r(#818181)).lineHeight(20)
    Row() {
        Column() {
            Text(this.userTaskInfo.taskAmounts?.toString() || '')
                .fontSize(18)
                .fontColor($r(#2A2929))
                .lineHeight(25)
                .margin({ bottom: 17 })
            Text("任务总量").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

        Column() {
            Text(this.userTaskInfo.completedAmounts?.toString() || '')
                .fontSize(18)
                .fontColor($r(#2A2929))
                .lineHeight(25)
                .margin({ bottom: 17 })
            Text("完成任务量").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)

        Column() {
            Text(this.userTaskInfo.transportMileage?.toString() || '')
                .fontSize(18)
                .fontColor($r(#2A2929))
                .lineHeight(25)
                .margin({ bottom: 17 })
            Text("运输里程(km)").fontSize(12).fontColor($r(#2A2929)).lineHeight(17)
        }.justifyContent(FlexAlign.SpaceAround).layoutWeight(1)
    }
}

```

黑马程序员

```

1  // 本月任务
2  column() {

```

```

3      Text("- 本月任务 -
").fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
4      Row() {
5          Column() {
6
7              Text(this.userTaskInfo.taskAmounts?.toString()).fontSize(18).fontColor($r('
app.color.text_primary')).lineHeight(25).margin({
8                  bottom: 17
9              })
10             Text("任务总
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
11             }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
12             Column() {
13                 Text(this.userTaskInfo.completedAmounts?.toString())
14                     .fontSize(18)
15                     .fontColor($r('app.color.text_primary'))
16                     .lineHeight(25)
17                     .margin({ bottom: 17 })
18                 Text("完成任务
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
19                 }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
20             }
21             Column() {
22                 Text(this.userTaskInfo.transportMileage?.toString())
23                     .fontSize(18)
24                     .fontColor($r('app.color.text_primary'))
25                     .lineHeight(25)
26                     .margin({ bottom: 17 })
27                 Text("运输里程
(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
28                 }.justifyContent(FlexAlign.SpaceAround).layoutweight(1)
29             }.justifyContent(FlexAlign.SpaceBetween).width('100%').layoutweight(1)
30         }
31         .backgroundColor($r('app.color.white'))
32         .borderRadius(8)
33         .margin({ left: 14.5, right: 14.5, top: -55, bottom: 15 })
34         .height(134)
35         .padding({ top: 13.5, bottom: 13.5 })
36         .justifyContent(FlexAlign.SpaceBetween)

```

Fence 63

解释:

接口返回的是number类型, 而只有字符串才能显示到文本上, 我们使用了toString(), 只不过你需要 this.TaskInfo.transportMileage?.toString() 并且加上短路表达式来 如果前者为空 则处理成空字符

why?

因为TaskInfo我们定义的是空对象, 直接对undefined的数据进行toString, 会直接空指针异常的

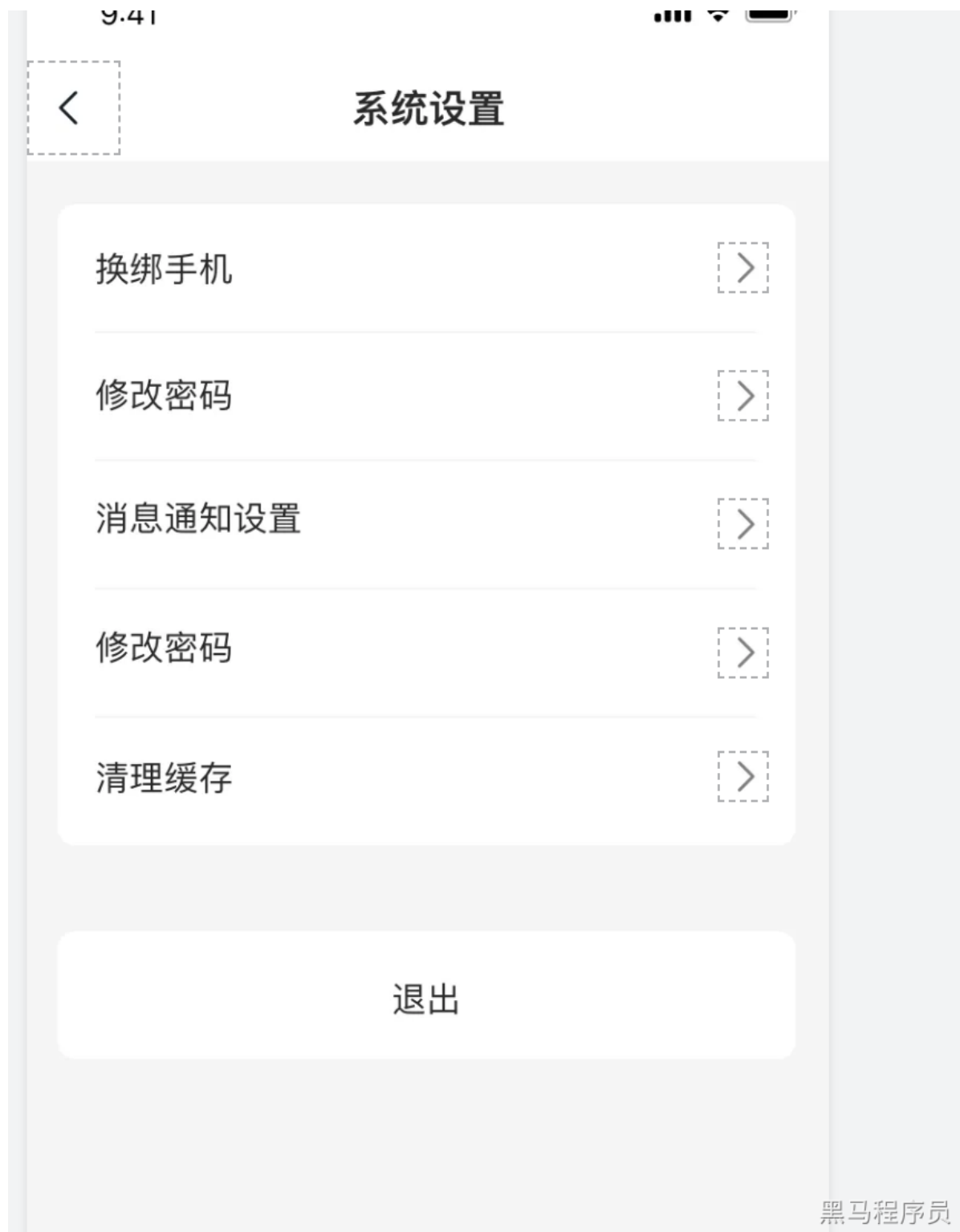
- 最后提交代码

:::info

- 提交代码

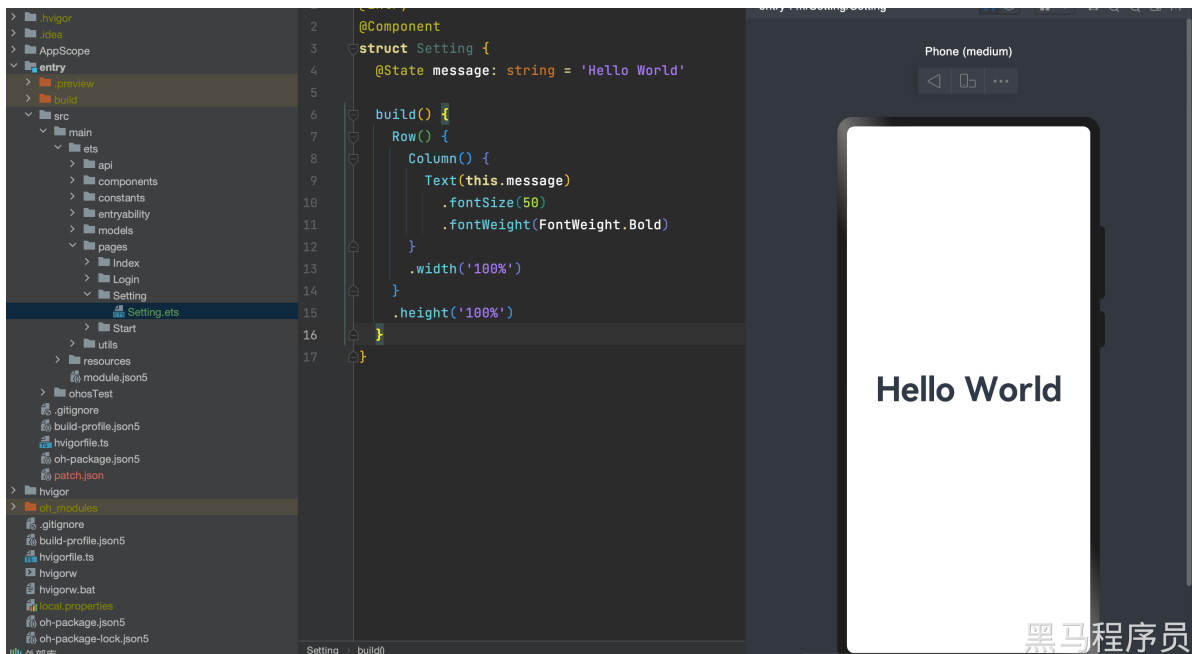
:::

系统设置页面



上面这个页面我们完全可以复用之前的HmCard和HmCardItem，但是顶部的内容还带标题比较通用，所以我们来封装一个新的组件

1. 新建一个Page，因为车辆设置是点击系统设置跳转的页面，所以它是一个页面，不是一个组件
- 在pages下新建Setting/Setting.ets页面

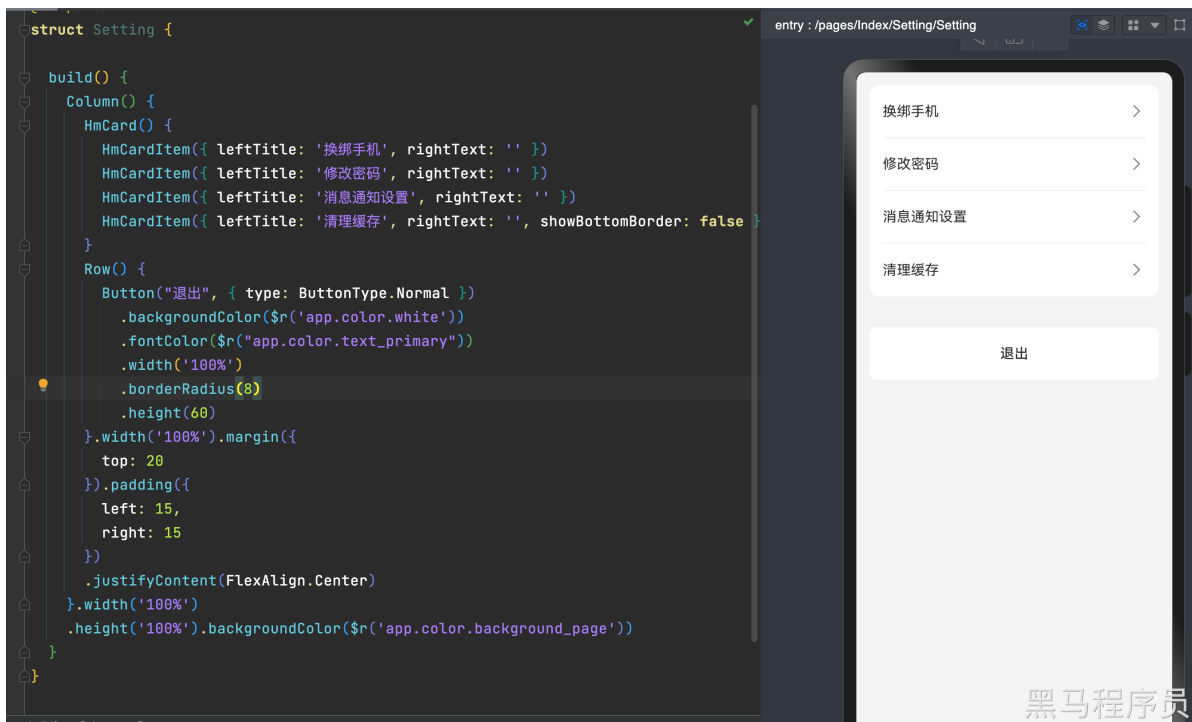


2. 使用之前封装好的组件HmCard HmCardItem 构建主内容区，并放置退出按钮

```

1  import { HmCard, HmCardItem } from '@hm/basic'
2
3  @Entry
4  @Component
5  struct Setting {
6      build() {
7          column() {
8              HmCard() {
9                  HmCardItem({ leftText: '换绑手机', rightText: '' })
10                 HmCardItem({ leftText: '修改密码', rightText: '' })
11                 HmCardItem({ leftText: '消息通知设置', rightText: '' })
12                 HmCardItem({ leftText: '清理缓存', rightText: '', showBottomBorder:
false })
13             }
14
15             Row() {
16                 Button("退出", { type: ButtonType.Normal })
17                     .backgroundColor($r('app.color.white'))
18                     .fontColor($r("app.color.text_primary"))
19                     .width('100%')
20                     .borderRadius(8)
21                     .height(60)
22             }
23             .width('100%')
24             .margin({ top: 20 })
25             .padding({ left: 15, right: 15 })
26             .justifyContent(FlexAlign.Center)
27         }
28         .width('100%')
29         .height('100%')
30         .backgroundColor($r('app.color.background_page'))
31     }
32 }

```



3. 在点击系统设置时，跳转到系统设置页-pages/Index/My/My.ets

```

1   HmCardItem({ leftText: '系统设置', rightText: '', onRightClick: () => {
2       router.pushUrl({
3           url: 'pages/Setting/Setting'
4       })
5   })

```

Fence 65



:::info

总结:

- 封装好的组件用起来很爽
 - 按照业务的需求和UI的设计抽提组件时要考虑复用性，这样就会积累起自己的一套组件库，甚至为开源做贡献
 - 提交代码
- :::

系统设置页面-封装HmNavBar

basic包



系统设置

黑马程序员

:::info

分析需求

- 能够设置中间的标题部分
- 点击左侧按钮能够返回上一层页面

:::



1. 新建components/HmNavBar组件

```
1  import router from '@ohos.router';
2
3  @Preview
4  @Component
5  export struct HmNavBar {
6      title: string = "测试个人中心"
7      showBackIcon: boolean = true
8
9      build() {
10         Stack({ alignContent: Alignment.TopStart }) {
11             Row() {
12                 if (this.title) {
13
14                     Text(this.title).fontColor($r('app.color.text_primary')).fontSize(18).fontWeight(600)
15                 }
16                 .justifyContent(FlexAlign.Center)
17                 .alignItems(VerticalAlign.Center)
18                 .width('100%')
19                 .height('100%')
20
21                 if (this.showBackIcon) {
22                     Row() {
23
24                         Image($r("app.media.ic_btn_nav_back")).width(44).height(44).onClick(() => {
25                             router.back() // 回上一页
26                         })
27                     }
28                 }
29             }
30         }
31     }
32 }
```

```

25         })
26         }.alignItems(VerticalAlign.Center).width(44)
27     }
28 }
29 .backgroundColor($r('app.color.white'))
30 .height(50)
31 .width('100%')
32 .padding(10)
33 }
34 }

```

Fence 66

2. 在components/index.ets中导出

```

1 export * from './HmNavBar'

```

Fence 67

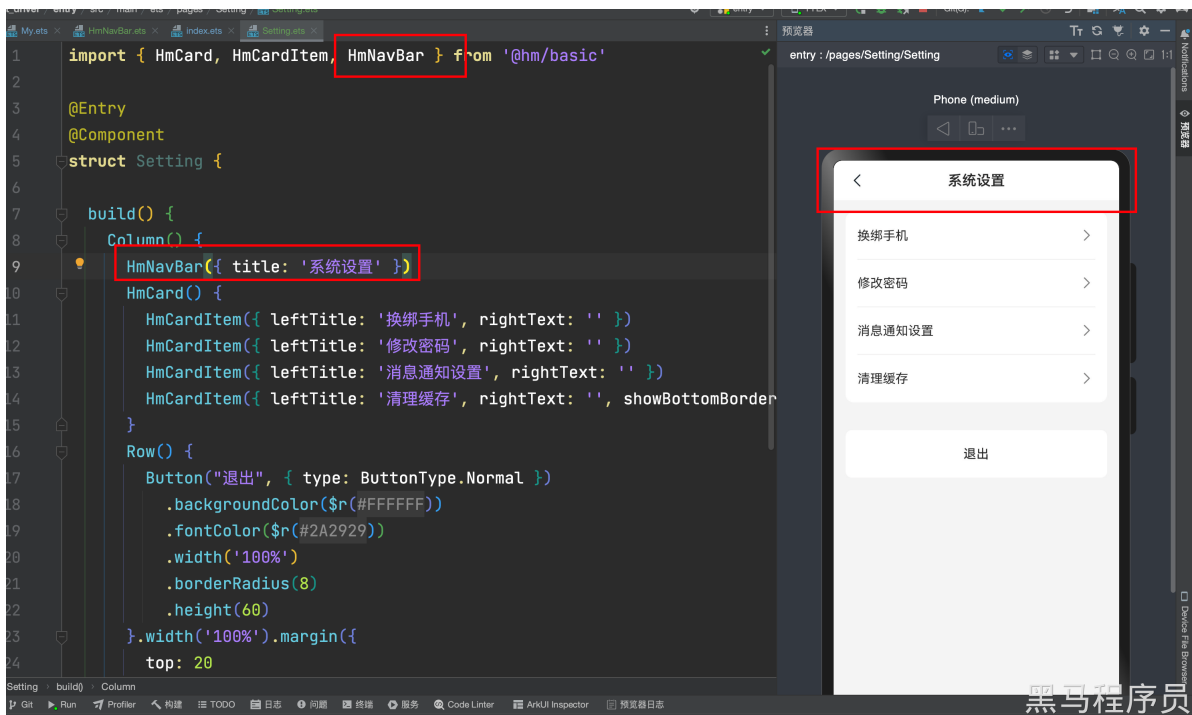
3. 直接在系统设置中引用，放置到最上方即可

```

1 HmNavBar({ title: '系统设置' })
2

```

Fence 68



:::info

总结

- 这里我们的HmNavBar的组件title和showBackIcon属性均没有采用@Link和@Prop，因为它并不需要向响应式更新，这里请知晓，需要的话再去封装对应的需求即可
- 提交代码
- ...

自定义弹窗-实现退出登录功能

Basic



:::danger

按道理说-这是个很简单的需求，但是!!! 目前鸿蒙提供的默认弹窗有点不忍直视，它是长这个样子的



- 差别有点大，为了避免以后我们和UI或者产品开撕的风险，我们还是自己来吧，系统默认的弹窗不好看，不符合要求，我们自己整一个呗，怎么整呢？我们需要使用鸿蒙提供的[自定义弹窗功能](#)
- 现在来浅析一下自定义弹窗的使用方式

1. 使用@CustomDialog装饰器装饰自定义弹窗。
2. @CustomDialog装饰器用于装饰自定义弹框，此装饰器内进行自定义内容（也就是弹框内容）。
3. 创建构造器，与装饰器呼应相连。
4. 点击与onClick事件绑定的组件使弹窗弹出

- 封装一个自定义弹窗组件


```

1  @CustomDialog
2  @Component
3  export struct HmConfirm {
4      // 必不可少的属性 用来控制这个结构的显示和隐藏的
5      controller: CustomDialogController
6      message: string = "确定要退出登录吗? "
7      buttonList: HmConfirmButton[] = [{
8          text: '确定',
9          fontSize: 14,
10         fontColor: $r('app.color.text_secondary')
11     }]
12     build() {
13         Row() {
14             Column() {
15                 Row() {
16                     Text(this.message)
17                         .fontSize(16)
18                         .fontColor($r('app.color.text_primary'))
19                 }
20                 .width('100%')
21                 .height(88)
22                 .justifyContent(FlexAlign.Center)
23                 .border({
24                     color: $r('app.color.background_divider'),
25                     width: {
26                         bottom: 0.5
27                     }
28                 })
29             })
30             Row() {
31                 ForEach(this.buttonList, (item: HmConfirmButton, index: number) => {
32                     Text(item.text)
33                         .fontSize(item.fontSize || 16)
34                         .textAlign(TextAlign.Center)
35                         .fontColor(item.fontColor || $r('app.color.text_secondary'))
36                         .layoutweight(1)
37                         .border({
38                             color: $r('app.color.background_divider'),
39                             width: {
40                                 right: this.buttonList.length > 1 && index !==
this.buttonList.length - 1
41                                     ? 0.5: 0
42                             }
43                         })
44                         .height('100%')
45                         .onClick(async () => {
46                             if(item.action) {
47                                 await item.action()
48                             }
49                             this.controller.close()
50                         })
51                     })
52                 }
53                 .height(49)

```

```

54     .width('100%')
55     }
56     }
57     .width(278)
58     .borderRadius(12)
59     .backgroundColor($r('app.color.white'))
60     }
61 }
62
63
64
65 class HmConfirmButton {
66     fontColor?: ResourceStr = ''
67     fontSize?: number = 16
68     text: string = ''
69     action?: () => void = () => {}
70 }

```

Fence 69

- 统一导出

```

1 export * from './HmConfirm'

```

Fence 70

Entry模块

- 在Setting中声明CustomDialogController

```

1 confirm: CustomDialogController = new CustomDialogController({
2     builder: HmConfirm({
3         message: '确定要退出登录吗?',
4         buttonList: [{
5             text: '取消'
6         }, {
7             text: '确定',
8             fontColor: $r('app.color.primary'),
9             action: () => {
10                 this.logout()
11             }
12         }]
13     }),
14     customStyle: true,
15     alignment: DialogAlignment.Center,
16     autoCancel: false
17 })

```

Fence 71

- 给退出登录按钮注册点击事件

```

1 .onClick(() => {
2     this.confirm.open()
3 })

```

Fence 72

- 实现logout方法

```
1  logout () {  
2      // 删除token  
3      AppStorage.set<string>(TOKEN_KEY, "")  
4      new UserSettingClass(getContext(this)).setUserToken("")  
5      router.replaceUrl({  
6          url: 'pages/Login/Login'  
7      })  
8  }
```

Fence 73



提交代码

封装骨架屏组件

basic模块



Section 1

Section 1.1

Section 1.2

Section 1.3



Section 2

Section 2.1

Section 2.2

Section 2.3



Section 3

Section 3.1

Section 3.2

Section 3.3


```

46         Row()
47             .width("100%")
48             .height(26)
49             .backgroundColor(this.currentColor)
50         Row()
51             .width("50%")
52             .height(26)
53             .backgroundColor(this.currentColor)
54     }
55     .layoutWeight(1)
56     .alignItems(HorizontalAlign.Start)
57
58 }
59 .alignItems(VerticalAlign.Top)
60 .width('100%')
61 }
62 .width('100%')
63
64 }
65
66 build() {
67     Column({ space: 30 }) {
68         ForEach(Array.from(Array(this.count)), () => {
69             this.getSingleItem()
70         })
71     }
72     .alignItems(HorizontalAlign.Start)
73     .padding(20)
74     .width('100%')
75     .height('100%')
76     .backgroundColor($r("app.color.white"))
77 }
78 }
79

```

Fence 74

- 在components/index.ets中导出

```
1 export * from './HmSkeleton'
```

Fence 75

entry模块

- 定义状态判断

```

1 @State
2 loading: boolean = true
3 async getUserInfo() {
4     if(this.currentName === 'my') {
5         this.loading = true
6         this.userInfo = await getUserInfo()
7         this.userTaskInfo = await getUserTaskInfo(this.queryTaskParams)
8         this.loading = false
9     }
10 }

```

```

build() {
  Column(){
    if(this.loading) {
      HmSkeleton({ count: 1 })
    }else {
      // 基本信息
      Column() {...}
      .backgroundImage($r("app.media.bg_page_my"))
      .backgroundImageSize(ImageSize.Cover)
      .width('100%')
      .alignItems(HorizontalAlign.Center)
      .justifyContent(FlexAlign.Center)
      .height(292)
      .margin({
        top: -2
      })
    }
  }
}

// 本月任务

```

黑马程序员

提交代码

设计任务模块tabs

entry模块



Unexpected Text in JSON



任务



消息



我的

黑马程序员

9:41



待提货

在途

已完成

任务编号: XAHH1234567

已延迟

起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00

提货

任务编号: XAHH1234568

起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00-14:00

提货



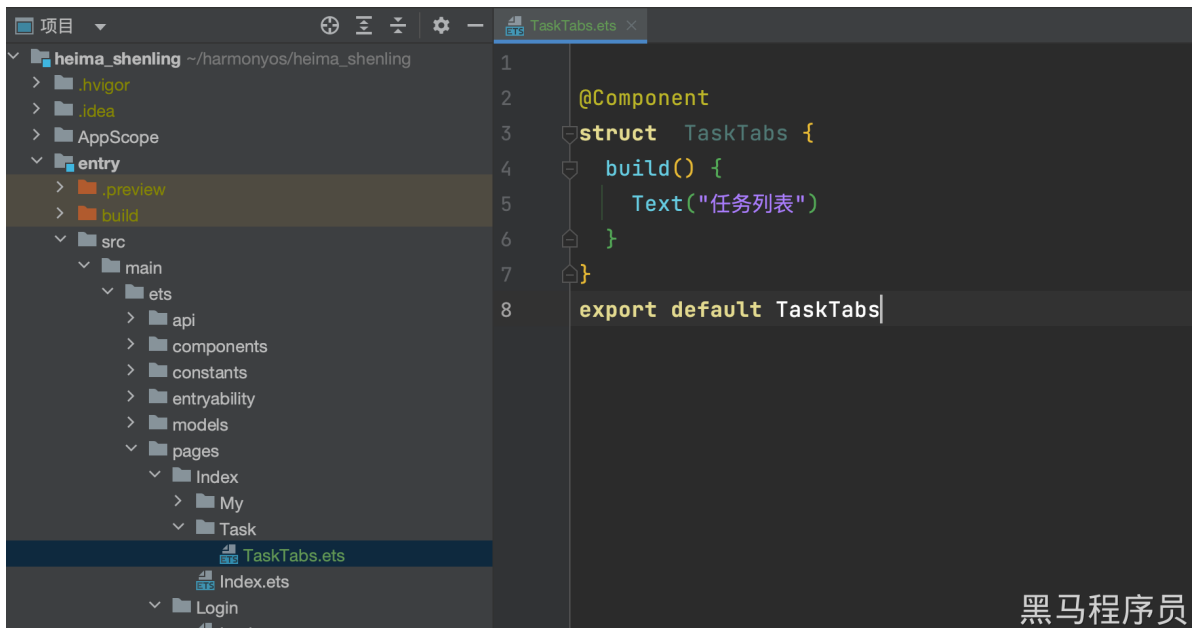
:::info

任务模块分析

- 任务模块又是一个tabs，位置居于上方
- 里面三个待提货 在途 已完成
- 先要完成任务的基本布局，才可以往下推进

:::

1. 新建一个TaskTabs组件-位置 pages/Index/Task/TaskTabs.ets(组件-非Page)



2. 在Index中引入该组件，并放入任务的tab内

```
import ...
```

```
import TaskTabs from '../Task/TaskTab'
```

```
@Entry
```

```
@Component
```

```
struct Index {}
```

```
@State currentName: string = 'task'
```

黑马程序员

```
build() {
```

```
Tabs({ barPosition: BarPosition.End }){
```

```
ForEach(this.tabsData, (item: TabClass) => {
```

```
TabContent(){
```

```
if(item.name === 'task') {
```

```
TaskTabs()
```

```
}
```

```
else if(item.name === 'message') {
```

```
Text("消息组件")
```

```
}
```

```
else {
```

```
My()
```

```
}
```

```
}.tabBar(this.getTabBar(item))
```

```
})
```

```
}
```

```
.onChange(index => {
```

```
this.currentName = this.tabsData[index].name
```

```
})
```

```
.animationDuration(300)
```

```
@Component
```

```
struct TaskTabs {
```

```
build() {
```

```
Text("任务列表")
```

```
}
```

```
}
```

```
export default TaskTabs
```

黑马程序员

entry - images/index/index

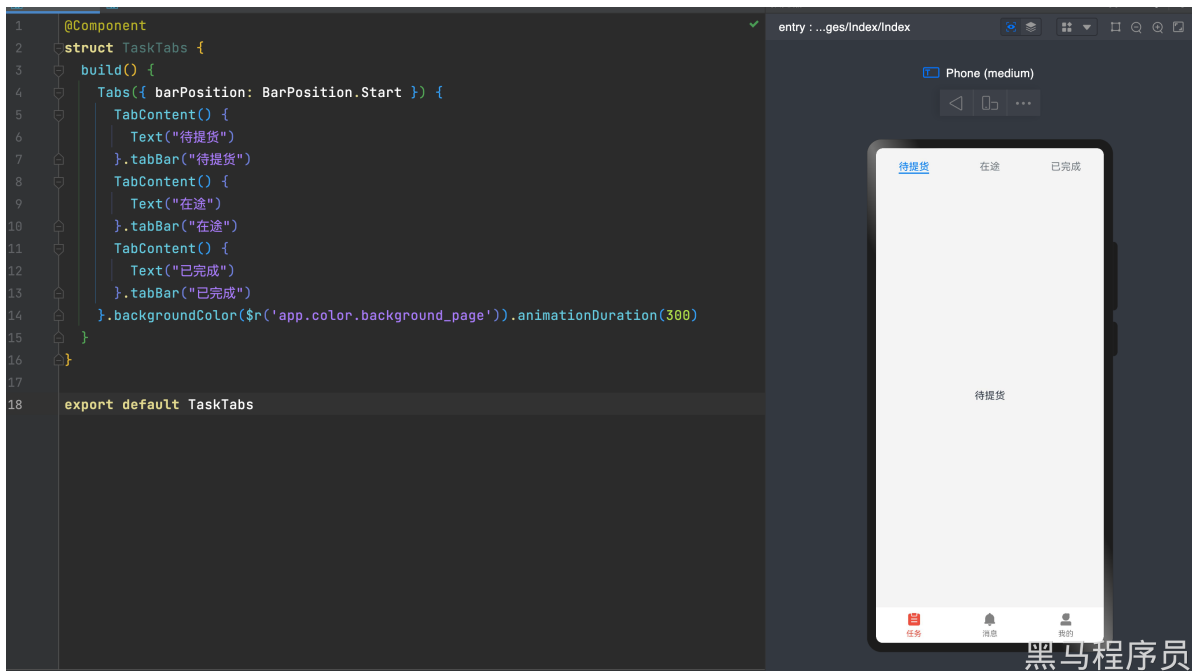
Phone (medium)



任务列表



3. 使用tabs组件构建任务组件的布局和内容



```
1  @Component
2  struct TaskTabs {
3    build() {
4      Tabs({ barPosition: BarPosition.Start }) {
5        TabContent() {
6          Text("待提货")
7        }.tabBar("待提货")
8        TabContent() {
9          Text("在途")
10         }.tabBar("在途")
11        TabContent() {
12          Text("已完成")
13         }.tabBar("已完成")
14      }.backgroundColor($r('app.color.background_page')).animationDuration(300)
15    }
16  }
17
18  export default TaskTabs
```

Fence 77

4. 似曾相识，和首页的Index不是一样吗，我们之前不是设置了一个TabClass吗，正好用上

- 定义state数据

```

1  @State
2  tabsData: TabClass[] = [{
3      name: 'waiting',
4      title: '待提货'
5  }, {
6      name: 'line',
7      title: '在途'
8  }, {
9      name: 'finish',
10     title: '已完成'
11  }]
12 @State
13 currentIndex: number = 0

```

Fence 78

- 完成循环渲染

```

1  build() {
2      Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex }) {
3          ForEach(this.tabsData, (item: TabClass) => {
4              TabContent(){
5                  Text(item.title)
6              }.tabBar(item.title)
7          })
8      }.backgroundColor($r('app.color.background_page')).animationDuration(300)
9  }

```

Fence 79

由于目前Tabs组件不支持设置上面页签的位置和方式，所以我们要采用自定义的方式来实现tabbar的设置

- 用Stack来包裹一下

```

1  build() {
2      Stack({ alignContent: Alignment.Top }) {
3          Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex }) {
4              ForEach(this.tabsData, (item: TabClass) => {
5                  TabContent(){
6                      Text(item.title)
7                  }.tabBar(item.title)
8              })
9          }
10         }.backgroundColor($r('app.color.background_page')).animationDuration(300)
11         Row ({ space: 30 }) {
12             }
13         }.padding({
14             left: 40,
15             right: 40
16         })
17         .width('100%')
18         .height(50)
19         .backgroundColor($r("app.color.white"))
20     }
21 }

```

- 自定义tabbar

```

1  @Builder
2  getTabBar(item: TabClass, index: number) {
3    Column() {
4      Text(item.title)
5        .fontSize(16)
6        .fontColor(this.currentIndex === index ?
7 $r('app.color.text_primary') : $r('app.color.text_secondary'))
8        .fontWeight(600)
9        .animation({
10      duration: 300
11    })
12    .margin({
13      bottom: 10
14    })
15    Divider()
16      .strokeWidth(4)
17      .color($r('app.color.primary'))
18      .lineCap(LineCapStyle.Round)
19      .width(this.currentIndex === index ? 23 : 0)
20      .animation({
21        duration: 300
22      })
23    }
24  }

```

- 定义tabController, 绑定Tabs组件

```

1  tabController: TabsController = new TabsController()
2
3  Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex,
4    controller: this.tabController }) {

```

- 点击时每个tab项时, 切换tab

```

1  .onClick(() => {
2    const index = this.tabsData.findIndex(i => i.name === item.name)
3    this.tabController.changeIndex(index)
4  })

```

- 实际效果



完整代码

```
1  import { TabClass } from '@hm/basic'
2
3  @Component
4  export struct TaskTabs {
5    tabList: TabClass[] = [
6      { name: 'waiting', title: '待提货' },
7      { name: 'line', title: '在途' },
8      { name: 'finish', title: '已完成' } // ctrl + shift + j
9    ]
10   @State currentIndex: number = 0
11   controller = new TabsController()
12
13   @Builder
14   getTabBar(item: TabClass, index: number) {
15     column() {
```



```

16      Text(item.title)
17      .fontSize(16)
18      .fontColor(this.currentIndex === index ?
$r('app.color.text_primary') : $r('app.color.text_secondary'))
19      .fontWeight(600)
20      .animation({
21        duration: 300
22      })
23      .margin({
24        bottom: 10
25      })
26      Divider()
27      .strokeWidth(4)
28      .color($r('app.color.primary'))
29      .lineCap(LineCapStyle.Round)
30      .width(this.currentIndex === index ? 23 : 0)
31      .animation({
32        duration: 300
33      })
34    }
35    .onClick(() => {
36      this.controller.changeIndex(index)
37    })
38  }
39
40  build() {
41    Stack({ alignContent: Alignment.TopStart }) {
42      Tabs({ index: $$this.currentIndex, controller: this.controller }) {
43        ForEach(this.tabList, (item: TabClass) => {
44          TabContent() {
45            Text(item.title)
46          }
47          .tabBar(item.title)
48        })
49      }
50
51      // Row 占满一行，高度跟Tabs的标题栏高度一致
52      Row({ space: 20 }) {
53        ForEach(this.tabList, (item: TabClass, index) => {
54          this.getTabBar(item, index)
55        })
56      }
57      .padding({
58        left: 40,
59        right: 40
60      })
61      .width('100%')
62      .height(50)
63      .backgroundColor($r('app.color.white'))
64    }
65  }
66 }

```

:::info

当无法调整tabbar位置的时候，换个思路，采用自定义的方式来实现

- 提交代码

...

待提货功能分析及数据加载

entry模块

任务编号：XAHH1234567

已延迟

起 北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止 河南省郑州市路北区北清路99号

提货时间
2022.05.04 13:00

提货

任务编号：XAHH1234568

起 北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止 河南省郑州市路北区北清路99号

提货时间
2022.05.04 13:00-14:00

提货

任务编号：XAHH1234569

起 北京市昌平区回龙观街道西三旗桥东金燕龙写
字楼8877号

止 河南省郑州市路北区北清路99号



任务



消息



我的

:::info

分析

- 待提货是一个列表
- 每一项都是单独的一个任务
- 每一项的结构较为个性-而且在其他位置有可能用到-要考虑封装组件
- 有下拉刷新和上拉加载的需求- 难点

该如何下手

- 无外乎就是数据视图的展示
 - 先把提货的组件建好,
 - 把数据加载出来
 - 再去实现后面的内容
- ...

1. 在pages/Index/Task下新建TaskList.ets组件

```
1  @Preview
2  @Component
3  struct TaskList {
4      build() {
5          Text("待提货列表")
6      }
7  }
8  export default TaskList
9
```

Fence 85

2. 在TaskTabs中导入并使用

```
TaskList.ets x TaskTabs.ets x
import TaskList from './TaskList'

}

build() {
  Tabs({ barPosition: BarPosition.Start }) {
    ForEach(this.tabsData, (item: TabClass) => {
      TabContent(){
        if(item.name === 'waiting') {
          TaskList()
        }
      }.tabBar(this.getTabBar(item))
    })
  }
  .backgroundColor($r(#F4F4F4))
  .animationDuration(300)
  .onChange((index: number) => {
    this.currentTabName = this.tabsData[index].name
  })
  .barWidth(200)
}
}
```

黑马程序员

黑马程序员

3. 定义任务数据的类型-[接口文档](#)

新建一个models/task.ts文件，负责管理所有任务数据的类型

步骤

- 拷贝请求参数类型和响应数据
- 这里将响应数据的数据改成TaskListData，将其Item[]改成TaskListItem名称-为了后续更好的辨别

```
1 export interface TaskListParamsModel {
2   /** 结束时间 */
3   endTime: string | null;
4
5   /** 页码 */
6   page: number;
7
8   /** 页面大小 */
9   pageSize: number;
10
11  /** 开始时间 */
12  startTime: string | null;
13
14  /** 作业状态，1为待提货）、2为在途(在途和已交付)、3为改派、5为已作废、6为已完成（已回车
    登记） */
15  status: number;
```

```
16
17     /** 运输任务id */
18     transportTaskId: string | null;
19 }
20
21 /** 响应数据，分页数据统一对象 */
22 export interface TaskListDataModel {
23     /** 总条目数 */
24     counts: number;
25
26     /** 数据列表 */
27     items: TaskInfoItemModel[];
28
29     /** 页码 */
30     page: number;
31
32     /** 总页数 */
33     pages: number;
34
35     /** 页尺寸 */
36     pageSize: number;
37 }
38
39 export interface TaskInfoItemModel {
40     /** 实际到达时间 */
41     actualArrivalTime: string;
42
43     /** 实际发车时间 */
44     actualDepartureTime: string;
45
46     /** 创建时间 */
47     created: string;
48
49     /** 司机id */
50     driverId: string;
51
52     /** 是否可提货 */
53     enablePickUp: boolean;
54
55     /** 目的机构地址 */
56     endAddress: string;
57
58     /** 目的机构id */
59     endAgencyId: number;
60
61     /** 交付对接人 */
62     finishHandover: string;
63
64     /** 司机作业单id */
65     id: string;
66
67     /** 计划到达时间 */
68     planArrivalTime: string;
69
70     /** 计划发车时间 */
```

```

71   planDepartureTime: string;
72
73   /** 起始机构地址 */
74   startAddress: string;
75
76   /** 起始机构id */
77   startAgencyId: number;
78
79   /** 提货对接人 */
80   startHandover: string;
81
82   /** 作业状态, 作业状态, 1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废 */
83   status: TaskTypeEnum;
84
85   /** 运输任务id */
86   transportTaskId: string;
87 }

```

Fence 86

- 声明status类型的枚举

请求类型采用的是一个枚举, 需要使用Enum来声明

```

1  // 查询状态的枚举
2  export enum TaskTypeEnum {
3    waiting = 1,
4    Line = 2,
5    Finish = 6
6  }

```

Fence 87

- 在models/index.ets中导出

```

1  export * from './task'

```

Fence 88

- 封装请求列表api - 新建api/task.ets

```

1  import { TaskListDataModel, TaskListParamsModel } from '../models'
2  import { Request } from '@hm/basic'
3
4  // 获取任务列表
5
6  export const getTaskListAPI = (params: TaskListParamsModel) => {
7    return Request.get<TaskListDataModel>("/driver/tasks/list", params)
8  }

```

Fence 89

- 在api/index.ets中导出

```

1  export * from './task'

```

Fence 90

- 在TaskList中声明数据, 封装方法, 获取数据, 显示获取的数据


```

1  @State
2  queryParams: TaskListParamsModel = {
3      status: TaskTypeEnum.Waiting,
4      page: 1,
5      pageSize: 5,
6
7  } as TaskListParamsModel
8  @State
9  taskListData: TaskInfoItemModel[] = []

```

Fence 91

- 封装方法调用-显示

```

1  import { getTaskListAPI } from '../.../api/task'
2  import { TaskInfoItemModel, TaskListParamsModel, TaskTypeEnum } from
  '../.../models'
3
4
5  @Preview
6  @Component
7  struct TaskList {
8      @State
9      queryParams: TaskListParamsModel = {
10          status: TaskTypeEnum.Waiting,
11          page: 1,
12          pageSize: 5,
13
14      } as TaskListParamsModel
15      @State
16      taskListData: TaskInfoItemModel[] = []
17      aboutToAppear() {
18          this.getTaskList()
19      }
20      async getTaskList() {
21          const result = await getTaskListAPI(this.queryParams)
22          this.taskListData = result.items
23      }
24      build() {
25          Text(JSON.stringify(this.taskListData, null, 2))
26      }
27  }
28  export default TaskList
29

```

Fence 92

如图



:::info

总结

- 标准的流程
- 定义类型
- 封装api
- 调用接口
- 获取数据
- 显示数据-待实现
- 提交代码

...

使用列表循环渲染数据

- 首先需要有一个统一的卡片



- 静态结构-新建一个组件pages/Index/Task/TaskItemCard.ets

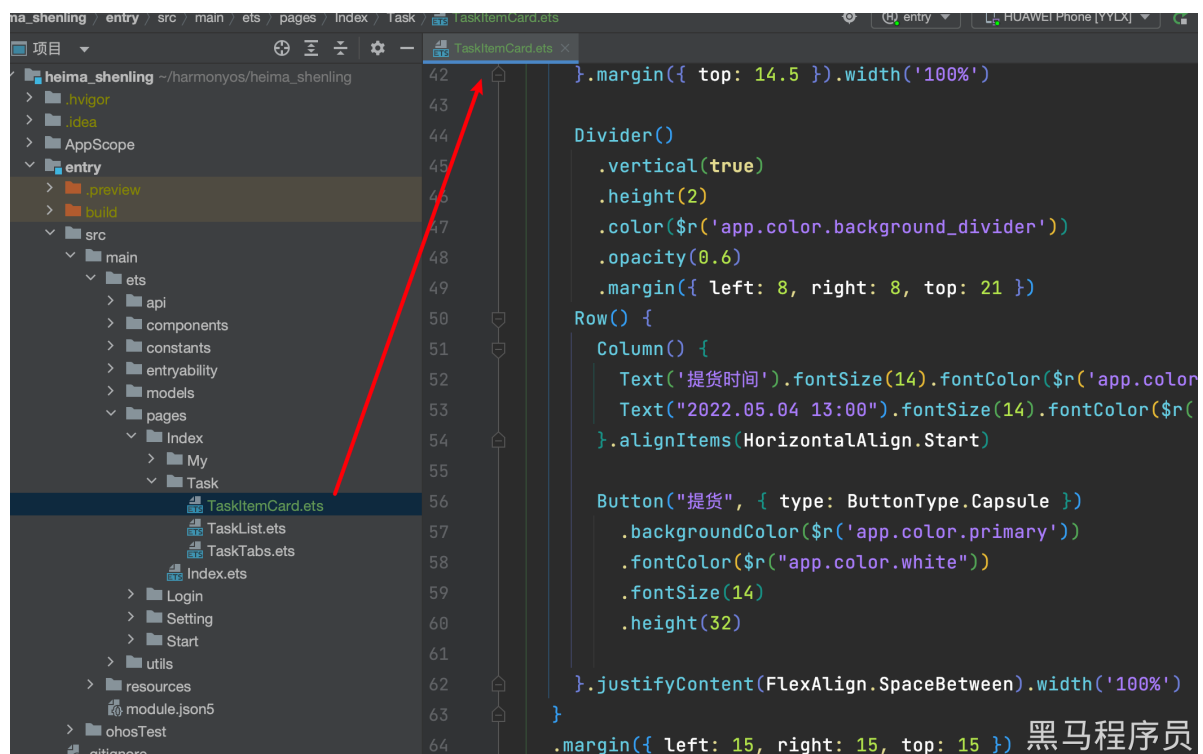
```
1  @Preview
2  @Component
3  struct TaskItemCard {
4      build() {
5          column() {
6              row() {
7                  Text(`任务编号: 213893928399283924`)
8                      .fontSize(16)
9                      .fontColor($r("app.color.text_primary"))
10                     .fontWeight(500)
11                     .lineHeight(22)
12             }.justifyContent(FlexAlign.SpaceBetween).width('100%')
13         }
14         row() {
15             Text("起")
16                 .fontSize(12)
17                 .fontColor($r("app.color.white"))
18                 .backgroundColor($r("app.color.text_primary"))
19                 .width(22)
20                 .height(22)
21                 .borderRadius(11)
22                 .textAlign(TextAlign.Center)
23             Text("北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号")
24                 .margin({ left: 11.5 })
25                 .fontColor($r('app.color.text_secondary'))
26                 .fontSize(14)
```

```

27     }.margin({ top: 21 }).width('100%')
28
29     Row() {
30         Text("止")
31             .fontSize(12)
32             .fontColor($r("app.color.white"))
33             .backgroundColor($r('app.color.primary'))
34             .width(22)
35             .height(22)
36             .borderRadius(11)
37             .textAlign(TextAlign.Center)
38         Text("河南省郑州市路北区北清路99号")
39             .margin({ left: 11.5 })
40             .fontColor($r('app.color.text_secondary'))
41             .fontSize(14)
42     }.margin({ top: 14.5 }).width('100%')
43
44     Divider()
45         .vertical(true)
46         .height(2)
47         .color($r('app.color.background_divider'))
48         .opacity(0.6)
49         .margin({ left: 8, right: 8, top: 21 })
50     Row() {
51         Column() {
52             Text('提货时
间').fontSize(14).fontColor($r('app.color.text_secondary'))
53             Text("2022.05.04
13:00").fontSize(14).fontColor($r('app.color.text_secondary')).margin({ top:
4 })
54         }.alignItems(HorizontalAlign.Start)
55
56         Button("提货", { type: ButtonType.Capsule })
57             .backgroundColor($r('app.color.primary'))
58             .fontColor($r("app.color.white"))
59             .fontSize(14)
60             .height(32)
61
62         }.justifyContent(FlexAlign.SpaceBetween).width('100%')
63     }
64     .margin({ left: 15, right: 15, top: 15 })
65     .padding({ left: 19.5, right: 19.5, bottom: 18.5, top: 18.5 })
66     .borderRadius(10)
67     .backgroundColor($r('app.color.white'))
68
69 }
70 }
71
72 export default TaskItemCard

```

- 封装TaskItemCard组件-位置-pages/Index/Task/TaskItemCard.ets



- 使用List和ListItem配合生成N个TaskItemCard



```
build() {  
  List () {  
    ForEach(this.taskListData, (item: TaskInfoItemModel) => {  
      ListItem(){  
        TaskItemCard()  
      }  
    })  
  }  
}
```

黑马程序员

```
1 build() {  
2     List () {  
3         ForEach(this.taskListData, (item: TaskInfoItemModel) => {  
4             ListItem(){  
5                 TaskItemCard()  
6             }  
7         })  
8     }  
9 }
```

Fence 94

:::info

- 目前实现了基本的首页的显示
 - 利用List和ListItem进行循环渲染-数据是假的-下一步换成真实的
 - 提交代码
- ...

任务卡片数据显示

- 把任务卡片的数据变成真实的

之前在获取任务列表数据时，已经声明了TaskListItem的类型可以直接使用，可以用interface也可以用class

1. 在TaskItemCard中声明一个属性，接收单个任务的数据

- TaskItemCard.ets

```
1 import { TaskInfoItemModel } from '../.../models'
2
3 @Component
4 struct TaskItemCard {
5     @Prop
6     taskItem: TaskInfoItemModel = {} as TaskInfoItemModel
7     ...
8 }
```

Fence 95

- 替换静态数据


```
Text(`任务编号: ${this.taskItemData.transportTaskId}`)
```

黑马程序员

```
fontSize(14)
```

```
Row() {
```

```
Text("止")
```

```
  .fontSize(12)
```

```
  .fontColor($r(#FFFFFF))
```

```
  .backgroundColor($r(#EF4F3F))
```

```
  .width(22)
```

```
  .height(22)
```

```
  .borderRadius(11)
```

```
  .textAlign(TextAlign.Center)
```

```
Text(this.taskItemData.endAddress)
```

```
  .margin({ left: 11.5 })
```

```
  .fontColor($r(#818181))
```

```
  .fontSize(14)
```

```
}.margin({ top: 14.5 }).width('100%')
```

黑马程序员

```
  .margin({ left: 8, right: 8, top: 21 })
```

```
Row() {
```

```
Column() {
```

```
Text('提货时间').fontSize(14).fontColor($r(#818181))
```

```
Text(this.taskItemData.planDepartureTime).fontSize(14)
```

```
  .alignItems(HorizontalAlign.Start)
```

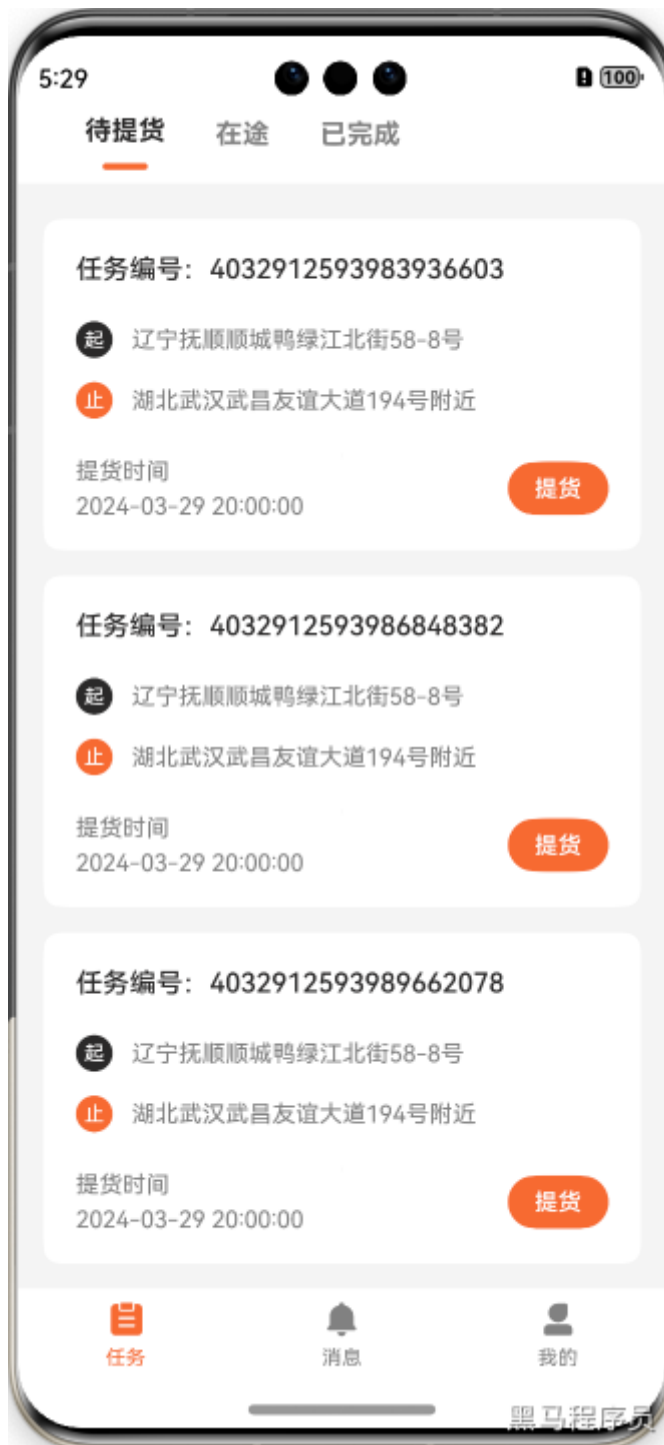
```
Button("提货", { type: ButtonType.Capsule })
```

```
  .backgroundColor($r(#EF4F3F))
```

黑马程序员

- 父组件TaskList传入item数据给TaskItemCard

```
1 build() {  
2   List () {  
3     ForEach(this.taskListData, (item: TaskInfoItemModel) => {  
4       ListItem({  
5         TaskItemCard({ taskItem: item })  
6       })  
7     })  
8   }  
9 }
```



:::info

总结

子组件TaskItemCard中声明数据，接受数据，并没有使用修饰符，因为此数据是只读的，父级数据只会渲染，不会操作发生删除和修改

- 提交代码

...

根据状态控制提货按钮

任务编号: 3111716124860458266

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-11-17 20:00:00

提货

任务编号: 3111716124861814870

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-11-17 20:00:00

提货

黑马程序员

...info

分析

当前司机只能有一个任务，必须完成提货，交货，回车登记才可以继续下一个提货

...

我们的接口中有一个字段可以帮助我们识别该任务用户可以提取

`driverId` string 司机id

`enablePickUp` boolean 是否可提货

`endAddress` string 目的机构地址

`endAgencyId` integer 目的机构id

`finishHandover` string 交付对接人

`id` string 司机作业单id

黑马程序员

我们根据这个字段true/false来控制一下提货按钮的显示颜色

```
1 ...
2
3 Button("提货", { type: ButtonType.Capsule })
4   .backgroundColor(this.taskItem.enablePickUp ?
5 $r('app.color.primary') : $r('app.color.primary_disabled'))
6   .fontColor($r("app.color.white"))
7   .fontSize(14)
8   .height(32)
9   .enabled(this.taskItem.enablePickUp)
```



:::info

总结

根据状态控制按钮颜色

提交代码

:::

封装抽提HmList组件实现上拉加载

basic模块

:::info

HmList组件要完成的功能

- 支持下拉刷新
- 支持上拉加载
- 支持自定义结构
- 支持传入数据渲染内容
- 支持设置加载提示文本
- 支持设置加载完成文本
- 支持控制是否显示加载的进度条

...

结构依然采用之前可以使用的Refresh包裹List的基本架构

- 只不过我们需要向外暴露更多的属性
- finished 是否加载结束（布尔值）
- dataSource 数据源（Link修饰符）
- renderItem 渲染单个Item的(BuilderParam)
- onLoad 执行上拉加载的逻辑(函数)
- onRefresh 执行下拉刷新的逻辑（函数）
- loadingText 加载时的文本（文本）
- finishText 结束时的文本（文本）
- showLoadingIcon 显示加载进度（布尔值）
- refreshIng 控制下拉刷新（State修饰 布尔值）
- loading 控制上拉加载（State布尔值）

在components下新建HmList.ets组件

```
1 import { promptAction } from '@kit.ArkUI'
2
3 @Component
4 struct HmList {
5     @State
6     refreshing: boolean = false // 控制下拉刷新的变量
7     // 传入数据的数组 根据数组进行渲染
8     @Prop
9     dataSource: object[] = [] // 数据源
10
11     // 上拉加载的方法
12     onLoad: () => void = () => {} // 由调用者传入
13
14     // 下拉刷新方法
15     onRefresh: () => void = () => {} // 下拉刷新的方法
16
17     // 需要一个标记 还没有数据的标记
18     @Prop
19     finished: boolean = false // 是否还有下一页数据
20
21     @State
```

```

22     loading: boolean = false // 是否正在加载中 1.显示加载中文本 2. 用来做阀门 当前这次
    请求没结束之前 下次请求滚远点
23
24     loadingText: string = "加载中.." // 加载中的文本
25
26     finishText: string = "没有数据啦" // 所有数据加载完成的文本
27
28     @BuilderParam
29     renderItem: (item: object) => void // 由调用者传入 由HmList调用 传入每一个项的数
    据
30     @Builder
31     getBottomDisplay () {
32         // 获取底部的展示内容
33         Row({ space: 10 }) {
34             if(this.finished) {
35                 // 此时应该没有动画的loading
36                 Text(this.finishText)
37                     .fontSize(14)
38                     .fontColor($r("app.color.text_secondary"))
39             }else {
40                 Text(this.loadingText)
41                     .fontSize(14)
42                     .fontColor($r("app.color.text_secondary"))
43                 LoadingProgress()
44                     .width(20)
45                     .aspectRatio(1)
46                     .color($r("app.color.text_secondary"))
47             }
48         }
49         .width('100%')
50         .height(50)
51         .justifyContent(FlexAlign.Center)
52     }
53     build() {
54         Refresh({ refreshing: $$this.refreshing }) {
55             List() {
56                 ForEach(this.dataSource, (item: object) => {
57                     // 每一项的结构的UI内容不是由列表决定 而由使用者决定
58                     // 传入builderParam
59                     if(this.renderItem) {
60                         this.renderItem(item)
61                     }
62                 })
63                 // 最后放置提示文本的地方
64                 ListItem () {
65                     this.getBottomDisplay()
66                 }
67             }
68             .onReachEnd(async () => {
69                 // 实现上拉加载
70                 // 需要一个标记 是否已经加载完所有数据
71                 // 在没有加载完所有页数据的情况 且没有请求在进程中的情况下
72                 if(!this.finished && !this.loading) {
73                     this.loading = true // 关闭阀门
74                     await this.onLoad() // 实现上拉加载

```

```

75         this.loading = false // 打开阀门
76     }
77 })
78
79 }
80 .onStateChange(async (state) => {
81     // 实现下拉刷新
82     if(state === RefreshStatus.Refresh) {
83         // 松手加载
84         await this.onRefresh() // 调用刷新方法
85         this.refreshing = false // 关闭下拉的动画效果
86         this.loading = false // 关闭上拉刷新的loading
87         // 下拉刷新意味着所有数据全都不要 重新来过
88     }
89 })
90 })
91
92 }
93 }
94 export { HmList }

```

Fence 98

- 在components/index.ets中导出

```
1 export * from './HmList'
```

Fence 99

Entry模块

- 在TaskList中应用组件，实现加载下一页

```

1 import { HmList } from '@hm/basic/Index'
2 import { getTaskList } from '../../api'
3 import { TaskInfoItem, TaskInfoItemModel, TaskListParams,
4 TaskListParamsModel, TaskTypeEnum } from '../../models'
5
6 // 待提货
7 @Component
8 struct TaskList {
9     @State
10     queryParams: TaskListParamsModel = new TaskListParamsModel(
11         {
12             status: TaskTypeEnum.Waiting, // 待提货的类型
13             page: 1, // 第几页
14             pageSize: 5 // 每页几条数据
15         } as TaskListParams
16     )
17     @State
18     taskListData: TaskInfoItem[] = []
19     @State
20     allPage: number = 1 // 默认只有一页
21
22     async getTaskList() {
23         const result = await getTaskList(this.queryParams)

```



```

24 // 追加数据
25 // this.taskListData = this.taskListData.concat(result.items) // 拿到返回
    的数组
26 this.taskListData.push(...result.items) // 延展运算符的写法
27 this.allPage = result.pages // 总页数
28 this.queryParams.page++ // 下次请求的页码
29 }
30 @Builder
31 renderItem (item: object) {
32     TaskItemCard({ taskItem: item as TaskInfoItemModel })
33 }
34 build() {
35     HmList({
36         dataSource: this.taskListData, // 数据源
37         finished: this.allPage < this.queryParams.page , // 是否还有下一页
38         // 上拉加载的函数
39         onLoad: async () => {
40             // 上拉加载
41             await this.getTaskList()
42         },
43         renderItem: (item: object) => {
44             // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
45             this.renderItem(item)
46         },
47         loadingText: '拼命加载中',
48         finishText: '没啦没啦'
49     })
50 }
51 }
52
53 export default TaskList

```

Fence 100

提交代码

实现下拉刷新

- 给HmList传入onRefresh函数

```
1 HmList({
2     onLoad: async () => {
3         await this.getTaskList(true)
4     },
5     onRefresh: async () => {
6         await this.onRefresh()
7     },
8     dataSource: $taskListData,
9     renderItem: this.renderItem,
10    finished: this.allPage < this.queryParams.page,
11    finishText: '没啦没啦',
12    loadingText: '拼命加载中'
13 })
```

Fence 101

- 实现onRefresh函数

```
1 // 下拉刷新函数
2 async onRefresh() {
3     // 重新请求第一页数据
4     this.queryParams.page = 1 // 重置第一页
5     await this.getTaskList(false) // 直接赋值
6 }
```

Fence 102

- 改造getTaskList方法

```
1 async getTaskList(append: boolean) {
2     const result = await getTaskList(this.queryParams)
3     if(append) {
4         this.taskListData.push(...result.items)
5     }else {
6         this.taskListData = result.items
7     }
8
9     this.allPage = result.pages
10    this.queryParams.page++
11 }
```

Fence 103

- 完整代码

```
1 import { HmList } from '@hm/basic/Index'
2 import { getTaskList } from '../api'
3 import { TaskInfoItem, TaskInfoItemModel, TaskListParams,
4 TaskListParamsModel, TaskTypeEnum } from '../models'
5 import TaskItemCard from './TaskItemCard'
6 import { promptAction } from '@kit.ArkUI'
```

```

7 // 待提货
8 @Component
9 struct TaskList {
10   @State
11   queryParams: TaskListParamsModel = new TaskListParamsModel(
12     {
13       status: TaskTypeEnum.waiting, // 待提货的类型
14       page: 1, // 第几页
15       pageSize: 5 // 每页几条数据
16     } as TaskListParams
17   )
18   @State
19   taskListData: TaskInfoItem[] = []
20   @State
21   allPage: number = 1 // 默认只有一页
22
23   async getTaskList(append: boolean) {
24     const result = await getTaskList(this.queryParams)
25     // 追加数据
26     // this.taskListData = this.taskListData.concat(result.items) // 拿到返回
    的数组
27     if (append) {
28       this.taskListData.push(...result.items) // 延展运算符的写法
29     } else {
30       this.taskListData = result.items // 直接赋值
31     }
32
33     this.allPage = result.pages // 总页数
34     this.queryParams.page++ // 下次请求的页码
35   }
36
37   @Builder
38   renderItem(item: object) {
39     TaskItemCard({ taskItem: item as TaskInfoItemModel })
40   }
41
42   // 下拉刷新函数
43   async onRefresh() {
44     // 重新请求第一页数据
45     this.queryParams.page = 1 // 重置第一页
46     await this.getTaskList(false) // 直接赋值
47   }
48
49   build() {
50     HmList({
51       dataSource: this.taskListData, // 数据源
52       finished: this.allPage < this.queryParams.page, // 是否还有下一页
53       // 上拉加载的函数
54       onLoad: async () => {
55         // 上拉加载
56         await this.getTaskList(true) // 追加逻辑
57       },
58       onRefresh: async () => {
59         // 下拉刷新
60         await this.onRefresh()

```

```

61     },
62     renderItem: (item: object) => {
63         // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
64         this.renderItem(item)
65     },
66     loadingText: '拼命加载中',
67     finishText: '没啦没啦'
68 })
69 }
70 }
71
72 export default TaskList

```

Fence 104

:::info

总结

下拉刷新要保证能够发出请求，所以设置allPage为1，查询页码为1

下拉刷新时覆盖数据，上拉加载是追加数据

提交代码

:::

改造下拉刷新的样式

basic模块

builder ¹⁰⁺	CustomBuilder	否	下拉时，自定义刷新样式的组件。 说明： API version 10及之前版本，自定义组件的高度限制在64vp之内。API version 11及以后版本没有此限制
------------------------	---------------	---	--

Refresh组件支持自定义构建样式

- 声明变量记录当前下拉状态

```
1  @State
2  refreshStatus: RefreshStatus = RefreshStatus.Inactive
```

Fence 105

- 下拉状态发生变化时赋值状态

```
onStateChange(async value => {
  this.refreshStatus = value
  if (value === RefreshStatus.Refresh) {
    await this.onRefresh()
    this.refreshIng = false
    this.loading = false
  }
})
}
```

- 实现Refresh的builder的函数

```
1  // 动态生成文本
2  getStatusText() {
3    switch (this.refreshStatus) {
4      case RefreshStatus.Inactive:
5        return ""
6      case RefreshStatus.Drag:
7        return "继续下拉"
8      case RefreshStatus.OverDrag:
9        return "松手加载"
10     case RefreshStatus.Refresh:
11       return "加载中"
12     }
13   }
14   return ""
15 }
```

```

16  @Builder
17  getRefreshDisplay() {
18    Row({ space: 10 }) {
19      LoadingProgress()
20        .color($r('app.color.primary'))
21        .width(40)
22        .height(40)
23      Text(this.getStatusText())
24        .fontColor($r('app.color.text_secondary'))
25        .fontSize(14)
26    }
27    .justifyContent(FlexAlign.Center)
28    .height(50)
29    .width('100%')
30  }

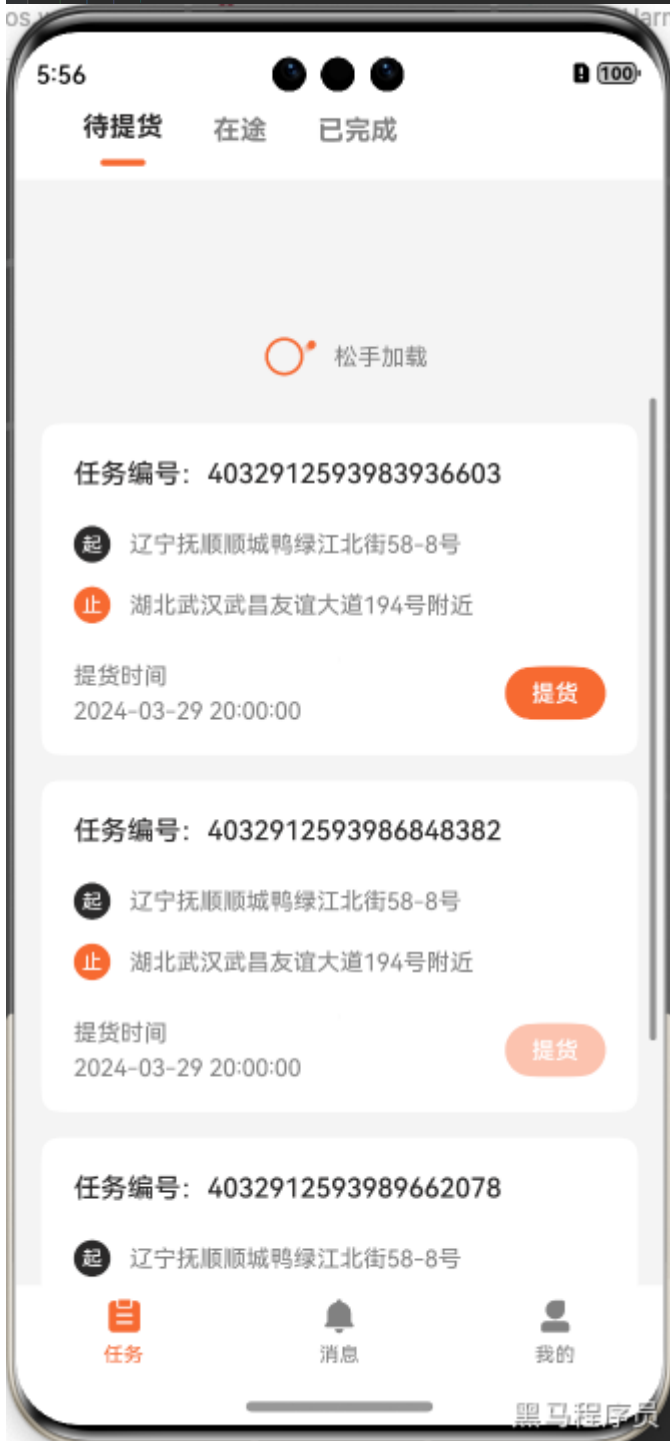
```

Fence 106

- 绑定builder

```
}  
build() {  
  Refresh({ refreshing: $$this.refreshing, builder: this.getRefreshDisPlay }) {  
    List() {  
      ForEach(this.dataSource, (item: object) => {  
        ListItem() {  
          if (this.renderItem) {  
            this.renderItem(item)  
          }  
        }.width('100%')  
      })  
    }  
    ListItem() {  
      this.getBottomDisplay()  
    }  
  }  
}
```

黑马程序员



黑马程序员

提交代码

任务列表跳转到任务详情

entry模块

待提货

在途

已完成

任务编号：XAHH1234567

已延迟

起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00

提货

任务编号：XAHH1234568

起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

提货时间

2022.05.04 13:00-14:00

提货

任务编号：XAHH1234569

起

北京市昌平区回龙观街道西三旗桥东金燕龙写
字楼8877号

止

河南省郑州市路北区北清路99号



任务



消息



我的

黑马程序员

9:41



任务详情

取消任务

基本信息



起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

任务编号

1557211886558850

联系人

张三

联系电话

13512345678



提货时间

2022.05.04 13:00

预计送达时间

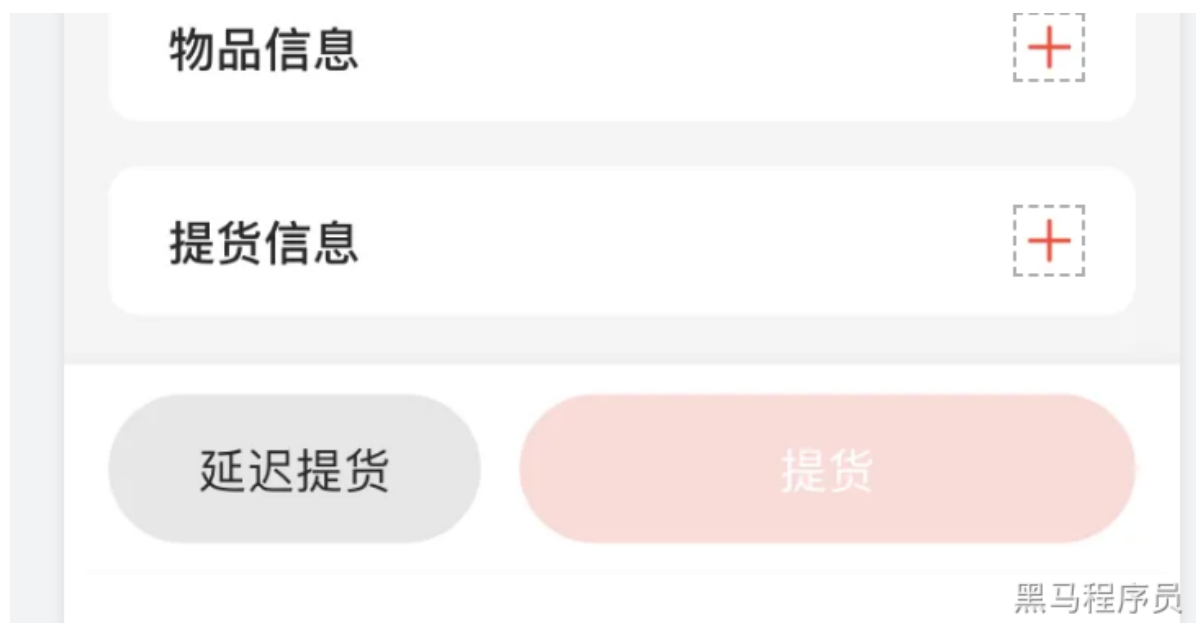
2022.05.05 10:00

车辆司机信息



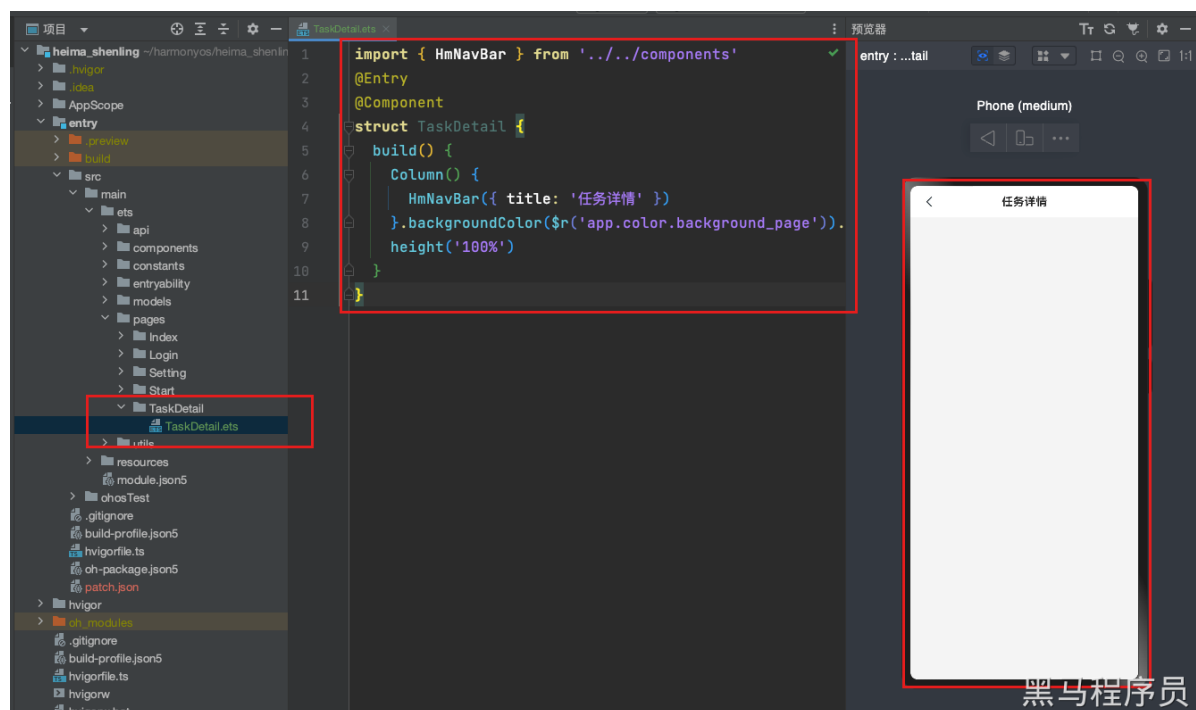
运输路线





:::info
分析

1. 在提货按钮可用情况下，点击提货，跳转到任务详情
2. 通过路由传入任务详情的id
3. 在任务详情接受任务id
- ...
4. 新建一个TaskDetail的页面，因为是列表 => 详情，所以TaskDetail应该是个页面，在pages/TaskDetail/中新建一个TaskDetail.ets



代码

```

1 import { HmNavBar } from '@hm/basic'
2 @Entry
3 @Component
4 struct TaskDetail {
5     build() {
6         Column() {
7             HmNavBar({ title: '任务详情' })
8             .backgroundColor($r('app.color.background_page'))
9             .height('100%')
10        }
11    }

```

Fence 107

2. 在TaskItemCard组件中，监听onClick事件中，通过路由跳转传入任务id到任务详情页

```

1 // 去提货
2 toPickup () {
3     router.pushUrl({
4         url: 'pages/TaskDetail/TaskDetail',
5         params: {
6             id: this.taskItem.id
7         }
8     })
9 }

```

Fence 108

```

Button("提货", { type: ButtonType.Capsule })
    .backgroundColor(this.getBtnEnable() ? $r(#EF4F3F) : $r(#FADCD9))
    .fontColor($r("app.color.white"))
    .fontSize(14)
    .height(32)
    .enabled(this.getBtnEnable())
    .onClick(() => {
        this.toPickup()
    })

```

黑马程序员

3. 在TaskDetail中接收id参数，并显示

basic模块

- 声明一个统一的路由interface来接收Params参数
- models/index.ets

```

1 export interface CommonRouterParams {
2     id?: string
3 }

```

Fence 109

- 在

entry模块

```
1 aboutToAppear() {
2     const params = router.getParams() as CommonRouterParams
3     if(params && params.id) {
4         AlertDialog.show({
5             message: params.id
6         })
7     }
8 }
```

Fence 110

:::info

- 这里特别注意下，大家需要用一個剛注册的司机账号，才可以拥有一个待提货权限，不要和老师同学使用一个账号，这样状态会极其混乱，每个同学请注意自己的司机账号-注册地址-<https://fence-manager.itheima.net/#/driver-register>

:::

6:06

100

待提货

在途

已完成

任务编号: 4032912593983936603

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货

任务编号: 4032912593986848382

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货

任务编号: 4032912593989662078

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2024-03-29 20:00:00

提货



任务



消息



我的

黑马程序员



:::info

总结

通过路由跳转传递id，就可以进行数据查询了

提交代码

:::

根据id获取任务详情数据

:::info

标准流程

- 定义数据类型
- 封装api
- 获取数据
- 赋值数据
- 显示数据
- ...

basic模块

- 定义一个公共的图片类型

```
1 // 后续上传 basic模块也需要类型
2 export interface ImageListModel {
3   /** 图片url */
4   url: string;
5 }
```

Fence 111

entry模块

1. 拷贝接口返回的详情类型 [接口文档](#)

apifox生成的接口类型太多重复，直接拷贝下面的接口

```
1 import { ImageListModel } from '@hm/basic'
2
3 /** 响应数据，响应数据 */
4 export interface TaskDetailInfoModel {
5   /** 实际到达时间 */
6   actualArrivalTime: string;
7
8   /** 实际发车时间 */
9   actualDepartureTime: string;
10
11   /** 提货凭证 */
12   cargoPickUpPictureList: ImageListModel[];
13
14   /** 提货图片 */
15   cargoPictureList: ImageListModel[];
16
17   /** 回单凭证 */
18   certificatePictureList: ImageListModel[];
19
20   /** 回单图片 */
21   deliverPictureList: ImageListModel[];
22
23   /** 司机id */
24   driverId: string;
25 }
```

```
26  /** 司机姓名 */
27  driverName: string;
28
29  /** 目的市 */
30  endAddress: string;
31
32  /** 目的机构id */
33  endAgencyId: string;
34
35  /** 目的机构详细地址 */
36  endCity: string;
37
38  /** 目的省份 */
39  endProvince: string;
40  exceptionList: ExceptionListModel[];
41
42  /** 交付对接人 */
43  finishHandoverName: string;
44
45  /** 交付对接人电话 */
46  finishHandoverPhone: string;
47
48  /** 司机作业单id */
49  id: string;
50
51  /** 车牌号码 */
52  licensePlate: string;
53
54  /** 计划到达时间 */
55  planArrivalTime: string;
56
57  /** 计划发车时间 */
58  planDepartureTime: string;
59
60  /** 起始机构详细地址 */
61  startAddress: string;
62
63  /** 起始机构id */
64  startAgencyId: string;
65
66  /** 起始市 */
67  startCity: string;
68
69  /** 提货对接人 */
70  startHandoverName: string;
71
72  /** 提货对接人电话 */
73  startHandoverPhone: string;
74
75  /** 起始省份 */
76  startProvince: string;
77
78  /** 作业状态, 1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废、6为已完成(已
    回车登记) */
79  status: number;
```



```

80
81     /** 运输任务id */
82     transportTaskId: string;
83 }
84
85
86 export interface ExceptionListModel {
87     /** 异常描述 */
88     exceptionDescribe: string;
89
90     /** 异常图片 */
91     exceptionImagesList: ImageListModel[];
92
93     /** 上报的位置 */
94     exceptionPlace: string;
95
96     /** 异常时间 */
97     exceptionTime: string;
98
99     /** 异常类型(中文) */
100    exceptionType: string;
101
102    /** 处理结果 */
103    handleResult: string;
104 }
105

```

Fence 112

- 统一导出

```

1 | export * from './task_detail'

```

Fence 113

2. 封装请求任务详情api- task.ets

```

1 | import { TaskDetailInfoModel } from '../models'
2
3 | export const getTaskDetailAPI = (id: string) => {
4 |     return Request.get<TaskDetailInfoModel>(`/driver/tasks/details/${id}`)
5 | }

```

Fence 114

3. 在任务详情封装方法，调用，赋值状态显示数据

```

1 | import { HmNavBar, CommonRouterParams } from '@hm/basic'
2 | import { router } from '@kit.ArkUI'
3 | import { TaskDetailInfoModel } from '../models'
4 | import { getTaskDetailAPI } from '../api'
5
6 | @Entry
7 | @Component
8 | struct TaskDetail {
9 |     @State
10 |     taskDetailData: TaskDetailInfoModel = {} as TaskDetailInfoModel
11 |     aboutToAppear() {

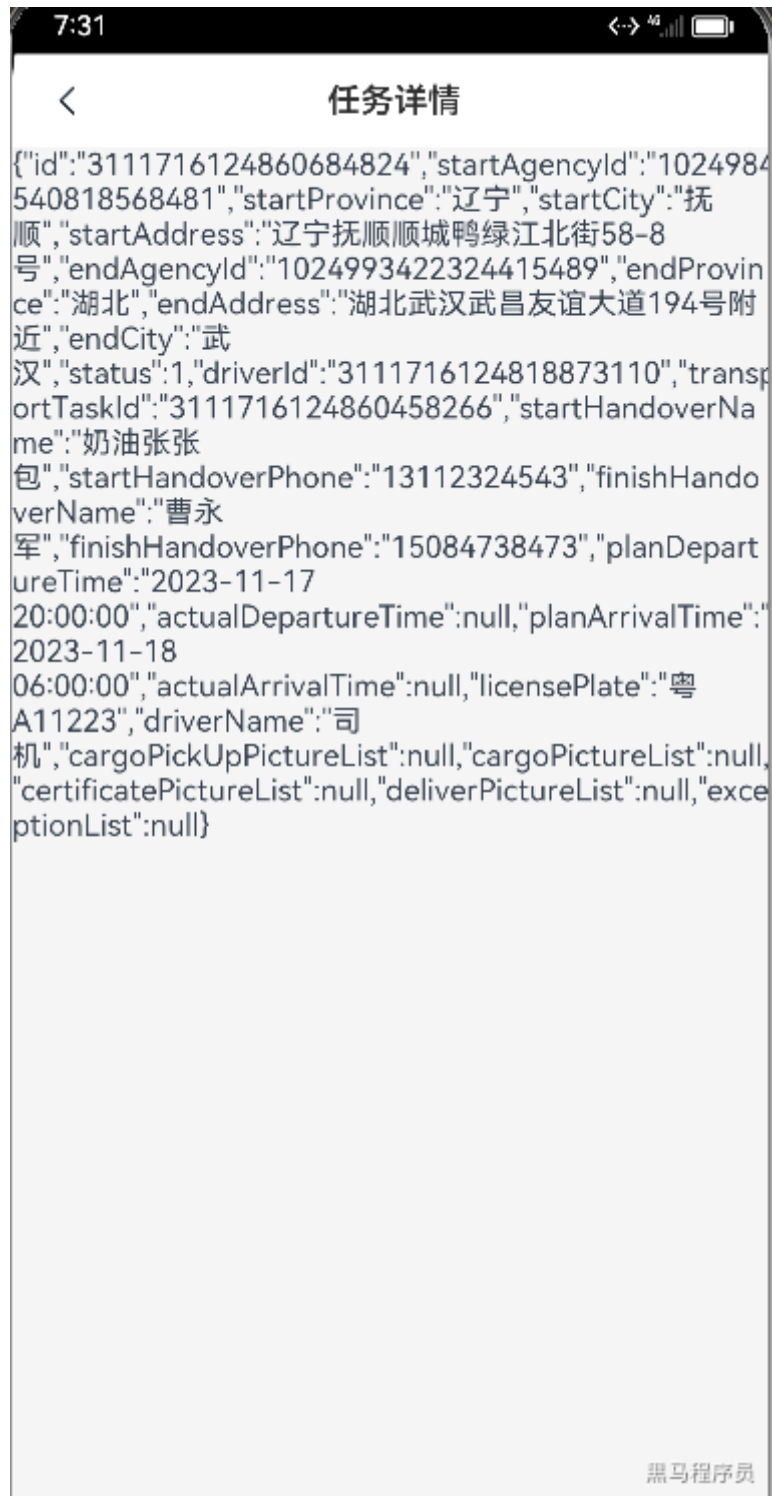
```

```

12     const params = router.getParams() as CommonRouterParams
13     if(params.id) {
14         this.getTaskDetail(params.id)
15     }
16 }
17 async getTaskDetail(id: string) {
18     this.taskDetailData = await getTaskDetailAPI(id)
19 }
20 build() {
21     column() {
22         HmNavBar({ title: '任务详情' })
23         Text(JSON.stringify(this.taskDetailData, null, 2))
24     }.backgroundColor($r('app.color.background_page')).
25     height('100%')
26 }
27 }

```

在页面中显示数据，如图



:::info

提交代码

:::

任务详情结构-封装折叠容器-HmToggleCard

任务详情结构-封装折叠容器



基本信息



起

北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止

河南省郑州市路北区北清路99号

任务编号

1557211886558850

联系人

张三

联系电话

13512345678



提货时间

2022.05.04 13:00

预计送达时间

2022.05.05 10:00

车辆司机信息



运输路线



物品信息



提货信息



延迟提货

提货

basic模块

:::info

- 容器可以展开-可以折叠,
- 可以存放子组件内容
- 可以显示标题

:::

在components下新建 HmToggleCard.ets

```

1  @Component
2  struct HmToggleCard {
3      title: string = "基本信息"
4      @State
5      toggleCard: boolean = true // 是否展示
6      // 尾随闭包
7      @BuilderParam
8      cardContent: () => void
9
10     build() {
11         Column() {
12             Row() {
13                 Text(this.title)
14                     .fontSize(16)
15                     .fontColor($r("app.color.text_primary"))
16                     .fontWeight(500)
17                 Image(this.toggleCard ? $r("app.media.ic_btn_cut") :
$r("app.media.ic_btn_add"))
18                     .width(24)
19                     .aspectRatio(1)
20                     .onClick(() => {
21                         animateTo({ duration: 300 }, () => {
22                             this.toggleCard = !this.toggleCard
23                         })
24                     })
25             })
26         }
27         .width("100%")
28         .height(50)
29         .justifyContent(FlexAlign.SpaceBetween)
30
31         // 外部传入的内容
32         if (this.toggleCard && this.CardContent) {
33             this.CardContent()
34         }
35     }
36     .backgroundColor($r("app.color.white"))
37     .borderRadius(10)
38     .padding({
39         left: 18,
40         right: 18,
41         bottom: 18
42     })

```

```

43     .margin({ left: 10, right: 10, top: 10 })
44
45   }
46 }
47
48 export { HmToggleCard }

```

Fence 116

在index.ts中导出

```

1 export * from './HmToggleCard'

```

Fence 117

entry模块

- 在任务详情中导入测试

```

1 import { HmToggleCard } from '@hm/basic'
2

```

Fence 118

```

build() {
  Column() {
    HmNavBar({ title: '任务详情' })
    HmToggleCard({ title: '基本信息' }) {
      Text(JSON.stringify(this.TaskDetailData))
    }
  }.backgroundColor($r(#F4F4F4)).
  height('100%')
}

```

黑马程序员



:::info
提交代码
:::

使用Scroll组件实现滚动结构

:::info

注意:

Scroll有且只能有一个子组件，不要设置Scroll的高度，让内容去自动撑满

:::

```
1  scroller: Scroller = new Scroller() // 滚动
2
3  build() {
4      Column () {
5          HmNavBar({ title: '任务详情' })
6          Scroll(this.scroller) {
7              Column() {
8                  HmToggleCard({ title: '基本信息' }) {
9                      Text(JSON.stringify(this.taskDetailData))
10                 }
11                 HmToggleCard({ title: '基本信息' }) {
12                     Text(JSON.stringify(this.taskDetailData))
13                 }
14                 HmToggleCard({ title: '基本信息' }) {
15                     Text(JSON.stringify(this.taskDetailData))
16                 }
17             }.padding({
18                 bottom: 20
19             })
20         }
21     }.backgroundColor($r('app.color.background_page')).height('100%')
22 }
```



任务详情

基本信息

```
{
  "id": "3122712195975834073",
  "startAgencyId": "1024984540818568481",
  "startProvince": "辽宁",
  "startCity": "抚顺",
  "startAddress": "辽宁抚顺顺城鸭绿江北街58-8号",
  "endAgencyId": "1024993422324415489",
  "endProvince": "湖北",
  "endAddress": "湖北武汉武昌友谊大道194号附近",
  "endCity": "武汉",
  "status": 1,
  "driverId": "3122712195756902310",
  "transportTaskId": "3122712195975415176",
  "startHandoverName": "奶油张张包",
  "startHandoverPhone": "13112324543",
  "finishHandoverName": "曹永军",
  "finishHandoverPhone": "15084738473",
  "planDepartureTime": "2023-12-27 20:00:00",
  "actualDepartureTime": null,
  "planArrivalTime": "2023-12-28 06:00:00",
  "actualArrivalTime": null,
  "licensePlate": "粤A11223",
  "driverName": "司机",
  "cargoPickUpPictureList": null,
  "cargoPictureList": null,
  "certificatePictureList": null,
  "deliverPictureList": null,
  "exceptionList": null,
  "enablePickUp": true
}
```

基本信息

```
{
  "id": "3122712195975834073",
  "startAgencyId": "1024984540818568481",
  "startProvince": "辽宁",
  "startCity": "抚顺",
  "startAddress": "辽宁抚顺顺城鸭绿江北街58-8号",
  "endAgencyId": "1024993422324415489",
  "endProvince": "湖北",
  "endAddress": "湖北武汉武昌友谊大道194号附近",
  "endCity": "武汉",
  "status": 1,
  "driverId": "3122712195756902310",
  "transportTaskId": "3122712195975415176",
  "startHandoverName": "奶油张张包",
  "startHandoverPhone": "13112324543",
  "finishHandoverName": "曹永军",
  "finishHandoverPhone": "15084738473",
  "planDepartureTime": "2023-12-27 20:00:00",
  "actualDepartureTime": null,
  "planArrivalTime": "2023-12-28 06:00:00",
  "actualArrivalTime": null,
  "licensePlate": "粤A11223",
  "driverName": "司机",
  "cargoPickUpPictureList": null,
  "cargoPictureList": null,
  "certificatePictureList": null,
  "deliverPictureList": null,
  "exceptionList": null,
  "enablePickUp": true
}
```



黑马程序员

:::info

提交代码

:::

任务详情静态-基本信息展示

PushKit

mpass

:::info

准备任务详情的静态结构

基本信息



起 北京市昌平区回龙观街道西三旗桥东金燕龙
写字楼8877号

止 河南省郑州市路北区北清路99号

任务编号 1557211886558850

联系人 张三

联系电话 13512345678 

提货时间 2022.05.04 13:00

预计送达时间 2022.05.05 10:00

黑马程序员

车辆司机信息



车牌号

京A123456

司机姓名

张三

运输路线



北京市
北京市



河南省
郑州市

提货信息



请拍照上传回单凭证



请拍照上传货品照片



黑马程序员

设计图中的货物详情-因接口没有实际数据，所以不在实现。

...

1. 实现基本信息数据展示-拷贝静态结构

```
1  @Extend(Text)
2  function baseTextIconStyle() {
3    .fontSize(12)
4    .fontColor($r('app.color.white'))
5    .backgroundColor($r('app.color.text_primary'))
6    .width(22)
7    .height(22)
8    .borderRadius(11)
9    .textAlign(TextAlign.Center)
10 }
11
12 @Extend(Text)
13 function baseTextStyle() {
14   .margin({ left: 11.5 })
15   .fontColor($r('app.color.text_secondary'))
16   .fontSize(14)
```

```

17     .lineHeight(20)
18 }

```

Fence 120

- 准备一个class提供给builder函数使用

builder里面的数据 如果用传参数的形式- 基础数据不响应 引用数据类型是响应式的
`builder($$: { title: string, name: string })` .Next版本不允许这种声明类型的方式

```

1 interface BaseBuilderClass {
2     title: string
3     value: string
4     icon?: ResourceStr
5 }

```

Fence 121

同学们问：为什么这里不在models中声明了，因为这里只是在我们业务组件内容进行一个简单的使用，不涉及以后的数据通用型，所以在这里使用是可以的

自定义builder

```

1  @Builder
2  getBaseContentItem(item: BaseBuilderClass) {
3      Row() {
4
5          Text(item.title).fontSize(14).fontColor($r('app.color.text_secondary'))
6              .lineHeight(20)
7          Row() {
8              Text(item.value).fontSize(14).fontColor($r('app.color.text_secondary'))
9                  if (item.icon) {
10                      Image(item.icon).width(24).height(24)
11                  }
12              }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
13                  top: 14
14              })
15          }
16
17      // 获取基础信息
18      @Builder
19      getBaseContent() {
20          Row() {
21              Column() {
22                  Row() {
23                      Text("起").baseTextIconStyle()
24                      Text("北京市昌平区回龙观街道西三旗桥东金燕龙写字楼8877号").baseTextStyle()
25                  }.margin({ top: 21 })
26
27                  Row() {
28
29                      Text("止").baseTextIconStyle().backgroundColor($r('app.color.primary'))
30                      Text("河南省郑州市路北区北清路99号").baseTextStyle()
31                  }.margin({ top: 14.5 })
32              }
33          }
34      }
35  }

```

```

32     .alignItems(HorizontalAlign.Start)
33     .layoutWeight(1)
34     .margin({ right: 20 })
35
36     Column() {
37         Image($r("app.media.ic_navigation")).width(22).height(22)
38         Text("开始导航").fontSize(14).margin({ top: 10, bottom: 10 })
39     }.justifyContent(FlexAlign.SpaceBetween)
40     .margin({
41         top: 20
42     })
43
44
45     }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%')
46
47     Divider().vertical(false).height(2).color($r('app.color.background_divider')).margin({ left: 8, right: 8, top: 21 })
48
49     this.getBaseContentItem({
50         title: '任务编号',
51         value: '2132324324343434'
52     })
53     this.getBaseContentItem({
54         title: '联系人',
55         value: '2132324324343434'
56     })
57     this.getBaseContentItem({
58         title: '联系电话',
59         value: '2132324324343434',
60         icon: $r('app.media.ic_phone')
61     })
62     this.getBaseContentItem({
63         title: '提货时间',
64         value: '2132324324343434'
65     })
66     this.getBaseContentItem({
67         title: '预计送达时间',
68         value: '2132324324343434'
69     })
70 }

```

基本信息

 北京市昌平区回龙观街道西三旗桥东
金燕龙写字楼8877号



 河南省郑州市路北区北清路99号

开始导航

任务编号	2132324324343434
联系人	2132324324343434
联系电话	2132324324343434 
提货时间	2132324324343434
预计发货时间	2132324324343434
预计送达时间	2132324324343434

黑马程序员

替换为真实数据

基本信息

 辽宁抚顺顺城鸭绿江北街58-8号



 湖北武汉武昌友谊大道194号附近

开始导航

任务编号	3111716124860458266
联系人	奶油张张包
联系电话	13112324543 
提货时间	2023-11-17 20:00:00
预计送达时间	2023-11-18 06:00:00

黑马程序员

```
1  this.getBaseContentItem({
2      title: '任务编号',
3      value: this.taskDetailData.transportTaskId
4  })
5  this.getBaseContentItem({
6      title: '联系人',
7      value: this.taskDetailData.startHandoverName
8  })
9  this.getBaseContentItem({
10     title: '联系电话',
11     value: this.taskDetailData.startHandoverPhone,
```



```

12         icon: $r('app.media.ic_phone')
13     })
14     this.getBaseContentItem({
15         title: '提货时间',
16         value: this.taskDetailData.planDepartureTime
17     })
18     this.getBaseContentItem({
19         title: '预计送达时间',
20         value: this.taskDetailData.planArrivalTime
21     })

```

Fence 123

- 准备builder函数

```

1 // 司机信息
2 @Builder
3 getDriverContent() {
4
5     this.getBaseContentItem({
6         title: '车牌号',
7         value: "京A123456"
8     })
9
10    this.getBaseContentItem({
11        title: '司机姓名',
12        value: "张三"
13    })
14
15 }
16
17 // 运输路线
18 @Builder
19 getTransLineContent() {
20     Row() {
21         Column() {
22             Text("北京
23 市").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22).font
24 weight(600)
25             Text("北京市").fontSize(14).lineHeight(22)
26         }.width(50)
27         Image($r("app.media.ic_right_arrow")).width(36).height(16)
28         Column() {
29             Text("河南
30 省").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22).font
31 weight(600)
32             Text("郑州市").fontSize(14).lineHeight(22)
33         }.width(50)
34     }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).wi
35 dth('100%').padding({
36     left: 60,
37     right: 60
38 })
39 }

```

```
1 build() {  
2   column () {  
3     HmNavBar({ title: '任务详情' })  
4     Scroll(this.scroller) {  
5       column() {  
6         HmToggleCard({ title: '基本信息' }) {  
7           this.getBaseContent()  
8         }  
9         HmToggleCard({ title: '车辆司机信息' }) {  
10          this.getDriverContent()  
11        }  
12        HmToggleCard({ title: '运输路线' }) {  
13          this.getTransLineContent()  
14        }  
15        }.padding({  
16          bottom: 20  
17        })  
18        .layoutweight(1)  
19      }  
20    }.backgroundColor($r('app.color.background_page')).height('100%')  
21  }
```

替换真实的数据

```
getDriverContent() {  
  Row() {  
    Text("车牌号").fontSize(14).fontColor($r('app.color.text_secondary'))  
    .lineHeight(20)  
    Text(this.TaskDetailData.licensePlate).fontSize(14).fontColor($r('app.color.text_secondary'))  
  }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({  
    top: 14  
  })  
  
  Row() {  
    Text('司机姓名').fontSize(14).fontColor($r('app.color.text_secondary'))  
    .lineHeight(20)  
    Text(this.TaskDetailData.driverName).fontSize(14).fontColor($r('app.color.text_secondary'))  
  }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({  
    top: 14  
  })  
}  
}  
  
getTransLineContent() {  
  Row() {  
    Column() {  
      Text(this.TaskDetailData.startProvince).fontSize(16).fontColor($r('app.color.text_secondary'))  
      Text(this.TaskDetailData.startCity).fontSize(14).lineHeight(22)  
    }.width(50)  
  
    Image($r("app.media.ic_right_arrow")).width(36).height(16)  
  
    Column() {  
      Text(this.TaskDetailData.endProvince).fontSize(16).fontColor($r('app.color.text_secondary'))  
      Text(this.TaskDetailData.endCity).fontSize(14).lineHeight(22)  
    }.width(50)  
  }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center)  
  left: 60,  
  right: 60  
}
```

黑马程序员

黑马程序员



任务详情

基本信息

 辽宁抚顺顺城鸣绿江北街58-8号  开始导航

 湖北武汉武昌友谊大道194号附近

任务编号 3111716124860458266

联系人 奶油张张包

联系电话 13112324543 

提货时间 2023-11-17 20:00:00

预计送达时间 2023-11-18 06:00:00

车辆司机信息

车牌号 粤A11223

司机姓名 司机

运输路线



:::info

总结

使用静态模版，利用插槽技术，替换真实数据
提交代码

:::

提货信息-上传组件HmUpload基本封装

basic模块

提货信息



请拍照上传回单凭证



请拍照上传货品照片



黑马程序员

:::info

分析

需要上传组件进行图片的上传，我们需要封装一个公共的组件HmUpload
先完成基本的布局

:::

1. 新建components下的HmUpload.ets文件，完成最基本布局

请拍照上传回单凭证



黑马程序员

```
1  @Component
2  export struct HmUpload {
3      title: string = ""
4      build() {
5          column() {
6
7              Text(this.title).fontSize(14).fontColor($r("app.color.text_secondary")).margin({
8                  top: 16,
9                  bottom: 16
10             })
11             Row() {
12                 Image($r("app.media.ic_add_img")).width(30).height(30)
13             }
14             .width(95)
15             .height(95)
16             .backgroundColor('#F2F2F2')
17             .alignItems(VerticalAlign.Center)
18             .justifyContent(FlexAlign.Center)
19         }.alignItems(HorizontalAlign.Start).width('100%')
20     }
21 }
```

Fence 126

2. 在components/index.ets导出

```
1  export * from './HmUpload'
```

Fence 127

Entry模块

3. 在TaskDetail中使用

```
1  // 提货信息内容
2  @Builder
3  getPickUpContent() {
4      HmUpload({ title: '请拍照上传回单凭证' })
5      HmUpload({ title: '请拍照上传货品照片' })
6  }
```

```

7   build() {
8     column() {
9       HmNavBar({ title: '任务详情' })
10      Scroll(this.scroller) {
11        column() {
12          HmToggleCard({ title: '基本信息' }) {
13            this.getBaseContent()
14          }
15
16          HmToggleCard({ title: '车辆司机信息' }) {
17            this.getDriverContent()
18          }
19
20          HmToggleCard({ title: '运输路线' }) {
21            this.getTransLineContent()
22          }
23
24          HmToggleCard({ title: '提货信息' }) {
25            this.getPickUpContent()
26          }
27        }.padding({
28          bottom: 120
29        })
30      }
31      }.backgroundColor($r('app.color.background_page')).height('100%')
32    }

```

Fence 128



:::info

总结

先封装HmUpload的基本结构，后续再去实现上传
提交代码

:::

任务详情-底部提货按钮结构



黑马程序员

:::info

实现上传组件前，先把底部的提交结构实现

:::

1. 从静态页面中拷贝结构

```
1 // 底部按钮结构
2 @Builder
3 getBottomBtn() {
4   //已完成不显示任何按钮
5   Row() {
6     Button("延迟收货", { type: ButtonType.Capsule })
7       .backgroundColor($r('app.color.btn_gray'))
8       .fontColor($r('app.color.text_primary'))
9       .fontSize(16)
10      .height(50)
11      .width(125)
12     Button("提货", { type: ButtonType.Capsule })
13       .backgroundColor($r('app.color.primary_disabled'))
14       .fontColor($r('app.color.white'))
15       .height(50)
16       .flexGrow(1)
17       .margin({ left: 13 })
18   }
19   .width('100%')
20   .padding({ left: 15, right: 15 })
21   .height(70).
22   justifyContent(FlexAlign.SpaceBetween).
23   alignItems(VerticalAlign.Center)
24   .backgroundColor($r('app.color.white'))
25 }
```

Fence 129

2. 放置在Scroll组件的同级，因为我们希望底部按钮是不滚动的

```
1 build() {
2   Column() {
3     HmNavBar({ title: '任务详情' })
4     Scroll(this.scroller) {
5       Column() {
```



```

6      HmToggleCard({ title: '基本信息' }) {
7          this.getBaseContent()
8      }
9
10     HmToggleCard({ title: '车辆司机信息' }) {
11         this.getDriverContent()
12     }
13
14     HmToggleCard({ title: '运输路线' }) {
15         this.getTransLineContent()
16     }
17
18     HmToggleCard({ title: '提货信息' }) {
19         this.getPickUpContent()
20     }
21     }.padding({
22         bottom: 20
23     })
24     .layoutWeight(1)
25
26     }
27     this.getBottomBtn() // 底部按钮结构
28 }.backgroundColor($r('app.color.background_page')).height('100%')
29 }

```



:::info

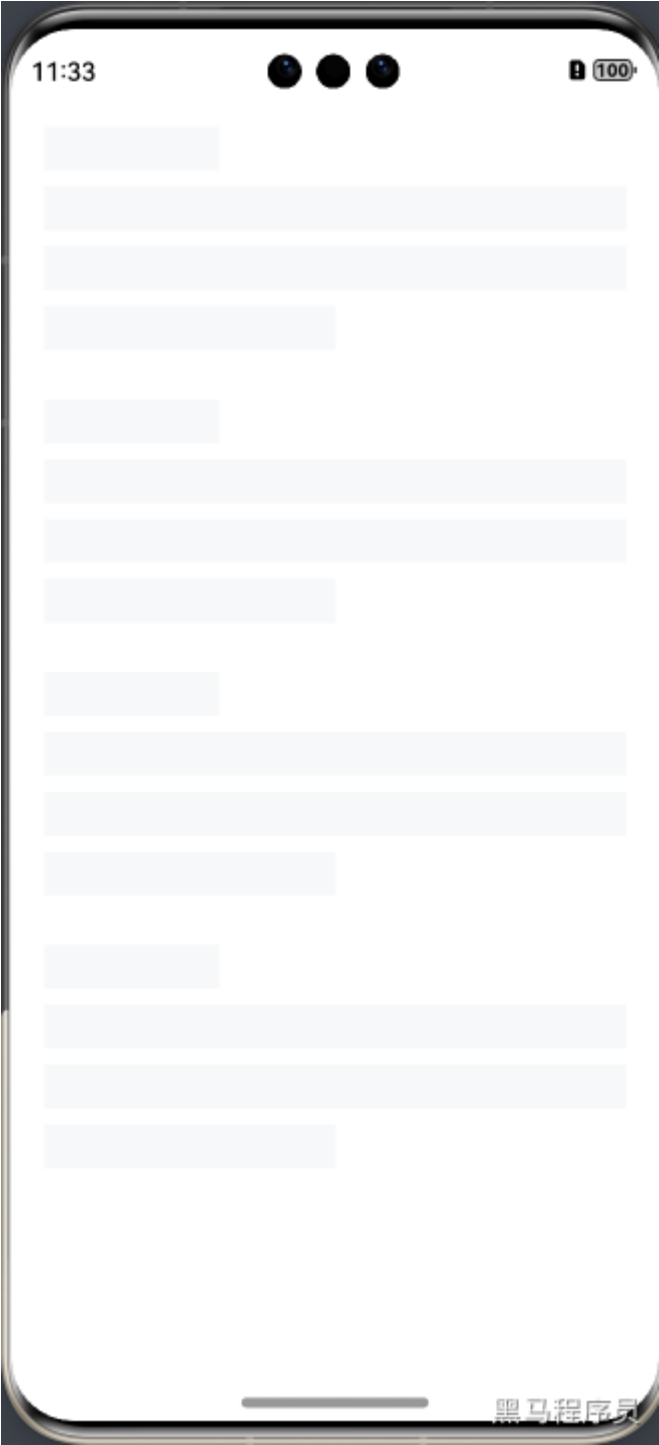
总结

提交代码

:::

添加骨架屏判断

```
1  build() {
2    Column () {
3      if(this.taskDetailData.id) {
4        HmNavBar({ title: '任务详情' })
5        Scroll(this.scroller) {
6          Column() {
7            HmToggleCard({ title: '基本信息' }) {
8              this.getBaseContent()
9            }
10           HmToggleCard({ title: '车辆司机信息' }) {
11             this.getDriverContent()
12           }
13           HmToggleCard({ title: '运输路线' }) {
14             this.getTransLineContent()
15           }
16           HmToggleCard({ title: '提货信息' }) {
17             this.getPickupContent()
18           }
19         }
20       }
21       .padding({
22         bottom: 20
23       })
24       .layoutWeight(1)
25       this.getBottomBtn() // 底部按钮结构
26     }else {
27       HmSkeleton({ count: 4, showAvatar: false })
28     }
29   }.backgroundColor($r('app.color.background_page')).height('100%')
30 }
31 }
```



提交代码

在其他项目上新建一个工具

:::info

因为模拟器不让拖动图片到相册目录，所以我们只能自己手动的将线上的图片下载到相册

:::

- 在pages/Index.ets中实现

↓ 下载

吨吨吨~



屁用没有



腻害

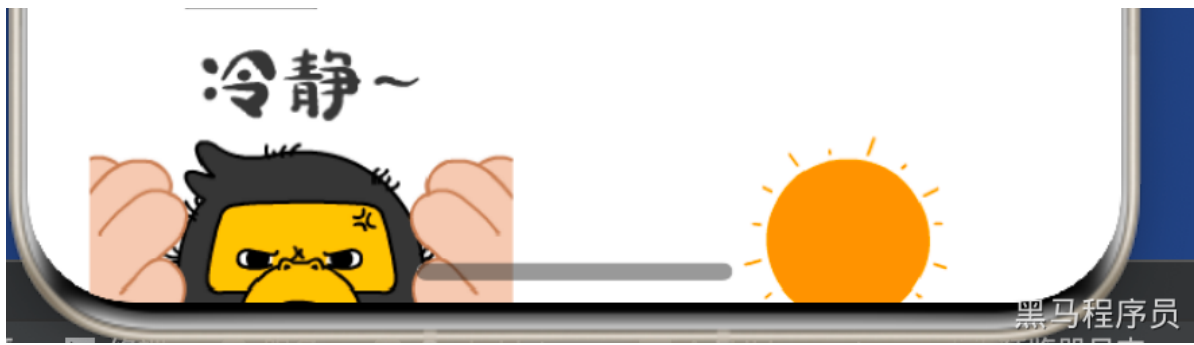


浮

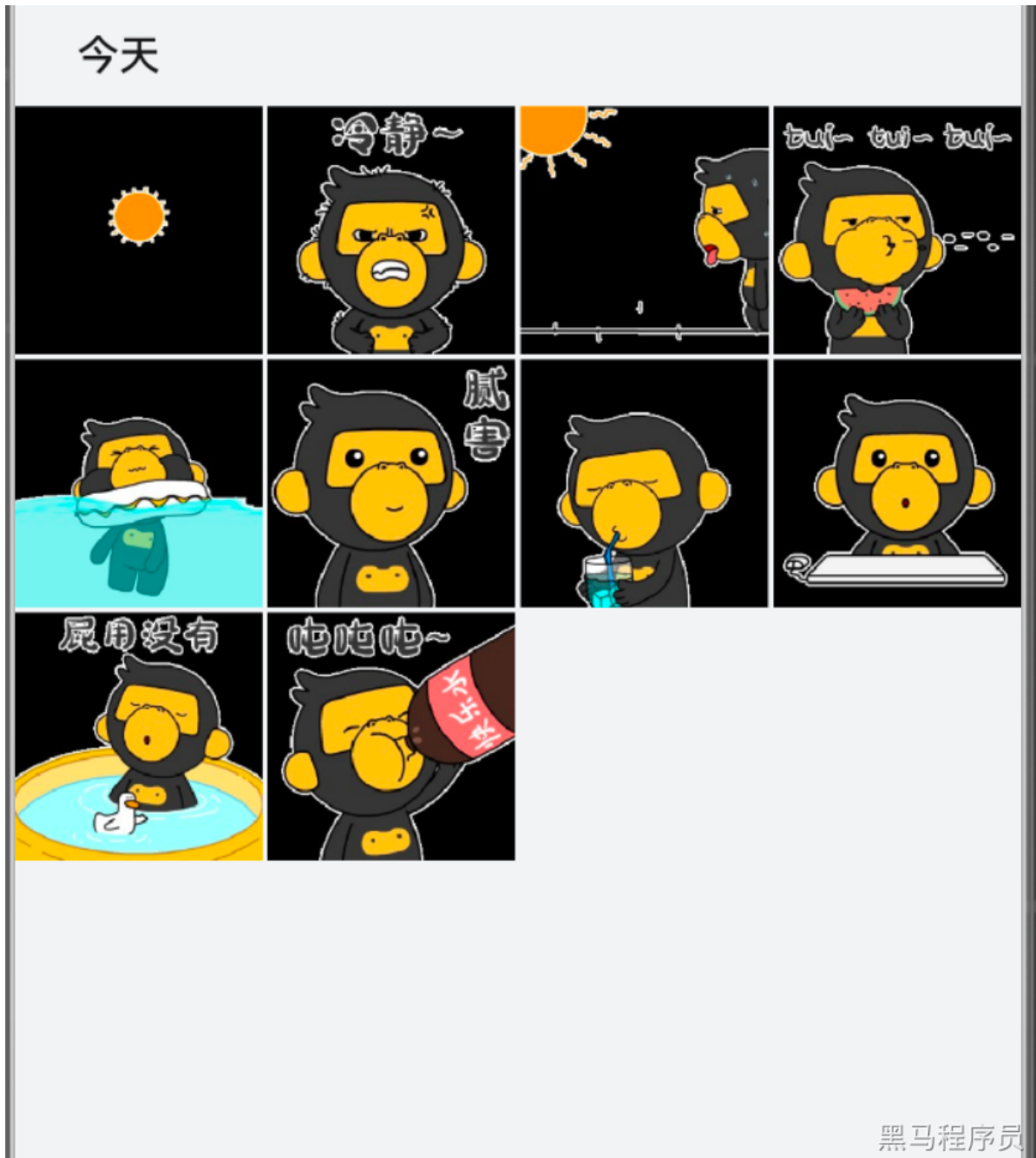


tui~ tui~ tui~





- 点击按钮拷贝图片



:::success

项目仓库地址: https://gitee.com/shuiruohanyu/copy_image

:::

上传功能-唤出相册- 拿到选择图片

basic模块

1. 点击上传区域，弹出图片选择器

```
1 // 弹出相册选择器
2 selectImage () {
3     let photoPicker = new picker.PhotoViewPicker();
4     photoPicker.select({
5         MimeType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
6         maxSelectNumber: 3
7     })
8 }
```

Fence 132

```
Row() {
    Image($r("app.media.ic_add_img")).width(30).height(30)
}
.width(95)
.height(95)
.backgroundColor('#F2F2F2')
.alignItems(VerticalAlign.Center)
.justifyContent(FlexAlign.Center).onClick(() => {
    this.selectImage()
})
})
```

黑马程序员

2. 将图片选择数量设置为可选

```
struct HmUpload {
    title: string = ""
    maxNumber: number = 3
    // 弹出相册选择器
    selectImage () {
        let photoPicker = new picker.PhotoViewPicker();
        photoPicker.select({
            MimeType: picker.PhotoViewMIMETypes.IMAGE_TYPE,
            maxSelectNumber: this.maxNumber
        })
    }
}

build() {
```

黑马程序员

此时可以在使用HmUpload时，自由传入maxNumber属性来控制数量

3. 获取选择后的图片列表

接收返回的选择图片列表


```

1 // 弹出相册选择器
2 async selectImage () {
3   let photoPicker = new picker.PhotoViewPicker();
4   const result = await photoPicker.select({
5     MIMETYPE: picker.PhotoViewMIMETypes.IMAGE_TYPE,
6     maxSelectNumber: this.maxNumber
7   })
8   AlertDialog.show({
9     message: JSON.stringify(result.photoUris)
10  })
11 }

```

Fence 133



:::info

总结

1. 通过file.picker的图片选择器选择图片，得到了一些临时路径
 2. 临时路径要传到我们自己的服务器-待实现
 3. 提交代码
- ...

拿到图片先将图片显示到预览区

- 声明一个数组来管理选择的图片

```
1  @State
2  imageUrl: ImageList[] = []
```

Fence 134

- 将选择的图片赋值给状态

```
1  // 循环数组
2  if (result.photoUris?.length) {
3      this.imageUrl = result.photoUris.map(url => {
4          return { url } as ImageList
5      })
6  }
```

Fence 135

- 循环渲染图片

```
1  // 渲染图片
2  Grid() {
3      ForEach(this.imageUrl, (item: ImageList) => {
4          GridItem() {
5              Image(item.url)
6                  .width(95)
7                  .height(95)
8                  .borderRadius(4)
9          }
10     })
11     if(this.imageUrl.length < this.maxNumber) {
12         GridItem() {
13             Row() {
14                 Image($r("app.media.ic_add_img")).width(30).height(30)
15             }
16             .width(95)
17             .height(95)
18             .backgroundColor($r("app.color.upload_panel"))
19             .alignItems(VerticalAlign.Center)
20             .justifyContent(FlexAlign.Center)
21             .onClick(() => {
22                 this.selectImage()
23             })
24         }
25     }
26 }
27 .height(Math.ceil(this.maxNumber / 3 ) * 105)
28 .columnsTemplate("1fr 1fr 1fr")
29 .columnsGap(10)
```

Fence 136

- 发现问题
:::success

当选择了两张之后，再选一张，发现只剩最后一次选的一张图片了，这是因为我们之前用的直接替换，
随意这里我们对图片的选择的图片张数做一下限制
...

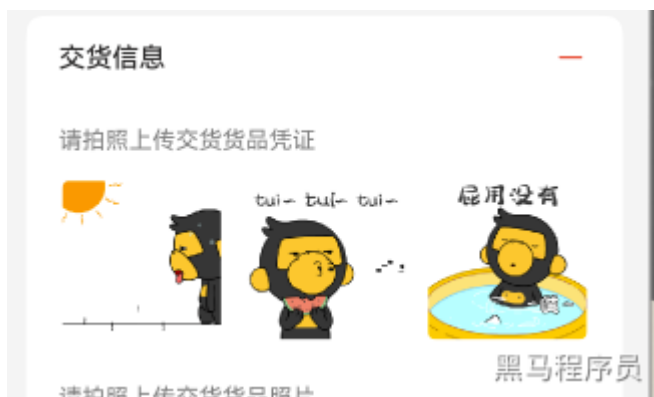
```
1  const photoPicker = new picker.PhotoViewPicker()
2  const result = await photoPicker.select({
3    MIMETYPE: picker.PhotoViewMIMETypes.IMAGE_TYPE,
4    maxSelectNumber: this.maxNumber - this.imageList.length
5  })
```

Fence 137

- 然后进行追加

```
1  // 循环数组
2  if (result.photoUris?.length) {
3    this.imageList = this.imageList.concat(result.photoUris.map(url => {
4      return { url } as ImageListModel
5    }))
6  }
```

Fence 138



:::success

提交代码

...

封装图片预览HmPreview

basic模块

:::info

分析，当我们上传的图片时，我们希望能去放大预览该图片



并且图片还支持多张一起，比如上传了三张，点第二张可以预览，左右滑可以切到第一张和第三张

:::

1. 新建components/HmPreview.ets文件

```
1  @CustomDialog
2  @Component
3  struct HmPreview {
4      controller: CustomDialogController
5      // 支持多张图片预览 给多张地址 给一个需要预览的索引
6      urls: string[] = [] // 多张地址
7      selectIndex: number = 0 // 当前索引
8      build() {
9          column() {
10             swiper() {
11                 ForEach(this.urls, (url: string) => {
12                     Image(url)
13                         .width("100%") // 只给宽度 不给高度 让自己撑开
14                         .onClick(() => {
15                             this.controller.close()
16                         })
17                 })
18             }
19             .indicator(false) // 去掉点的显示
20             .index(this.selectIndex) // 当前要看的是第几张图片
21         }
22         .justifyContent(FlexAlign.Center)
23         .width("100%")
24         .height("100%")
25         .backgroundColor($r("app.color.black"))
26     }
27 }
```

```

28 | }
29 | export { HmPreview }

```

Fence 139

在components/index.ets导出

```

1 | export * from './HmPreview'

```

Fence 140

- 在HmUpload中引入, 并初始化dialogController

```

1 | import { HmPreview } from './HmPreview'
2 |
3 | preview = new CustomDialogController({
4 |     builder: HmPreview({
5 |         urls: this.imageList.map(item => item.url),
6 |         selectIndex: this.index
7 |     }),
8 |     customStyle: true, // 自定义样式
9 | })

```

Fence 141

- 在HmUpload中声明index属性

```

1 |
2 | index: number = -1

```

Fence 142

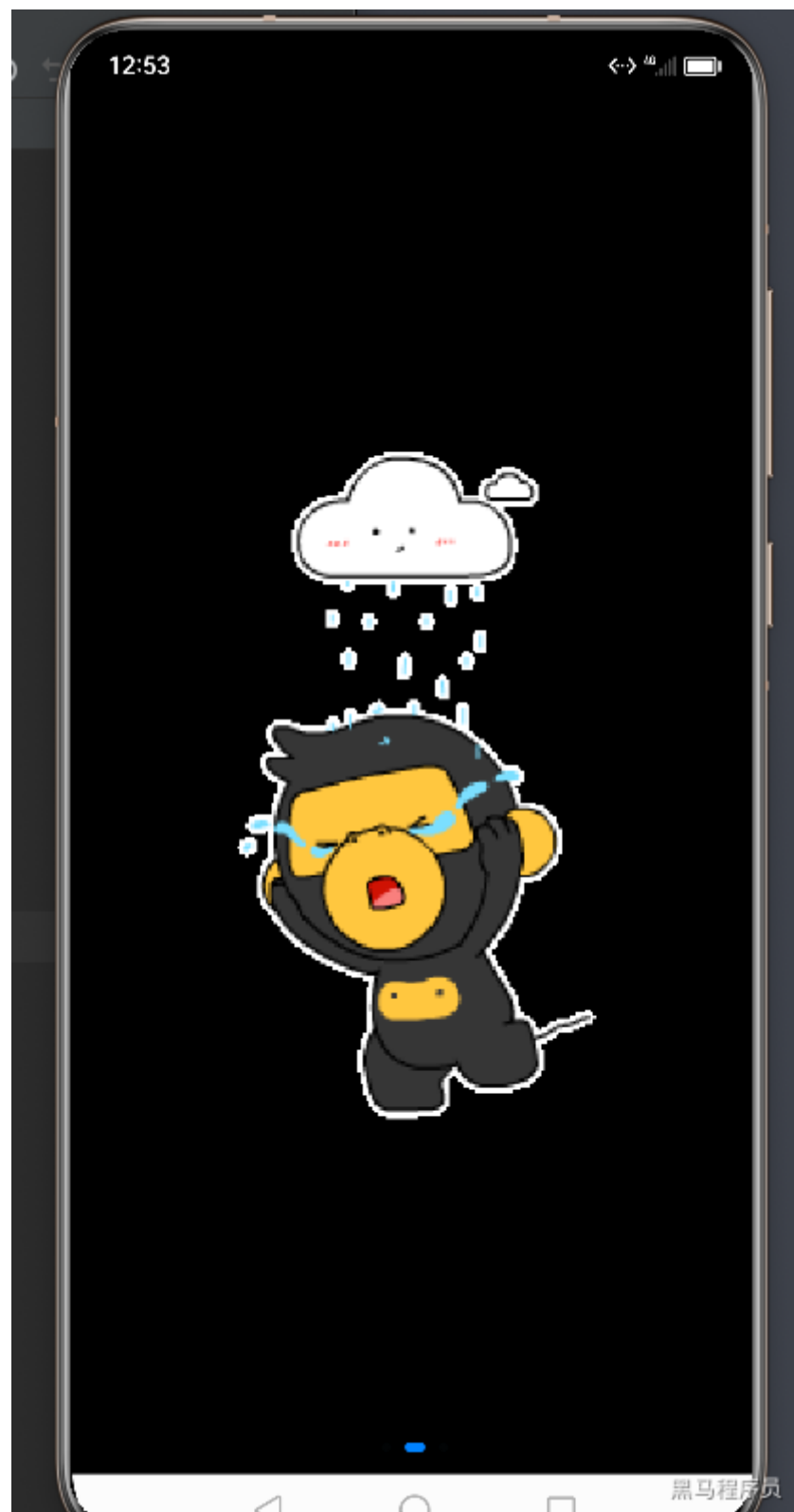
- 切换点击图片时, 切换索引, 打开弹层

```

1 |   ForEach(this.imageList, (item: ImageListModel, index) => {
2 |       GridItem() {
3 |           Image(item.url)
4 |               .width(95)
5 |               .height(95)
6 |               .onClick(() => {
7 |                   this.index = index
8 |                   this.preview.open()
9 |               })
10 |      }
11 |  })

```

Fence 143



:::info
提交代码
:::

接收父组件传入

:::success

因为图片的来源有可能是已经有的图片，所以我们需要外部传入该图片列表

:::

- 将State修饰符变成Prop修饰符

```
1 | @Prop
2 | imageUrl: ImageList[] = []
```

Fence 144

entry模块

- 在提货详情的页面中进行传入

```
1 | @Builder
2 | getPickUpContent() {
3 |   HmUpload({ title: '请拍照上传货品凭证',
4 |     imageUrl: this.taskDetailData.cargoPickUpPictureList || []
5 |   })
6 |   HmUpload({ title: '请拍照上传货品照片',
7 |     imageUrl: this.taskDetailData.cargoPictureList || []
8 |   })
9 | }
```

Fence 145

:::success

当图片地址发生变化时，通知父组件更新

:::

basic模块

- 在HmUpload中定义一个外部传入的函数

```
1 | onImageListChange: (list: ImageListModel[]) => void = () => {}
```

Fence 146

- 在selectImage方法中选择完毕图片之后调用该方法

```
1 | // 循环数组
2 | if (result.photoUris?.length) {
3 |   this.imageUrl = this.imageUrl.concat(result.photoUris.map(url => {
4 |     return { url } as ImageListModel
5 |   })))
6 |   this.onImageListChange(this.imageUrl)
7 | }
```

Fence 147

- 父组件传入函数更新对应的字段

entry模块

```
1 | // 专门负责提货信息的结构
```

```

2 | @Builder
3 | getPickUpContent() {
4 |     HmUpload({ title: '请拍照上传货品凭证',
5 |         imageUrl: this.taskDetailData.cargoPickUpPictureList || [],
6 |         onImageListChange: (list: ImageListModel[]) => {
7 |             this.taskDetailData.cargoPickUpPictureList = list
8 |         }
9 |     })
10 |     HmUpload({ title: '请拍照上传货品照片',
11 |         imageUrl: this.taskDetailData.cargoPictureList || [],
12 |         onImageListChange: (list: ImageListModel[]) => {
13 |             this.taskDetailData.cargoPictureList = list
14 |         }
15 |     })
16 | }

```

Fence 148

- 根据提货图片存在判断是否可点击按钮

```

1 | // 获取提货的状态
2 | getPickUpState() {
3 |     return this.taskDetailData.cargoPickUpPictureList?.length > 0 &&
4 |         this.taskDetailData.cargoPictureList?.length > 0
5 | }

```

Fence 149

- 设置状态

```

1 | Button("提货", { type: ButtonType.Capsule })
2 |     .backgroundColor($r('app.color.primary'))
3 |     .fontColor($r('app.color.white'))
4 |     .height(50)
5 |     .flexGrow(1)
6 |     .margin({ left: 13 })
7 |     .enabled(this.getPickUpState())

```

Fence 150

:::success

提交代码

:::

沙箱文件拷贝

:::info

官网上传案例-[链接](#)

1. 上传文件我们需要使用官方的uploadFile方法

...

:::success

我们需要在点击提货按钮时，将所有的图片进行上传，并且监听上传进度，显示在页面上

...

request.uploadFile⁹⁺

uploadFile(context: BaseContext, config: UploadConfig): Promise<UploadTask>

上传，异步方法，使用promise形式返回结果。通过[on\('fail'\)](#)⁷⁺可获取任务上传时的错误信息。

需要权限：ohos.permission.INTERNET

系统能力：SystemCapability.MiscServices.Upload

参数：

参数名	类型	必填	说明
context	BaseContext	是	基于应用程序的上下文。
config	UploadConfig	是	上传的配置信息。

返回值：

黑马程序员

:::info

第一个参数 context可以直接使用 getContext(this)获得

第二个参数的config参数为

名称	类型	必填	说明
url	string	是	资源地址。
header	Object	是	添加要包含在上传请求中的HTTP或HTTPS标志头。
method	string	是	请求方法：POST、PUT。缺省为POST。
files	Array< File >	是	要上传的文件列表。请使用 multipart/form-data提交。
data	Array< RequestData >	是	请求的表单数据。

黑马程序员

其中File的参数为

名称	类型	必填	说明
filename	string	是	multipart提交时，请求头中的文件名。
name	string	是	multipart提交时，表单项目的名称，缺省为file。
uri	string	是	文件的本地存储路径。仅支持“internal”协议类型，“internal://cache/”为必填字段，示例：internal://cache/path/to/file.txt
type	string	是	文件的内容类型，默认根据文件名或路径的后缀获取。

黑马程序员

...

综上所述，我们需要得到File中的uri这个属性，uri属性仅支持“internal”协议类型
但是我们通过弹窗发现，前面所得到的图片路径为

["datashare:///media/image/356"
,"datashare:///media/image
/349"]

黑马程序员

:::info

格式不匹配！！

需要转化

怎么转？

我们需要将选择的图片列表 一个个的拷贝到cache目录下，得到cache目录后新的目标路径

[应用沙箱文件官网介绍](#)

...

basic模块

- 在HmUpload中导出一个上传方法

```
1 | export const UploadFile = (list: ImageListModel[]) => {}
```

Fence 151

```
1 | import fs from '@ohos.file.fs';
2 | import { util } from '@kit.ArkTS';
3 | // 上传方法
4 | export const UploadFile = (list: ImageListModel[]) => {
5 |     // 因为上传文件只能从沙箱文件中拷贝所以 我们需要把传过来的所有的图片拷贝到沙箱
```

```

6   const saveDir = getContext().cacheDir // 存储的目录
7   const fileParams: request.File[] = [] // 要提交的参数
8   list.forEach(item => {
9       const file = fs.openSync(item.url, fs.OpenMode.READ_ONLY) // 读取相册的文
件
10      // 将文件拷贝到沙箱目录
11      // 相册的地址
12      const uniqueName = util.generateRandomUUID() + ".jpg"
13      fs.copyFileSync(file.fd, saveDir + "/" + uniqueName) // 将相册文件拷贝到沙
箱
14      // 需要生成参数
15      fileParams.push({
16          filename: uniqueName, // 文件名称
17          name: 'file', // 接口的参数名称
18          type: 'jpg', // 文件后缀
19          uri: `internal://cache/${uniqueName}` // 应该是文件放到cache目录下 如果是
cache协议 它会自动找这个文件
20      })
21      fs.closeSync(file.fd)
22    })
23    AlertDialog.show({ message: JSON.stringify(fileParams) })
24
25 }

```

Fence 152

- 得到对应的路径



总结

- 文件上传必须先把文件拷贝到沙箱文件中
 - 沙箱文件最终要上传时需要需要internal://cache目录 + 自己的文件路径
- 提交代码

封装上传文件的api

:::info

现在上传组件已经准备好了参数，我们封住一个统一的上传api

:::

在api下新建upload.ets

```
1 import request from '@ohos.request'
2 import { BASE_URL, TOKEN_KEY } from '../constants'
3 import { ImageListModel } from '../models'
4 import { ResponseData } from '../utils'
5
6 // 返回一个列表 ImageListModel[]
7 export const uploadImageAPI = async (context: Context, files:
  request.File[]) => {
8   let config: request.UploadConfig = {
9     url: BASE_URL + '/files/imageUpload', // 拼接上传完整地址
10    method: 'POST',
11    header: {
12      Authorization: AppStorage.get(TOKEN_KEY) || "", // 应用中的token
13      "Content-Type": "multipart/form-data" // 上传文件的参数类型
14      // application/json { name: '张三', age: 18 }
15      // application/x-www-form-urlencoded 拼接 name=张三&age=18
16      // text/html 结构
17      // multipart/form-data 专门传文件的结构
18    },
19    files,
20    data: [] // 用不上
21  }
22   return new Promise<ImageListModel[]>(async (resolve, reject) => {
23     try {
24       let arr: ImageListModel[] = []
25       const task = await request.uploadFile(context, config) // 成功执行并不意
  味着 上传成功 只是任务创建成功
26       task.on("fail", () => {
27         // 上传失败
28         AlertDialog.show({
29           message: "上传失败"
30         })
31         reject(new Error("上传失败"))
32       })
33       // 每上传成功一次就会进来
34       task.on("headerReceive", (headers: object) => {
35         if (headers["body"]) {
36           const result = JSON.parse(headers["body"]) as ResponseData<string>
37           if (result.code === 200) {
38             arr.push({
39               url: result.data as string
40             })
41           }
42         }
43       })
44       task.on("complete", () => {
```

```

45      // 上传完成
46      // 也不能拿到上传成功的地址
47      // 走到这里认为 要返回的结构已经ok了
48      resolve(arr)
49    })
50
51    }catch(error) {
52      AlertDialog.show({
53        message: error.message // 提示错误消息
54      })
55      reject(error)
56    }
57  })
58
59 }

```

Fence 153

- 在api/index.ets中导出

```
1 export * from './upload'
```

Fence 154

- 在HmUpload组件的UploadFile方法中调用

```

1 export const UploadFile = async (list: ImageListModel[]) => {
2   const saveDir = getContext().cacheDir // 存储的目录
3   const fileParams: request.File[] = [] // 要提交的参数
4   list.forEach(item => {
5     const file = fs.openSync(item.url, fs.OpenMode.READ_ONLY) // 读取相册的文
      件
6     // 将文件拷贝到沙箱目录
7     // 相册的地址
8     const uniqueName = util.generateRandomUUID() + ".jpg"
9
10    const targetFolder = saveDir + "/" + uniqueName
11
12    fs.copyFileSync(file.fd, targetFolder) // 将相册文件拷贝到沙箱
13    // 需要生成参数
14    fileParams.push({
15      filename: uniqueName, // 文件名称
16      name: 'file', // 接口的参数名称
17      type: 'jpg', // 文件后缀
18      uri: `internal://cache/${uniqueName}` // 应该是文件放到cache目录下 如果是
      cache协议 它会自动找这个文件
19    })
20    fs.closeSync(file.fd)
21  })
22
23   return await uploadImageAPI(getContext(), fileParams)
24 }

```

Fence 155

:::info

提交代码

:::

提货时上传

entry模块

1. 拷贝接口类型-[接口文档](#)

- 新建一个pickup提货的ets类型声明文件- models/pickup.ets

```
1 import { ImageListModel } from '@hm/basic'
2 export interface PickupParamsModel {
3     /** 提货凭证照片数组 */
4     cargoPickUpPictureList: ImageListModel[];
5     /** 提货照片数组 */
6     cargoPictureList: ImageListModel[];
7     /** 司机作业id */
8     id: string;
9 }
```

Fence 156

- 导出统一的文件

```
1 export * from './pickup'
```

Fence 157

- 封装提货接口

```
1 import { PickupParamsModel } from '../models'
2
3 // 提货
4 export const pickupAPI = (data: PickupParamsModel) => {
5     return Request.post<null>("/driver/tasks/takeDelivery", data)
6 }
```

Fence 158

3. 提货点击事件-调用api

```
1 // 去提货
2 async onPickUp() {
3     const cargoPickUpPictureList = await
UploadFile(this.taskDetailData.cargoPickUpPictureList) // 传入提货凭证
4     const cargoPictureList = await
UploadFile(this.taskDetailData.cargoPictureList) // 货品照片
5     // 提货操作
6     await pickupAPI({
7         id: this.taskDetailData.id,
8         cargoPickUpPictureList,
9         cargoPictureList
10    })
11    // 只需要重新获取数据
12    // 重新获取数据
13    this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
14
15    this.scroller.scrollToEdge(Edge.Top)
16    // 滚动到顶部
```

```
17 |     promptAction.showToast({ message: '提货成功' })
18 |
19 | }
```

Fence 159

点击事件

```
1 | Button("提货")
2 | ....
3 | .onClick(() => {
4 |     this.onPickup()
5 | })
```

Fence 160

:::info

总结

提交代码

:::

实现删除图片



1. 在图片中放入右上角的图标

```
1 ForEach(this.imageList, (item: ImageList, index: number) => {
2      GridItem() {
3          Stack({ alignContent: Alignment.TopEnd }) {
4              Image(item.url)
5                  .width(95)
6                  .height(95)
7                  .borderRadius(4)
8                  .onClick() => {
9                      this.index = index
10                     this.preview.open()
11                 }
12              Image($r('app.media.ic_btn_delete')).width(30).height(30)
13                  .margin({ right: 15, bottom: 10 })
14          }
15      })
```

Fence 161

2. 注册点击事件-处理数据更新

```
1  Image($r('app.media.ic_btn_delete')).width(30).height(30)
2      .onClick() => {
3          this.imageList.splice(index, 1) // 移除索引
4          this.onImageListChange(this.imageList) // 通知父组件更新
5      }
```

Fence 162

:::info

测试并提交代码

:::

加载进度遮罩

basic模块

- 封装一个HmLoading的组件，用过弹层

```
1  @CustomDialog
2  @Component
3  @Preview
4  export struct HmLoading {
5      controller: CustomDialogController
6      title: string = '数据处理中..'
7
8      build() {
9          Row({ space: 6 }) {
10              Text(this.title)
11                  .fontSize(16)
12                  .fontColor($r("app.color.text_primary"))
13              LoadingProgress()
14                  .width(30)
15                  .height(30)
16                  .color($r("app.color.primary"))
17          }
18          .justifyContent(FlexAlign.Center)
19          .width('100%')
20          .height('100%')
21          .backgroundColor("rgba(0,0,0,0.1)")
22      }
23  }
```

Fence 163

- 在TaskDetail中定义loading弹出层的对象

```
1  loading = new CustomDialogController({
2      builder: HmLoading(),
3      customStyle: true
4  })
```

Fence 164

- 加载前后打开和关闭

```
1  async getTaskDetail(id: string) {
2      this.loading.open()
3      this.taskDetailData = await getTaskDetailAPI(id)
4      this.loading.close()
5  }
```

Fence 165

- 提货前后打开和关闭弹窗

```
1  async toPickup () {
2      this.loading.open()
```

```
3      const cargoPictureList = await
uploadFile(this.taskDetailData.cargoPictureList)
4      const cargoPickUpPictureList = await
uploadFile(this.taskDetailData.cargoPickUpPictureList)
5      await pickupAPI({
6        id: this.taskDetailData.id,
7        cargoPickUpPictureList: cargoPictureList,
8        cargoPictureList: cargoPickUpPictureList
9      })
10     this.getTaskDetail(this.taskDetailData.id) // 重新加载数据
11     this.scroller.scrollToEdge(Edge.Top)
12     this.loading.close()
13     promptAction.showToast({ message: '提货成功' })
14
15   }
```

Fence 166

:::success

提交代码

:::

交货列表加载

分析：
一个任务，如果已经提货，那么下一步应该就司机去送货，完成交货，上一节我们完成了提货，那么完成的这个任务应该在在途里面，所以我来先把在途的数据进行查询一下

待提货

在途

已完成



无在途任务



任务



消息



我的

- 在途和待提货的区别就是查询的状态不同，刚刚好，我们前面定义了枚举，所以这里可以直接复用前面提货的TaskList的组件

entry模块

```

1  build() {
2      Stack({ alignContent: Alignment.Top }) {
3          Tabs({ barPosition: BarPosition.Start, index: $$this.currentIndex,
4              controller: this.tabController }) {
5              ForEach(this.tabsData, (item: TabClass) => {
6                  TabContent(){
7                      if(item.name === "waiting") {
8                          TaskList()
9                      }
10                     else if(item.name === 'line') {
11                         TaskList({ params: {
12                             page: 1,
13                             pageSize: 5,
14                             status: TaskTypeEnum.Line
15                         } as TaskListParamsModel })
16                     }
17                     else {
18                         Text(item.title)
19                     }
20                 }
21             })
22         }.backgroundColor($r('app.color.background_page')).animationDuration(300)
23         Row ({ space: 30 }) {
24             ForEach(this.tabsData, (item: TabClass) => {
25                 this.getTabBar(item)
26             })
27         }.padding({
28             left: 40,
29             right: 40
30         })
31         .width('100%')
32         .height(50)
33         .backgroundColor($r("app.color.white"))
34     }
35 }
36 
```

Fence 167

因为我们TaskList默认是查的提货的数据-所以不用传参数了，传了和原来的参数也是一样的

```

1  import { getTaskListAPI } from '../../api/task'
2  import { TaskInfoItemModel, TaskListParamsModel, TaskTypeEnum } from
3  '../../models'
4  import TaskItemCard from './TaskItem'
5  import HmList from '@hm/basic/src/main/ets/components/HmList'
6
7  @Preview
8  @Component

```

```

8  struct TaskList {
9      params: TaskListParamsModel = {
10         page: 1, // 表示查询第几页的数据 ++
11         pageSize: 10, // 表示每页查几条数据
12         status: TaskTypeEnum.Waiting,
13     } as TaskListParamsModel
14     @State taskList: TaskInfoItemModel[] = []
15     @State finished: boolean = false // 服务器上没有更多数据了
16     onLoad = async (isRefresh?: boolean) => {
17
18         const data = await getTaskListAPI(this.params) // page: 1 2
19
20         const items = data.items || [] // data.items有可能为null
21
22         if (isRefresh) {
23             this.taskList = items // 刷新要覆盖数据
24         } else {
25             this.taskList.push(...items) // 无限滚动 要拼接数据
26         }
27         this.params.page++ // 2 3
28
29         if (this.params.page > data.pages) {
30             this.finished = true // 服务器上没有更多数据了
31         }
32     } // 防抖或者节流
33     onRefresh = async () => {
34         this.params.page = 1 // 重置数据
35         this.finished = false // 重置数据
36         await this.onLoad(true) // 再次发请求, 覆盖旧数据
37     }
38
39     @Builder
40     renderItem(item: object) {
41         TaskItemCard({ taskItem: item as TaskInfoItemModel })
42     }
43
44     build() {
45         HmList({
46             finished: this.finished,
47             dataSource: this.taskList,
48             onLoad: this.onLoad,
49             onRefresh: this.onRefresh,
50             finishedText: '我是有底线的',
51             loadingText: '疯狂加载中',
52             renderItem: this.renderItem
53         })
54     }
55 }
56 }
57
58 export default TaskList
59

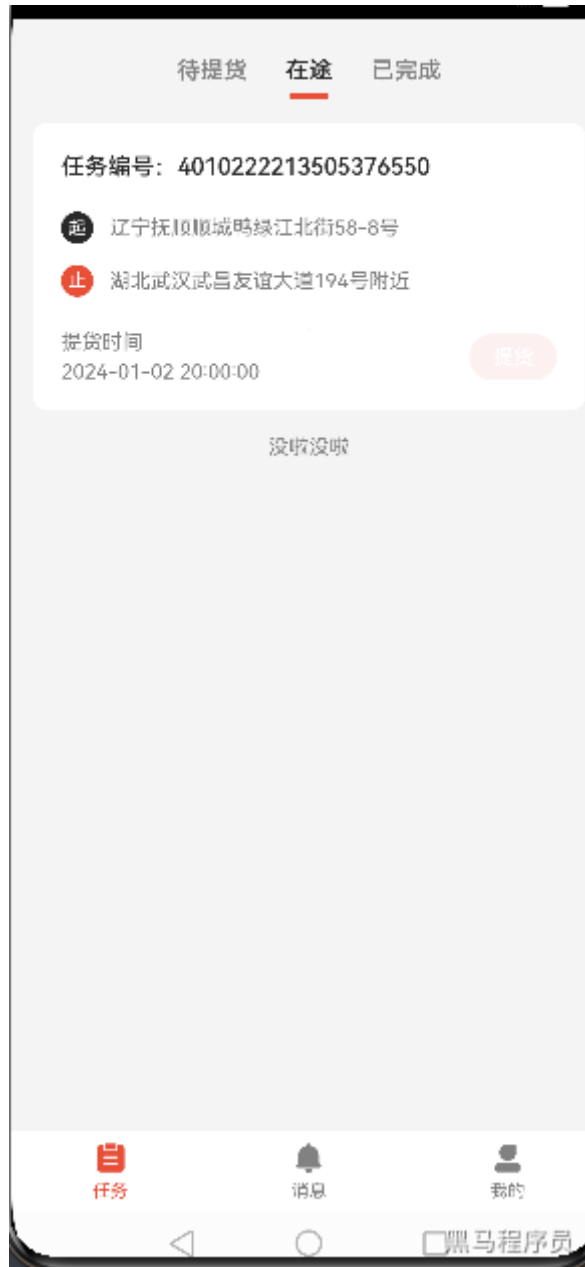
```



发现List只有一条数据时，List的内容是位于容器的中间位置，可以给TaskList中的HmList再加一层容器，
设置高度为100%

```
1 build() {
2     HmList({
3         onLoad: async () => {
4             await this.getTaskList(true)
5         },
6         onRefresh: async () => {
7             await this.onRefresh()
8         },
9         dataSource: $taskListData,
10        renderItem: this.renderItem,
11        finished: this.allPage < this.queryParams.page,
12        finishText: '没啦没啦',
13        loadingText: '拼命加载中'
14    })
15    .height('100%')
16 }
```

- 实现效果



接下来，根据状态去调整按钮的显示和文本内容

- 当任务的状态为待提货并且可提货时，按钮显示提货并且可用
 - 当任务的状态为待交货时，按钮显示提货并且可用
 - 当任务的状态为待回车登记时，按钮显示回车登记并且可用
- 给之前的任务状态枚举再加一个属性

```
1 export enum TaskTypeEnum {  
2     waiting = 1, // 待提货  
3     Line = 2, // 在途待交货  
4     Delivered = 4, // 已交货待回车登记  
5     Finish = 6 // 已完成  
6 }
```

Fence 170

- 在TaskItemCard组件中根据状态获取按钮的文本及可用状态

如果enablePickup为true 表示可提货
如果status 为 2 表示为待交货 显示交货
如果status 为 4表示 待回车登记

```
1  // 获取按钮可用性
2  getBtnEnable() {
3      const value = this.taskItem.status
4      if (this.taskItem.enablePickUp) {
5          return true
6      }
7      switch (value) {
8          case TaskTypeEnum.Line
9          case TaskTypeEnum.Delivered:
10         return true
11         default:
12         return false
13     }
14 }
15
16 // 获取按钮显示文本
17 getBtnText() {
18     const value = this.taskItem.status
19     switch (value) {
20         case TaskTypeEnum.Waiting:
21             return "提货"
22         case TaskTypeEnum.Line:
23             return "交货"
24         case TaskTypeEnum.Delivered:
25             return "回车登记"
26         default:
27             return ""
28     }
29 }
30
```

Fence 171

- 按钮显示文本

```
1  Button(this.getBtnText(), { type: ButtonType.Capsule })
2      .backgroundColor($r('app.color.primary') )
3      .fontColor($r("app.color.white"))
4      .fontSize(14)
5      .height(32)
6      .enabled(this.getBtnEnable())
7      .onClick(() => {
8          this.toPickUp()
9      })
```

Fence 172

⚠： 这里的去提货的逻辑不用发生任何变化，因为交货的业务也是到详情页去提货，完成可以复用一模一样的逻辑



提交代码

交货详情内容控制

1. 根据状态显示不同的按钮

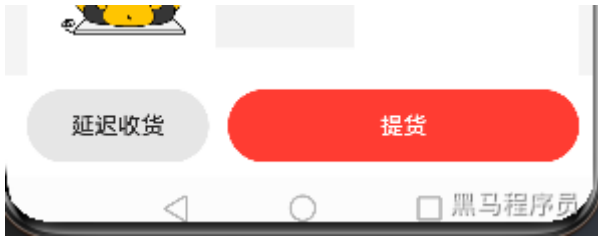
status	integer	作业状态	必需
1为待提货)、2为在途)、3为改派)、4为已交付)、5为已作废、6为已完成（已回车登记)			1, "t
certificatePictureList	array [object {1}]	回单凭证	必需
exceptionList	array [object {6}]		必需

:::info

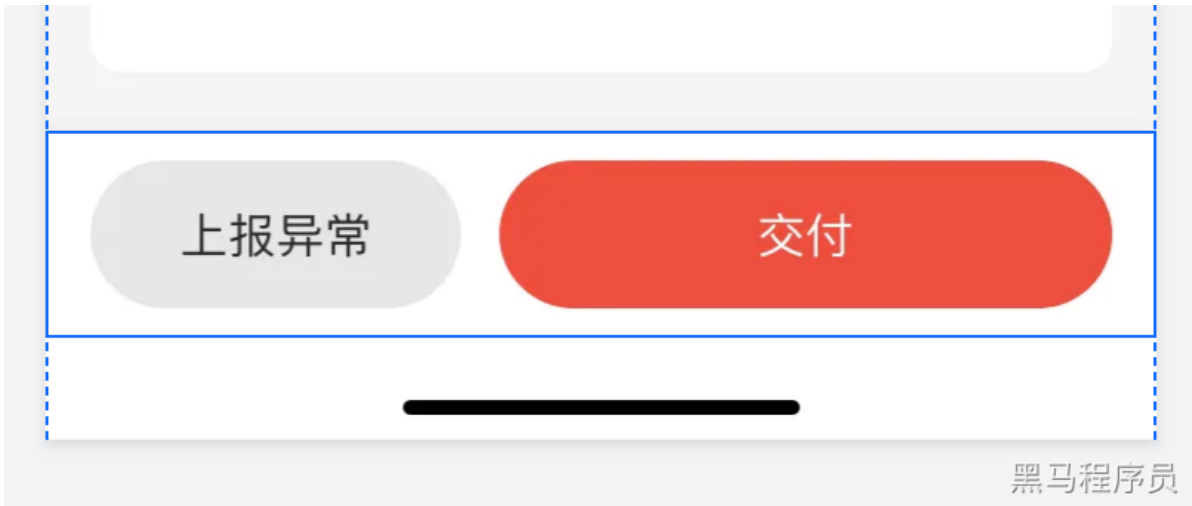
当一个任务提完货后，status应该是Line(在途)，如果是Waiting(待提货)的话显示提货按钮，如果是Line(在途)，显示交货按钮

:::

1. 提货



2. 交货



```
1 // 底部按钮结构
2 @Builder
3 getBottomBtn() {
4     //已完成不显示任何按钮
5     Row() {
6         if(this.taskDetailData.status === TaskTypeEnum.Waiting) {
7             Button("延迟收货", { type: ButtonType.Capsule })
8                 .backgroundColor($r('app.color.btn_gray'))
9                 .fontColor($r('app.color.text_primary'))
10                .fontSize(16)
11                .height(50)
12                .width(125)
```

```

13         Button("提货", { type: ButtonType.Capsule })
14             .backgroundColor($r('app.color.primary'))
15             .fontColor($r('app.color.white'))
16             .height(50)
17             .flexGrow(1)
18             .margin({ left: 13 })
19             .enabled(this.getPickUpState())
20             .onClick(() => {
21                 this.onPickUp()
22             })
23     }
24     else if(this.taskDetailData.status === TaskTypeEnum.Line) {
25         Button("上报异常", { type: ButtonType.Capsule })
26             .backgroundColor($r('app.color.btn_gray'))
27             .fontColor($r('app.color.text_primary'))
28             .fontSize(16)
29             .height(50)
30             .width(125)
31         Button("交货", { type: ButtonType.Capsule })
32             .backgroundColor($r('app.color.primary'))
33             .fontColor($r('app.color.white'))
34             .height(50)
35             .flexGrow(1)
36             .margin({ left: 13 })
37     }
38 }
39
40 }
41 .width('100%')
42 .padding({ left: 15, right: 15 })
43 .height(70)
44 .justifyContent(FlexAlign.SpaceBetween)
45 .alignItems(VerticalAlign.Center)
46 .backgroundColor($r('app.color.white'))
47 }
48

```

Fence 173

2. 交货时，**隐藏提货上传组件**，显示交货上传组件



- 实现一个Builder函数来放置交货的上传

```
1 // 专门负责交货信息的结构
2 @Builder
3 getDeliverContent() {
4   HmUpload({
5     title: '请拍照上传交货货品凭证',
6     imageUrl: this.taskDetailData.deliverPictureList || [],
7     onImageListChange: (list: ImageListModel[]) => {
8       this.taskDetailData.deliverPictureList = list
9     }
10  })
11  HmUpload({
12    title: '请拍照上传交货货品照片',
13    imageUrl: this.taskDetailData.certificatePictureList || [],
14    onImageListChange: (list: ImageListModel[]) => {
15      this.taskDetailData.certificatePictureList = list
16    }
17  })
18 }
```

Fence 174

```
1 // 提货时显示提货照片
2 if(this.taskDetailData.status === TaskTypeEnum.waiting) {
3   HmToggleCard({ title: '提货信息' }) {
4     this.getPickupContent()
5   }
6 }
7 // 交货时显示交货照片
8 if(this.taskDetailData.status === TaskTypeEnum.Line) {
9   HmToggleCard({ title: '交货信息' }) {
10    this.getDeliverContent()
11  }
12 }
```

Fence 175

:::info

提交代码

:::

交货状态控制

```
1 // 获取交货状态
2 getDeliverState () {
3     return this.taskDetailData.deliverPictureList?.length > 0 &&
4         this.taskDetailData.certificatePictureList?.length > 0
5 }
```

Fence 176

```
1 Button("交货", { type: ButtonType.Capsule })
2     .backgroundColor($r('app.color.primary'))
3     .fontColor($r('app.color.white'))
4     .height(50)
5     .flexGrow(1)
6     .margin({ left: 13 })
7     .enabled(this.getDeliverState())
8 
```

Fence 177



:::info
提交代码
:::

交货

Entry模块

:::info

标准流程

1. 定义类型封装api
2. 引入api
3. 注册事件-调用api重新加载
- ...
4. 定义请求参数类型

```
1 |  
2 | import { ImageListModel } from '@hm/basic'  
3 | export interface DeliverParamsTypeModel {  
4 |   /** 交付凭证列表 */  
5 |   certificatePictureList: ImageListModel[];  
6 |   /** 交付图片列表 */  
7 |   deliverPictureList: ImageListModel[];  
8 |   /** 司机作业id */  
9 |   id: string;  
10 | }  
11 |
```

Fence 178

- 在models/index.ets统一导出

```
1 | export * from './deliver'
```

Fence 179

2. 封装API

```
1 | // 交货  
2 | export const deliverAPI = (data: DeliverParamsTypeModel) => {  
3 |   return Request.post("/driver/tasks/deliver", data)  
4 | }
```

Fence 180

3. 调用


```

1  async onDeliver() {
2      this.loading.open()
3      await deliverAPI({
4          certificatePictureList: this.taskDetailData.certificatePictureList,
5          deliverPictureList: this.taskDetailData.deliverPictureList,
6          id: this.taskDetailData.id
7      })
8      this.loading.close()
9
10     promptAction.showToast({ message: '交货成功' })
11     this.getTaskDetail()
12 }
13

```

Fence 181

```

1      Button("交付", { type: ButtonType.Capsule })
2          .backgroundColor(this.getDeliverState() ? $r('app.color.primary')
: $r('app.color.primary_disabled'))
3          .fontColor($r('app.color.white'))
4          .height(50)
5          .flexGrow(1)
6          .enabled(this.getDeliverState())
7          .margin({ left: 13 })
8          .onClick(() => {
9              this.onDeliver()
10          })

```

Fence 182

:::info
 提交
 ...

底部显示回车登记按钮

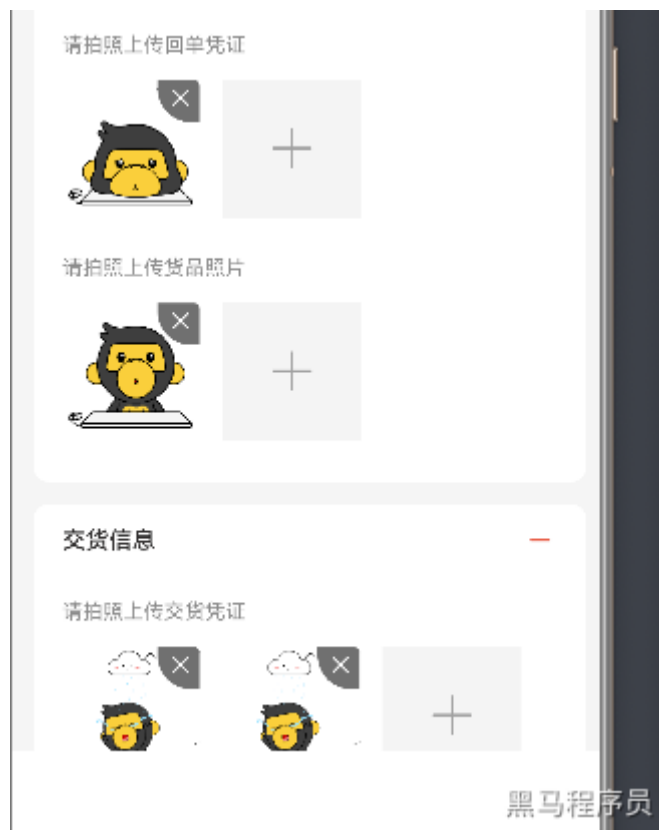
:::warning

在已经交货的情况下，显示**提货信息-交货信息- 底部**的**回车登记按钮**，**全部显示**

:::

```
1      // 回车时也要显示提货照片
2      if(this.taskDetailData.status === TaskTypeEnum.Waiting ||
this.taskDetailData.status === TaskTypeEnum.Delivered) {
3          HmToggleCard({ title: '提货信息' }) {
4              this.getPickUpContent()
5          }
6      }
7      // 回车时也要显示交货照片
8      if(this.taskDetailData.status === TaskTypeEnum.Line ||
this.taskDetailData.status === TaskTypeEnum.Delivered) {
9          HmToggleCard({ title: '交货信息' }) {
10             this.getDeliverContent()
11         }
12     }
```

Fence 183



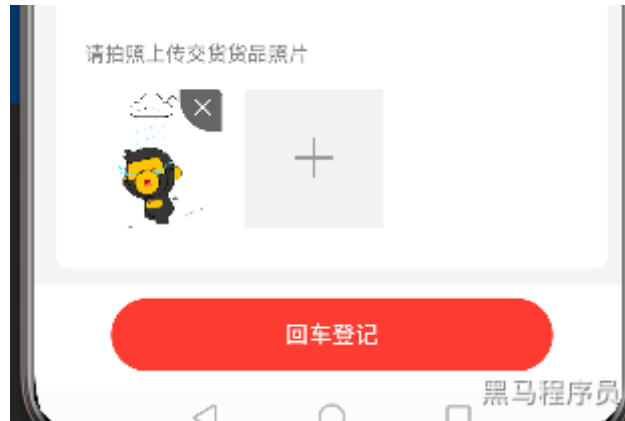
- 显示底部回车登记按钮

```

1   else if(this.taskDetailData.status === TaskTypeEnum.Delivered) {
2       Row() {
3           // 已交付显示回车登记
4           Button("回车登记", { type: ButtonType.Capsule })
5               .backgroundColor($r('app.color.primary'))
6               .fontColor($r('app.color.white'))
7               .height(50)
8               .width('80%')
9       }
10      .width('100%').justifyContent(FlexAlign.Center)
11  }

```

Fence 184



:::info
 提交代码
 :::

回车登记模式下-上传组件只读

:::info

因为货已经交了，所以回车登记模式下，此时只可以看，不能再传图片了

:::

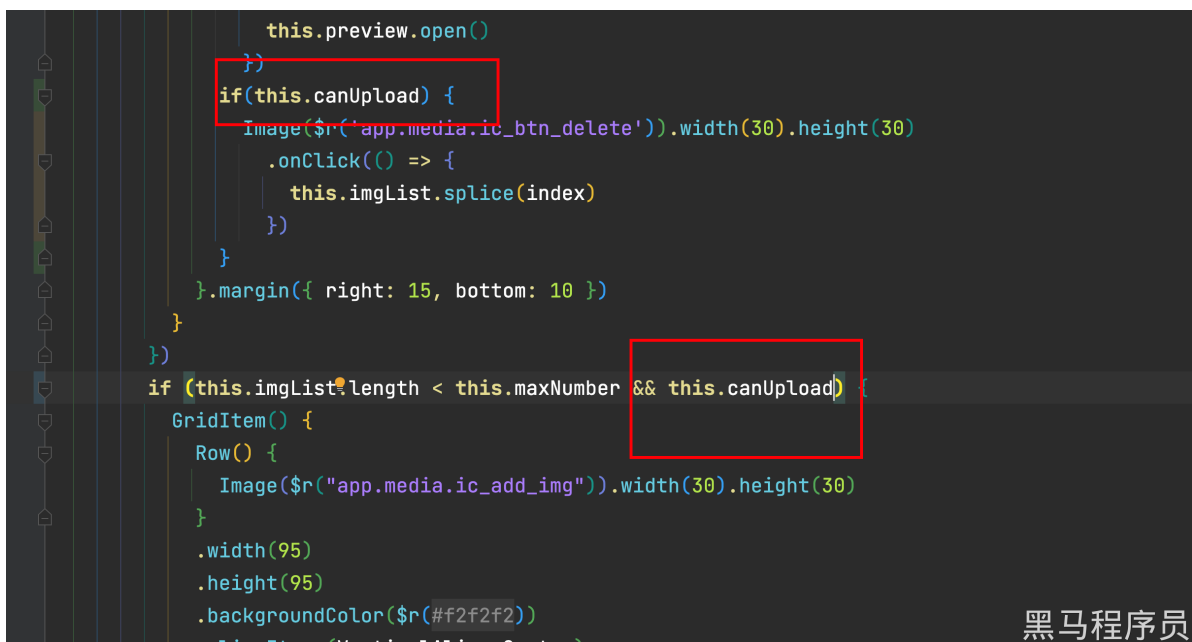
basic模块

1. 给HmUpload一个属性来控制是否可上传和删除

```
1 @Prop
2 canUpload:boolean = true
```

Fence 185

2. 通过该属性来控制上传部分的显示和隐藏



黑马程序员

3. 在TaskDetail中传入该属性

```
1
2 // 专门负责提货信息的结构
3 @Builder
4 getPickUpContent() {
5     HmUpload({
6         title: '请拍照上传货品凭证',
7         canUpload: this.taskDetailData.status !== TaskTypeEnum.Delivered,
8         imageUrl: this.taskDetailData.cargoPickUpPictureList || [],
9         onImageListChange: (list: ImageList[]) => {
10             this.taskDetailData.cargoPickUpPictureList = list
11         }
12     })
13     HmUpload({
14         title: '请拍照上传货品照片',
15         canUpload: this.taskDetailData.status !== TaskTypeEnum.Delivered,
16         imageUrl: this.taskDetailData.cargoPictureList || [],
17         onImageListChange: (list: ImageList[]) => {
18             this.taskDetailData.cargoPictureList = list
19         }
20     })
21 }
```

```

20     })
21 }
22
23 // 专门负责交货信息的结构
24 @Builder
25 getDeliverContent() {
26     HmUpload({
27         title: '请拍照上传交货货品凭证',
28         canUpload: this.taskDetailData.status !== TaskTypeEnum.Delivered,
29         imageUrl: this.taskDetailData.deliverPictureList || [],
30         onImageListChange: (list: ImageList[]) => {
31             this.taskDetailData.deliverPictureList = list
32         }
33     })
34     HmUpload({
35         title: '请拍照上传交货货品照片',
36         canUpload: this.taskDetailData.status !== TaskTypeEnum.Delivered,
37         imageUrl: this.taskDetailData.certificatePictureList || [],
38         onImageListChange: (list: ImageList[]) => {
39             this.taskDetailData.certificatePictureList = list
40         }
41     })
42 }
43

```



:::info

提交代码

:::

回车登记页面

- 新建回车登记页面 pages/CardRecord/Record

9:41

回车登记

出车时间2022.05.04 13:00

回车时间请选择>

车辆违章

☐

是

☒

否

车辆故障

☐

是

☒

否

车辆事故

☐

是

☒

否

交车

黑马程序员

```
1 import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
2 @Entry
3 @Component
```

```

4  struct CarRecord {
5      build() {
6          column() {
7              HmNavBar({ title: '回车登记' })
8              scroll() {
9                  column() {
10                     HmCard(){
11                         HmCardItem({
12                             leftText: '出车时间',
13                             rightText: '2022.05.04 13:00'
14                         })
15                         HmCardItem({
16                             leftText: '回车时间',
17                             rightText: '请选择',
18                             showBottomBorder: false
19                         })
20                     }
21                 }
22                 .height('100%')
23             }
24             .layoutweight(1)
25             // 底部内容
26             Row() {
27                 Button("交车",{ type: ButtonType.Capsule })
28                 .backgroundColor($r('app.color.primary'))
29                 .width(207)
30                 .height(50)
31             }
32             .backgroundColor($r('app.color.white'))
33             .height(70)
34             .width('100%')
35             .justifyContent(FlexAlign.Center)
36             .alignItems(VerticalAlign.Center)
37         }
38         .backgroundColor($r('app.color.background_page'))
39         .height('100%')
40     }
41 }

```




回车登记

出车时间

2022.05.04 13:00 >

回车时间

请选择 >

交车

黑马程序员

- TaskDetail页面跳转到回车登记

```
1      Button("回车登记", { type: ButtonType.Capsule })
2      .backgroundColor($r('app.color.primary'))
3      .fontColor($r('app.color.white'))
4      .height(50)
5      .width('80%')
6      .onClick(() => {
7          router.pushUrl({
8              url: 'pages/CarRecord/CarRecord',
9              params: {
10                 id: this.taskDetailData.id
11             }
12         })
13     })
```

Fence 188

- 在CarRecord接收参数id, 并且获取任务的详情, 赋值出车时间

```
1      @State taskDetailData: TaskDetailInfoModel = {} as TaskDetailInfoModel
2
3      async aboutToAppear() {
4          await this.getTaskDetail()
5      }
6
7      async getTaskDetail() {
8          const params = router.getParams() as CommonRouterParams
9          if (params.id) {
10             const result = await getTaskDetailAPI(params.id)
11             this.taskDetailData = result
12         }
13     }
```

Fence 189

- 赋值出车时间

```
1      HmCardItem({
2          leftText: '出车时间',
3          - rightText: '2022.05.04 13:00'
4          + rightText: this.taskDetailData.actualDepartureTime
5      })
```

Fence 190

<

回车登记

出车时间

2024-01-03 22:01:09 >

回车时间

请选择 >

黑马程序员

:::info
提交代码
:::

使用官方的时间弹窗选择器来选择时间

- 使用官方的选择时间弹窗

```
1      HmCardItem({ leftText: '回车时间',
2                    rightText: "请选择",
3                    showBottomBorder: false,
4                    onRightClick: () => {
5                        DatePickerDialog.show({
6                            showTime: true, // 展示日期的同时，是否显示时间
7                            useMilitaryTime: true, // 是否显示为24小时制
8                            onDateAccept: (value: Date) => {
9                                AlertDialog.show({ message: value.toString(), alignment:
DialogAlignment.Center })
10                            }
11                        })
12
13                    }
14
15                })
```

Fence 191

- 下包 ohpm i dayjs

```
1 | import dayjs from 'dayjs'
```

Fence 192

- 回车时间绑定对应的字段

```
1      HmCardItem({
2          leftText: '回车时间',
3          rightText: "请选择",
4          showBottomBorder: false,
5          onRightClick: () => {
6              DatePickerDialog.show({
7                  showTime: true,
8                  useMilitaryTime: true,
9                  onDateAccept: (value: Date) => {
10                      // 结束时间
11                      - AlertDialog.show({ message: value.toString(), alignment:
DialogAlignment.Center })
12                      + AlertDialog.show({ message: dayjs(value).format('YYYY-MM-DD
HH:mm'), alignment: DialogAlignment.Center })
13                  }
14              })
15
16          }
17
18      })
```

Fence 193

-



回车登记

出车时间

回车时间

请选择 >

车辆违章

☐ 是 ☒ 否

车辆故障

☐ 是 ☒ 否

车辆事故

☐ 是 ☒ 否

2024年4月27日星期六 ▾

4月25日

15

48

4月26日

16

49

4月27日

17

50

4月28日

18

51

4月29日

19

52

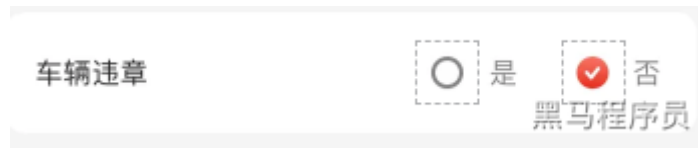
取消

确定

黑马程序员

封装选择组件-HmCheckBox

basic模块



分析

- 可以控制显示文本
 - 右侧的是否文本可以控制
 - 值改变时通知外部组件
 - 这个组件通用性不是那么大，简单封装一下
- 在components下新建HmCheckBox.ets - 直接复制

```
1  @Preview
2  @Component
3  struct HmCheckBox {
4      title: string = "测试"
5      confirmText: string = "是"
6      cancelText: string = '否'
7      @Prop
8      value: boolean = true // 决定选中左侧还是右侧\
9      checkChange: (value: boolean) => void = () => {}
10     build() {
11         Row () {
12             Row () {
13                 Text(this.title)
14                     .fontSize(14)
15                     .fontColor($r("app.color.text_primary"))
16
17                 // 右侧内容
18                 Row ({ space: 10 }) {
19                     Row () {
20                         Image(this.value ? $r("app.media.ic_radio_true") :
21 $r("app.media.ic_radio_false"))
22                             .width(32)
23                             .aspectRatio(1)
24                         Text(this.confirmText)
25                     }
26                     .onClick(() => {
27                         this.value = true
28                         this.checkChange(this.value)
29                     })
30                     Row () {
31                         Image(!this.value ? $r("app.media.ic_radio_true") :
32 $r("app.media.ic_radio_false"))
33                             .width(32)
34                             .aspectRatio(1)
35                         Text(this.cancelText)
36                     }
37                     .onClick(() => {
```

```

36         this.value = false
37         this.checkChange(this.value)
38     })
39     }
40 }
41 .width("100%")
42 .borderRadius(10)
43 .height(60)
44 .padding({
45     left: 15,
46     right: 15
47 })
48 .justifyContent(FlexAlign.SpaceBetween)
49 .backgroundColor($r("app.color.white"))
50
51 }
52 .width("100%")
53 .padding({
54     left: 15,
55     right: 15
56 })
57 .margin({
58     top: 15
59 })
60 }
61 }
62 export { HmCheckBox }

```

Fence 194

- 在components/index.ets中导出

```

1 export * from './HmCheckBox'

```

Fence 195

- 在回车登记中放置三个组件

```

1     Scroll() {
2         Column() {
3             HmCard() {
4                 HmCardItem({
5                     leftText: '出车时间',
6                     rightText: this.taskDetailData.actualDepartureTime
7                 })
8                 HmCardItem({
9                     leftText: '回车时间',
10                    rightText: '请选择',
11                    showBottomBorder: false
12                })
13            }
14
15            + HmCheckBox({ value: false, title: '车辆违章' })
16            + HmCheckBox({ value: false, title: '车辆故障' })
17            + HmCheckBox({ value: false, title: '车辆事故' })
18        }
19        .height('100%')

```

```
20 | }  
21 | .layoutweight(1)
```

Fence 196

entry模块

<

回车登记

出车时间 >

回车时间 请选择 >

车辆违章

☐ 是 ☒ 否

车辆故障

☐ 是 ☒ 否

车辆事故

☐ 是 ☒ 否

黑马程序员

提交代码

回车登记提交

1. 定义提交数据类型
2. 绑定数据到数据
3. 提交回车登记

entry模块

- 声明回车登记类型

```
1 import { ImageListModel } from '@hm/basic'
2 export interface CarRecordTypeModel {
3   /** 事故说明，类型为“其他”时填写 */
4   accidentDescription: string | null;
5   /** 事故图片列表 */
6   accidentImagesList: ImageListModel[] | null;
7   /** 事故类型，1-直行事故，2-追尾事故，3-超车事故，4-左转弯事故，5-右转弯事故，6-弯道事
  故，7-坡道事故，8-会车事故，9-其他，*/
8   accidentType: string | null;
9   /** 违章说明，类型为“其他”时填写 */
10  breakRulesDescription: string | null;
11  /** 违章类型，1-闯红灯，2-无证驾驶，3-超载，4-酒后驾驶，5-超速行驶，6-其他，可用 */
12  breakRulesType: string | null;
13  /** 扣分数数据 */
14  deductPoints: number | null;
15  /** 回车时间，回车时间，格式yyyy-MM-dd HH:mm:ss，比如：2023-07-18 17:00:00 */
16  endTime: string;
17  /** 故障说明，类型为“其他”时填写 */
18  faultDescription: string | null;
19  /** 故障图片列表 */
20  faultImagesList: ImageListModel[] | null;
21  /** 故障类型，1-发动机启动困难，2-不着车，3-漏油，4-漏水，5-照明失灵，6-有异响，7-排
  烟异常，8-温度异常，9-其他，可用 */
22  faultType: string | null;
23  /** 运输任务id */
24  id: string;
25  /** 是否出现事故 */
26  isAccident: boolean | null;
27  /** 车辆是否可用 */
28  isAvailable: boolean | null;
29  /** 车辆是否违章 */
30  isBreakRules: boolean | null;
31  /** 车辆是否故障 */
32  isFault: boolean | null;
33  /** 罚款金额 */
34  penaltyAmount: string | null;
35  /** 出车时间，出车时间，格式yyyy-MM-dd HH:mm:ss，比如：2023-07-18 17:00:00 */
36  startTime: string;
37 }
38
```

Fence 197

- 在models/index.ets中导出

```
1 | export * from './car_record'
```

Fence 198

- 在CarRecord中定义数据

```
1 | @State
2 | carRecord: CarRecordTypeModel = {} as CarRecordTypeModel
```

Fence 199

- 绑定车辆违章-车辆故障-车辆事故的属性

```
1 | HmCardItem({
2 |   leftText: '回车时间',
3 |   - rightText: '请选择',
4 |   + rightText: this.carRecord.endTime || '请选择',
5 |   showBottomBorder: false,
6 |   onRightClick: () => {
7 |     DatePickerDialog.show({
8 |       showTime: true,
9 |       useMilitaryTime: true,
10 |      onDataAccept: (value: Date) => {
11 |        + this.carRecord.endTime = dayjs(value).format('YYYY-MM-DD
HH:mm')
12 |      }
13 |    })
14 |  }
15 | })
16 |
17 | // ... 省略其余代码...
18 |
19 |
20 | HmCheckBox({ value: !!this.carRecord.isBreakRules, title: '车辆违章',
21 | +   checkChange: (value) => {
22 | +     this.carRecord.isBreakRules = value
23 | +   }
24 | })
25 | HmCheckBox({ value: !!this.carRecord.isFault, title: '车辆故障',
26 | +   checkChange: (value) => {
27 | +     this.carRecord.isFault = value
28 | +   }
29 | })
30 | HmCheckBox({ value: !!this.carRecord.isAccident, title: '车辆事故',
31 | +   checkChange: (value) => {
32 | +     this.carRecord.isAccident = value
33 | +   }
34 | })
```

Fence 200

- 封装交车api

```
1 | /** 交车API */
2 | export const carRecordAPI = (data: CarRecordTypeModel) => {
3 |   return Request.post("/driver/tasks/truckRegistration", data)
4 | }
```

id string 运输任务id 必需

startTime string 出车时间 必需

出车时间，格式yyyy-MM-dd HH:mm:ss，比如：2023-07-18 17:00:00

endTime string 回车时间 必需

回车时间，格式yyyy-MM-dd HH:mm:ss，比如：2023-07-18 17:00:00

黑马程序员

有三个字段是必须要传的，id来自任务详情中的transportTaskId，startTime来自任务详情中的actualDepartureTime，endTime是我们自己选择的时间

- 提交

```

1 // 检查再提交
2 async onCarRecord() {
3   if (!this.carRecord.endTime) {
4     promptAction.showToast({ message: '请选择回车时间' })
5     return
6   }
7   this.carRecord.startTime = this.taskDetailData.actualDepartureTime
8   // 运输任务id
9   this.carRecord.id = this.taskDetailData.transportTaskId // 注意注意再注意
10  await carRecordAPI(this.carRecord)
11  promptAction.showToast({ message: '回车登记成功' })
12  router.clear() // 清空页面栈
13  router.replaceUrl({ url: 'pages/Index/Index' })
14 }

```

同学们到现在，我们已经将神领物流中的 提货-交货-回车登记业务跑通，剩下的将完成 对整个项目的一些核心点的优化

实现延迟收货业务-基本布局

entry模块

9:41

<

延迟提货

原定时间

2022.05.04 13:00

延迟时间

不可超过2个小时>

请输入延迟提货原因

0/50

提交

黑马程序员

1. 新建pages/Delay/Delay.ets -(Page)

- 复制静态-快速布局

```

1 import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
2 import dayjs from "dayjs";
3
4
5 @Entry
6 @Component
7 struct Delay {
8   build() {
9     column() {
10       HmNavBar({ title: '延迟提货' })
11       HmCard() {
12         HmCardItem({ leftText: '原定时间', rightText: '', showRightIcon:
false })
13         HmCardItem({
14           leftText: '延迟时间',
15           rightText: '',
16           onRightClick: () => {
17             DatePickerDialog.show({
18               useMilitaryTime: true,
19               showTime: true,
20               onDateAccept: (value) => {
21                 AlertDialog.show({
22                   message: dayjs(value).format('YYYY-MM-DD HH:mm'),
23                   alignment: DialogAlignment.Center
24                 })
25               }
26             })
27           }
28         })
29         TextArea({ placeholder: '请输入延迟提货原因' })
30           .backgroundColor($r('app.color.background_page'))
31           .margin({ top: 20 })
32           .borderRadius(8)
33           .height(130)
34           .placeholderColor($r('app.color.text_secondary'))
35           .fontSize(14)
36         Text('0/50')
37           .margin({ top: -30 })
38           .textAlign(TextAlign.End)
39           .width('100%')
40           .padding({ right: 15 })
41           .fontColor($r('app.color.text_secondary'))
42         Row() {
43           Button("提交")
44             .height(50)
45             .width(207)
46             .backgroundColor($r('app.color.primary_disabled'))
47         }
48         .justifyContent(FlexAlign.Center).padding({
49           top: 20,
50           bottom: 20
51       })
52     }
53   }
54 }

```

```
55 | .height('100%').backgroundColor($r('app.color.background_page'))
56 | }
57 | }
```

Fence 203

:::info

提交代码

:::

延迟收货-接收参数-显示

:::info

TaskDeail中点击延迟收货跳转到延迟收货，传入参数

:::

1. 点击延迟收货，跳转，传入参数

这里偷个懒直接把要的参数id和时间传递过去，利用原来封装的公共路由参数

basic模块

```
1 export interface CommonRouterParamsModel {
2   id?: string = ''
3   oldTime?: string
4 }
```

Fence 204

entry模块

```
1 Button("延迟收货", { type: ButtonType.Capsule })
2   .backgroundColor($r('app.color.btn_gray'))
3   .fontColor($r('app.color.text_primary'))
4   .fontSize(16)
5   .height(50)
6   .width(125)
7   .onClick(() => {
8     router.pushUrl({
9       url: 'pages/Delay/Delay',
10      params: {
11        id: this.taskDetailData.id,
12        oldTime: this.taskDetailData.planDepartureTime
13      }
14    })
15  })
```

Fence 205

2. 在延迟收货接收该参数

- 定义延迟提货提交类型参数

```
1 export interface DelayParamTypeModel {
2   /** 延迟原因 */
3   delayReason: string;
4   /** 延迟时间，格式: yyyy-MM-dd HH:mm */
5   delayTime: string;
6   /** 司机作业单id */
7   id: string;
8 }
```

Fence 206

- 在models/index.ets中导出

```
1 export * from './delay'
```

- 接收赋值id和原有时间

Fence 207

```
1  @State
2  delayForm: DelayParamsTypeModel = {} as DelayParamsTypeModel
3
4  @State oldTime: string = ""
5
6  aboutToAppear() {
7      const params = router.getParams() as CommonRouterParams
8      if(params.id && params.oldTime) {
9          this.delayForm.id = params.id
10         this.oldTime = params.oldTime
11     }
12 }
```

Fence 208

- 赋值原有时间

```
1  HmCardItem({
2      leftText: '原定时间',
3      rightText: this.oldTime,
4      showRightIcon: false
5  })
6
```

Fence 209



延迟提货

原定时间

2023-11-17 20:00:00

延迟时间



请输入延迟提货原因

0/50

提交

黑马程序员

:::info
提交代码
:::

延迟收货-日期选择

- 右侧点击弹出

```
1      HmCardItem({
2          leftText: '延迟时间',
3      +   rightText: this.delayForm.delayTime || '',
4          onRightClick: () => {
5              DatePickerDialog.show({
6                  useMilitaryTime: true,
7                  showTime: true,
8                  onDateAccept: (value) => {
9      +               this.delayForm.delayTime =dayjs(value).format('YYYY-MM-DD
      HH:mm')
10             }
11         })
12     }
13 })
```



...info
提交代码
...

延迟收货-字数控制

1. 声明状态

```
1  @State
2  maxSizeNumber:number = 50
3
```

Fence 211

2. 双向绑定原因字段

3. 监听onChange事件

:::warning
\$\$不支持嵌套对象里面的属性绑定
:::

```
1  TextArea({
2    placeholder: '请输入延迟提货原因',
3    + text: this.delayForm.delayReason,
4  })
5  + .maxLength(this.maxSizeNumber)
6  + .onChange((value) => {
7    + this.delayForm.delayReason = value
8  + })
9
```

Fence 212

4. 设置显示文本

```
1  - Text('0/50')
2  + Text(`${this.delayForm.delayReason?.length ||
3    0}/${this.maxSizeNumber}`)
4    .margin({ top: -30 })
5    .textAlign(TextAlign.End)
6    .width('100%')
7    .padding({ right: 15 })
8    .fontColor($r('app.color.text_secondary'))
```

Fence 213

>

50/50

黑马程序员

• • •

• • •

控制按钮状态及颜色

1. 封装方法检查必填项

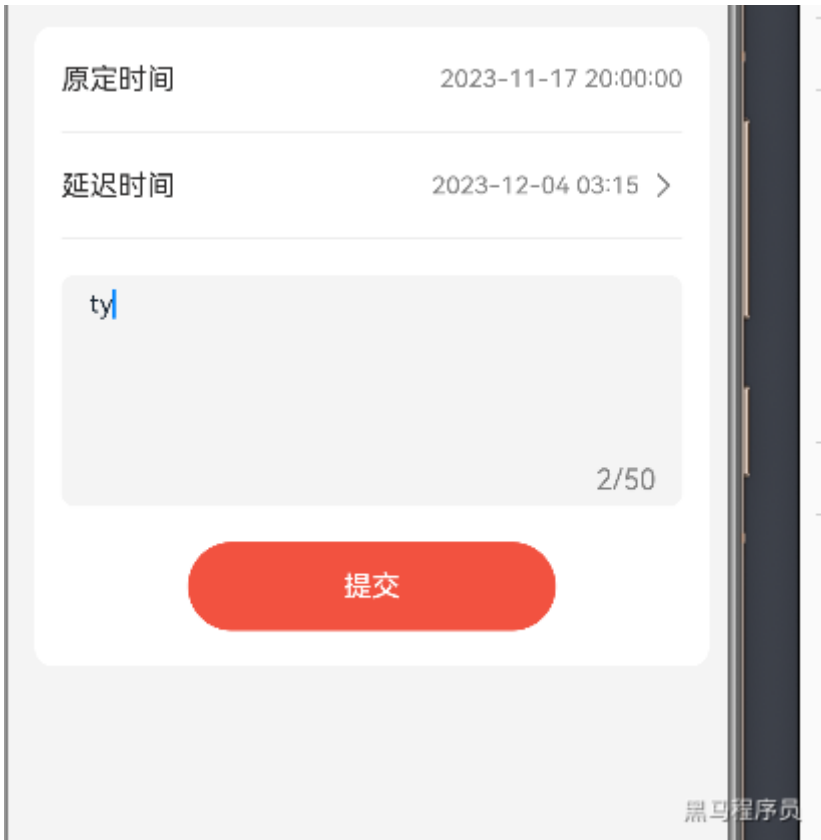
```
1 getBtnEnable() {  
2     !!this.delayForm.delayReason && !!this.delayForm.delayTime  
3 }
```

Fence 214

2. 控制按钮及颜色

```
1 Button("提交")  
2     .height(50)  
3     .width(207)  
4     .backgroundColor($r('app.color.primary'))  
5     .enabled(this.getBtnEnable())  
6     .onClick() => {  
7  
8     }
```

Fence 215



:::info
提交代码
:::

提交延迟收货

1. 封装api

```
1 // 延迟收货
2 export const delayAPI = (data: DelayParamsTypeModel) => {
3   return Request.put("/driver/tasks/delay", data)
4 }
```

Fence 216

2. 调用api, 返回页面

```
1 async onDelay() {
2   await delayAPI(this.delayForm)
3   promptAction.showToast({ message: '延迟收货成功' })
4   router.back()
5 }
```

Fence 217

:::info

提交代码

:::

上报异常页面基础布局

entry模块



:::info
:::

上报异常页面选择时间

entry模块

- 点击上报异常进入-TaskDetail

```
1      Button("上报异常", { type: ButtonType.Capsule })
2          .backgroundColor($r('app.color.btn_gray'))
3          .fontColor($r('app.color.text_primary'))
4          .fontSize(16)
5          .height(50)
6          .width(125)
7          .onClick(() => {
8              router.pushUrl({
9                  url: 'pages/Exception/ExceptionReport',
10                 params: {
11                     id: this.taskDetailData.transportTaskId
12                 }
13             })
14         })
```

Fence 218

- 声明数据类型-新建models/exception.ets

```
1 import { ImageListModel } from '@hm/basic'
2 export interface ExceptionParamsTypeModel {
3     /** 异常描述, 200字以内 */
4     exceptionDescribe: string | null;
5     /** 异常图片 */
6     exceptionImagesList: ImageListModel[] | null;
7     /** 上报异常位置 */
8     exceptionPlace: string;
9     /** 异常时间, 精确到分钟 */
10    exceptionTime: string;
11    /** 异常类型(传中文), 发动机启动困难, 不着车, 漏油, 漏水, 照明失灵, 有异响, 排烟异常, 温度异常, 其他 */
12    exceptionType: string;
13    /** 运输任务id */
14    transportTaskId: string;
15 }
16
```

Fence 219

- 在models/index.ets中导出

```
1 export * from './exception'
```

Fence 220

新建上报异常页面-复制静态-pages/ExceptionReport/ExceptionReport.ets

```
1 import { CommonRouterParams, HmCard, HmCardItem, HmNavBar, HmUpload } from
  '@hm/basic'
2 import dayjs from 'dayjs'
```

```

3 import { ExceptionParamTypeModel } from '../models'
4 import { router } from '@kit.ArkUI'
5
6 @Entry
7 @Component
8 struct ExceptionReport {
9   @State
10   exceptionForm: ExceptionParamTypeModel = {} as ExceptionParamTypeModel
11
12   aboutToAppear() {
13     const params = router.getParams() as CommonRouterParams
14     if (params.id) {
15       this.exceptionForm.transportTaskId = params.id
16     }
17   }
18
19   build() {
20     column() {
21       HmNavBar({ title: '上报异常' })
22       scroll() {
23         column() {
24           HmCard() {
25             HmCardItem({
26               leftText: '异常时间', rightText:
this.exceptionForm.exceptionTime || '请选择',
27               onRightClick: () => {
28                 DatePickerDialog.show({
29                   showTime: true,
30                   useMilitaryTime: true,
31                   onDateAccept: (value) => {
32                     this.exceptionForm.exceptionTime =
dayjs(value).format('YYYY-MM-DD HH:mm')
33                   }
34                 })
35               }
36             })
37             HmCardItem({ leftText: '上报位置', rightText: '请选择' })
38             HmCardItem({ leftText: '异常类型', rightText: '请选择' })
39             HmCardItem({
40               leftText: '异常描述',
41               rightText: '',
42               showRightIcon: false,
43               showBottomBorder: false
44             })
45             TextArea({
46               placeholder: '请输入异常描述'
47
48             }).height(130).borderRadius(8).placeholderColor($r('app.color.text_secondary'))
49               .fontSize(14)
50               Text(`0/50`)
51               .margin({
52                 top: -30
53               })
54               .textAlign(TextAlign.End)
55               .width('100%')

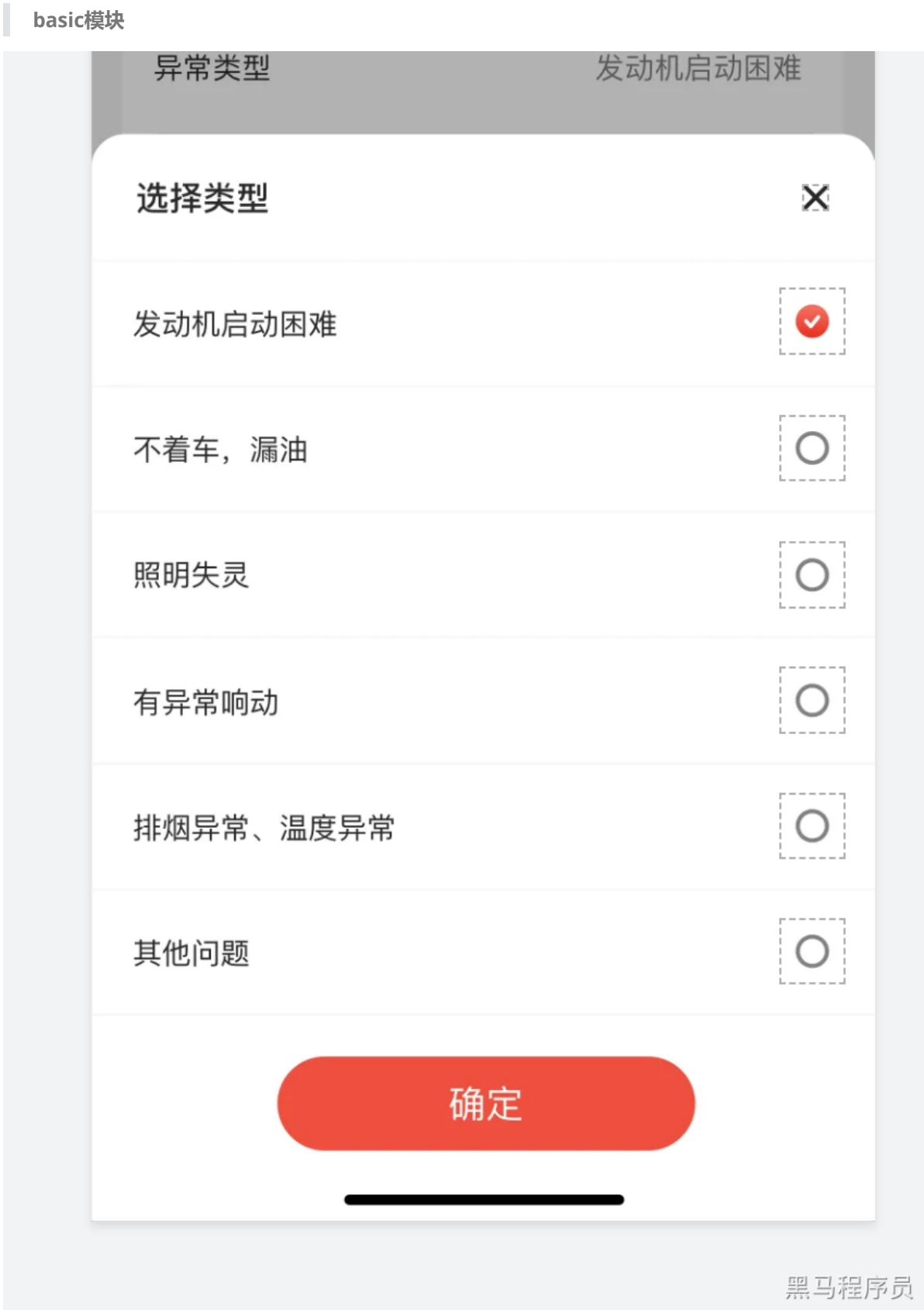
```

```

54         .padding({ right: 15 })
55         .fontColor($r('app.color.text_secondary'))
56         Row().height(20)
57
58     }
59
60     HmCard() {
61         HmUpload({
62             title: '上传图片(最多6张)',
63             imageUrl: [],
64             , canUpload: true
65         })
66         Row().height(20)
67     }
68 }
69 }.padding({
70     bottom: 80
71 })
72 .layoutweight(1)
73
74
75 Row() {
76     Button("提
交").height(50).width(207).backgroundColor($r('app.color.primary_disabled'))
77 }
78 .position({
79     y: '100%'
80 })
81 .height(70)
82 .translate({
83     y: -70
84 })
85 .width('100%')
86 .justifyContent(FlexAlign.Center)
87 .backgroundColor($r('app.color.white'))
88 }
89 .height('100%').backgroundColor($r('app.color.background_page'))
90 }
91 }

```

封装HmSelectCard组件



- 顶部内容可以设置标题，可以设置关闭按钮显示隐藏
 - 可以设置顶部按钮显示隐藏，可以设置底部按钮的文本和颜色
 - 可以自定义传入的结构
 - 支持自定义Dialog的形式弹出
- 在components下新建HmSelectCard.ets组件

HmSelectCard

请选择



确定

黑马程序员

粘贴静态结构

```
1  @Preview
2  @CustomDialog
3  @Component
4  struct HmSelectCard {
5      controller: CustomDialogController
6      build() {
7          Column() {
8              Row() {
9                  Text("请选择").fontSize(16).fontColor($r('app.color.text_primary'))
10                 Image($r("app.media.ic_btn_close")).width(13).height(13)
11             }
12             .width('100%')
13             .justifyContent(FlexAlign.SpaceBetween)
14             .alignItems(VerticalAlign.Center)
15             .height(60)
16             .borderRadius({
17                 topLeft: 16,
18                 topRight: 16
19             })
20             // 渲染内容
21
22             Row() {
23                 Button('确定', { type: ButtonType.Capsule
24                 }).width(200).backgroundColor($r('app.color.primary')).height(45)
25                 .width('100%').justifyContent(FlexAlign.Center)
26             }.backgroundColor($r('app.color.white')).justifyContent(FlexAlign.SpaceBetween).padding({
27                 left: 21.36,
28                 right: 21.36
29             })
30         }
31     }
32     export { HmSelectCard }
```

Fence 222

- 定义开放属性，用于设置组件的内容

```

1 title: string = "请选择" // 显示标题
2 showClose: boolean = true // 显示关闭按钮
3 showButton: boolean = true // 显示底部按钮
4 buttonText: string = "确定" // 底部按钮文本
5 confirm: () => void = () => {}
6 @BuilderParam
7 cardContent: () => void

```

Fence 223

- 控制内容显示

```

1 @Preview
2 @CustomDialog
3 @Component
4 struct HmSelectCard {
5     controller: CustomDialogController
6     title: string = "请选择" // 显示标题
7     showClose: boolean = true // 显示关闭按钮
8     showButton: boolean = true // 显示底部按钮
9     buttonText: string = "确定" // 底部按钮文本
10    confirm: () => void = () => {}
11    @BuilderParam
12    cardContent: () => void
13    build() {
14        Column() {
15            Row() {
16
17                Text(this.title).fontSize(16).fontColor($r('app.color.text_primary'))
18                    if (this.showClose) {
19                        Image($r("app.media.ic_btn_close")).width(13).height(13)
20                            .onClick(() => {
21                                this.controller.close()
22                            })
23                    }
24                .width('100%')
25                .justifyContent(FlexAlign.SpaceBetween)
26                .alignItems(VerticalAlign.Center)
27                .height(60)
28                .borderRadius({
29                    topLeft: 16,
30                    topRight: 16
31                })
32
33                // 渲染内容
34                if(this.cardContent) {
35                    this.cardContent()
36                }
37                if (this.showButton) {
38                    Row() {
39                        Button(this.buttonText, { type: ButtonType.Capsule })
40                            .width(200)
41                            .backgroundColor($r('app.color.primary'))
42                            .height(45)
43                            .onClick(() => {

```

```

44         this.confirm()
45     })
46     }.width('100%').justifyContent(FlexAlign.Center)
47 }
48
49 }.backgroundColor($r('app.color.white')).justifyContent(FlexAlign.SpaceBetween).padding({
50     left: 21.36,
51     right: 21.36
52 })
53 }
54
55 export { HmSelectCard }

```

Fence 224

在components/index.ets导出

```

1 | export * from './HmSelectCard'

```

Fence 225

entry模块

在上报异常中使用

```

1 | // 异常类型弹层
2 | typeDialog: CustomDialogController = new CustomDialogController({
3 |     builder: HmSelectCard({ cardContent: () => { this.getCardContent } }),
4 |     autoCancel: false,
5 |     customStyle: true,
6 |     alignment: DialogAlignment.Bottom
7 | })
8 | @Builder
9 | getCardContent () {}

```

Fence 226

- 这里必须得使用箭头函数来给cardContent赋值，否则在当前组件中无法正确使用this

点击右侧内容打开

```

1 | HmCardItem({ leftText: '异常类型', rightText: '请选择', onRightClick: () => {
2 |     this.typeDialog.open()
3 | } })

```

Fence 227



:::info
提交
:::

渲染异常内容数据

发动机启动困难	
不着车，漏油	
照明失灵	
有异常响动	
排烟异常、温度异常	
其他问题	

黑马程序员

- 定义循环项

```
1  @State
2  exceptErrorList: string[] = ["发动机启动困难", "不着车、漏油", "照明失灵", "有异常
3  响动", "排烟异常、温度异常", "其他问题"]
```

Fence 228

- 使用一个轻量Builder来实现单个的渲染

```
1  // 单个的渲染选项
2  @Builder
3  getSingleItem() {
4      Row() {
5          Text("发动机漏
6          油").fontSize(14).fontWeight(400).fontColor($r('app.color.text_primary'))
7          Row() {
8              Image($r("app.media.ic_check_true")).width(32).height(32)
9          }
10     }
```

```

9      }
10     .justifyContent(FlexAlign.SpaceBetween)
11     .alignItems(VerticalAlign.Center)
12     .border({
13         width: {
14             bottom: 1
15         },
16         color: $r('app.color.background_divider')
17     })
18     .width('100%')
19     .height(60)
20 }
21

```

Fence 229

- 根据数据动态生成N个的选项

```

1  @Builder
2  getCardContent() {
3      ForEach(this.exceptionErrorList, () => {
4          this.getSingleItem()
5      })
6  }

```

Fence 230

- 定义一个class用来给Builder传递参数

```

1  class ExceptionItemClass {
2      name: string = ""
3      index: number = 0
4  }

```

Fence 231

- 给Builder传递class参数

```

1  @Builder
2  getSingleItem(item: string, index: number, showBorder: boolean) {
3      Row() {
4          Text(item)
5              .fontSize(14)
6              .fontColor($r("app.color.text_primary"))
7          Image(this.selectIndex === index ? $r("app.media.ic_check_true") :
8              $r("app.media.ic_check_false"))
9              .width(32)
10             .height(32)
11      }
12      .onClick(() => {
13          this.selectIndex = index // 赋值索引
14      })
15      .height(60)
16      .width("100%")
17      .justifyContent(FlexAlign.SpaceBetween)
18      .border({
19          color: $r("app.color.background_divider"),
20          width: {

```

```
20         bottom: showBorder ? 1 : 0
21     }
22 })
23 }
24
25 // 传入HmSelectCard组件
26 @Builder
27 getCardContent() {
28     ForEach(this.exceptionList, (item: string, index: number) => {
29         this.getSingleItem(item, index, index != this.exceptionList.length -
30     1)
31     })
32 }
```

异常时间

请选择 >

上报位置

请选择 >

异常类型

请选择 >

异常描述

请选择



发动机启动困难



不着车、漏油



照明失灵



有异常响动



排烟异常、温度异常





确定

黑马程序员

- 定义一个状态索引，根据该索引判断当前选项是否被选中

```
1 @State
2 selectedIndex: number = 0
```

Fence 233

```
1 @Builder
2   getSingleItem(item: string, index: number, showBorder: boolean) {
3     Row() {
4       Text(item)
5         .fontSize(14)
6         .fontColor($r("app.color.text_primary"))
7       Image(this.selectedIndex == index ? $r("app.media.ic_check_true") :
8         $r("app.media.ic_check_false"))
9         .width(32)
10        .height(32)
11     }
12     .onClick(() => {
13       this.selectedIndex = index // 赋值索引
14     })
15     .height(60)
16     .width("100%")
17     .justifyContent(FlexAlign.SpaceBetween)
18     .border({
19       color: $r("app.color.background_divider"),
20       width: {
21         bottom: showBorder ? 1 : 0
22       }
23     })
24   }
25   // 传入HmSelectCard组件
26   @Builder
27   getCardContent() {
28     ForEach(this.exceptionList, (item: string, index: number) => {
29       this.getSingleItem(item, index, index != this.exceptionList.length -
30     1)
31   })
32 }
```

Fence 234

请选择



发动机启动困难



不着车、漏油



照明失灵



有异常响动



排烟异常、温度异常



其他问题



确定

黑马程序员

:::info

提交代码

:::

选择类型

- 点击确定时，将选择的类型赋值给当前的类型属性

```
1 // 选择类型的弹层
2 selectTypeDialog: CustomDialogController = new CustomDialogController({
3     builder: HmSelectCard({
4         cardContent: () => {
5             // 必须调用一个builder的函数
6             this.getCardContent()
7         },
8         confirm: () => {
9             if (this.selectIndex > -1) {
10                 this.exceptionForm.exceptionType =
11                 this.exceptionList[this.selectIndex]
12                 this.selectTypeDialog.close()
13             }
14         },
15         customStyle: true,
16         alignment: DialogAlignment.Bottom
17     })
```

Fence 235

- 绑定数据到CardItem组件

```
1 HmCardItem({ leftText: '异常类型', rightText: this.exceptionForm.exceptionType
2 || '请选择', showBottomBorder: false,
3     onRightClick: () => {
4         this.typeDialog.open()
5     })
```

Fence 236

<

上报异常

异常时间

请选择 >

上报位置

请选择 >

异常类型

其他问题 >

异常时间

请输入异常描述

请选择异常类型

×

发动机启动困难

☐

不着车、漏油

☐

照明失灵

☒

有异常响动

☐

排烟异常、温度异常

☐

其他问题

☐

确定

黑马程序员

:::info
提交代码
:::

·上报异常-跳转当前位置页面

<

上报异常

异常时间

请选择 >

上报位置

请选择 >

异常类型

其他问题 >

异常时间

请输入异常描述

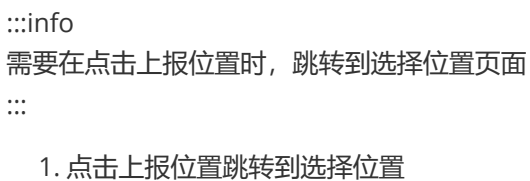
0/50

上传图片(最多6张)

+

提交





需要在点击上报位置时，跳转到选择位置页面

1. 点击上报位置跳转到选择位置

```

1  HmCardItem({
2    leftText: '上报位置',
3    rightText: '请选择',
4    onRightClick: () => {
5      router.pushUrl({
6        url: 'pages/SelectLocation/SelectLocation'
7      })
8    })

```

Fence 237

2. 新建pages/SelectLocation/SelectLocation页面

```

1  import { HmNavBar, HmCard, HmCardItem } from '@hm/basic'
2  import { MapComponent } from '@hms.core.map.MapComponent'
3
4  @Entry
5  @Component
6  struct SelectLocation {
7    build() {
8      column() {
9        HmNavBar({ title: '当前位置' })
10       Stack({ alignContent: Alignment.Bottom }) {
11         // 地图区域
12         MapComponent({
13           mapOptions: {
14             position: {
15               target: {
16                 latitude: 39.9,
17                 longitude: 116.4
18               },
19               zoom: 10
20             }
21           },
22           mapCallback: () => {}
23         })
24         .width('100%')
25         .height('100%')
26       column() {
27         HmCard() {
28           HmCardItem({ leftText: '回龙观街道', rightText: '100m以内' })
29           HmCardItem({ leftText: '金燕龙科研楼', rightText: '100m以内' })
30           HmCardItem({ leftText: '春野画室', rightText: '100m以内' })
31           HmCardItem({ leftText: '光滑时代(金艳龙科研楼店)', rightText: '100m
以内' })
32         }
33       }
34       .padding({
35         bottom: 60
36       })
37     }
38   }
39   .height('100%').backgroundColor($r('app.color.background_page'))
40 }
41 }

```



:::info
提交代码
:::

配置地图

官网文档: <https://developer.huawei.com/consumer/cn/doc/app/agc-help-harmonyos-releaseapp-0000001126380068>

按照微信案例中的签名规则配置如下签名

p12

p7b

cer

csr



黑马程序员

- 在agc中开启地图使用

常规

API管理

Server SDK

数据处理位置

项目套餐

项目配额

项目费用

定位服务

混合定位帮助您应用快速精准获取用户位置信息。[展开](#)

位置服务

帮助用户更加方便的使用位置相关的服务。[展开](#)

地图服务

助力全球开发者实现个性化地图呈现与交互，全面提升您应用的LBS 体验。[展开](#)

- 在module.json5中配置client_id

黑马程序员

应用

SDK配置: 下载最新的配置文件 (如果您修改了项目、应用信息或者更改了某个开发服务设置, 可能需要更新该文件)

⬇

agconnect-services.json

☐ 不包含密钥 ?

包名: com.ltheima.fastdriver

APP ID: 5765880207854187755

SHA256证书/公钥指纹: [添加证书指纹](#) [添加公钥指纹 \(HarmonyOS API 9及以上\)](#)

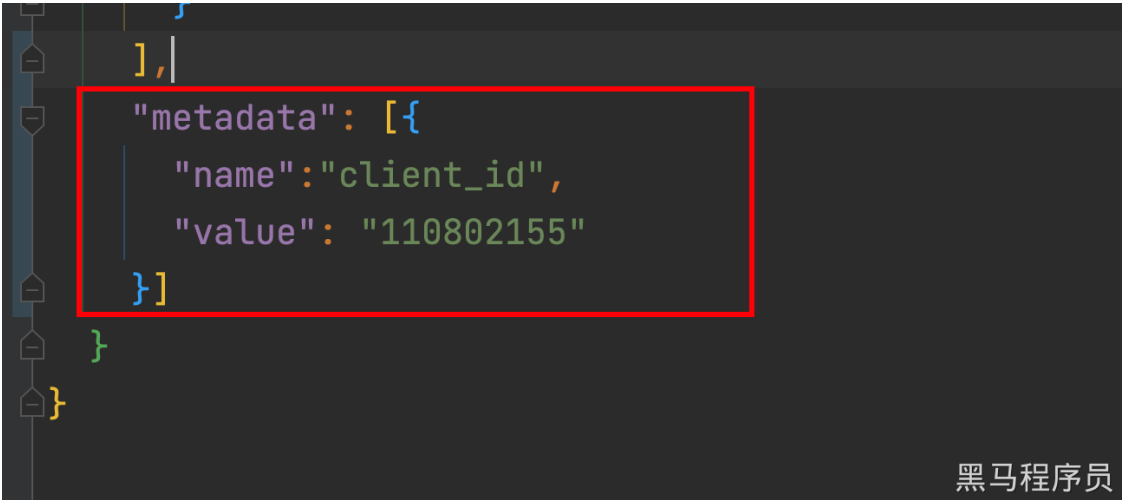
OAuth 2.0客户端ID (凭据): Client ID 110802155

Client Secret e7f8d8040936e915dc54e596dd962068a28a638da1f3d1fcea9...

回调地址:

删除应用

黑马程序员



黑马程序员

添加公钥指纹

⬇

agconnect-services.json

☐ 不包含密钥 ?

包名: com.ltheima.fastdriver

APP ID: 5765880207854187755

SHA256证书/公钥指纹: [添加证书指纹](#) [添加公钥指纹 \(HarmonyOS API 9及以上\)](#)

OAuth 2.0客户端ID (凭据): Client ID 110802155

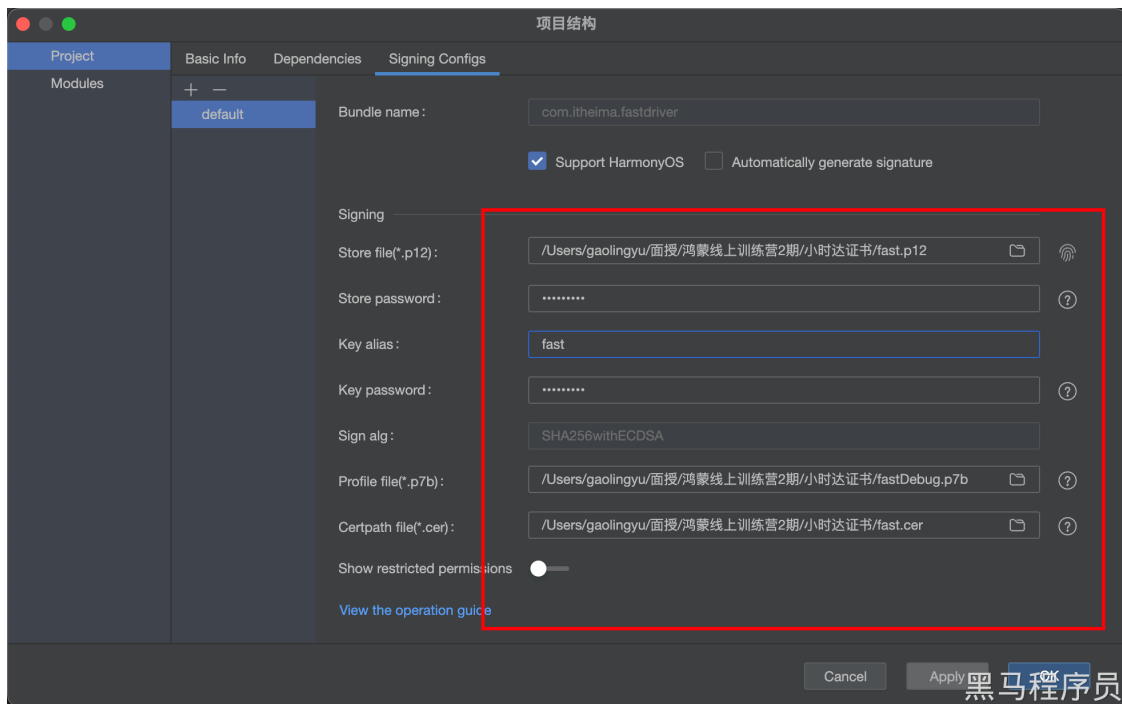
Client Secret e7f8d8040936e915dc54e596dd962068a28a638da1f3d1fcea9...

回调地址:

删除应用

黑马程序员

配置手动签名





获取当前位置的经纬度

[官方位置文档](#)

- 引入经纬度管理

```
1 import geoLocationManager from '@ohos.geoLocationManager';  
2
```

Fence 239

- 初始化时获取经纬度

```
1  aboutToAppear() {  
2      this.getLocation()  
3  }  
4  async getLocation () {  
5      try {  
6          const result = await geoLocationManager.getCurrentLocation()  
7          AlertDialog.show({  
8              message: JSON.stringify(result)  
9          })  
10     }catch (error) {  
11         AlertDialog.show({  
12             message: error.message  
13         })  
14     }  
15 }  
16 }
```

Fence 240

...info

没反应???



- 获取用户的地理位置**必须经过用户同意**-需要在初始化UIAbility的时候就发起请求
- 并且需要在module.json5中配置地址位置的权限
- ...
- module.json5

```
1  {
2      "name": "ohos.permission.LOCATION",
3      "reason": "$string:LOCATION_REASON",
4      "usedScene": {
5          "abilities": ["EntryAbility"],
6          "when": "always"
7      }
8  },
9  {
```

```

10     "name": "ohos.permission.APPROXIMATELY_LOCATION",
11     "reason": "$string:LOCATION_REASON",
12     "usedScene": {
13         "abilities": ["EntryAbility"],
14         "when": "always"
15     }
16 },

```

Fence 241

- 在string.json中填写原因

```

1  {
2      "name": "LOCATION_REASON",
3      "value": "申请地理位置"
4  }

```

Fence 242

- src/main/ets/entryability/EntryAbility.ts

```

1  import abilityAccessCtrl from '@ohos.abilityAccessCtrl'
2
3
4  async onCreate(want: Want, launchParam: AbilityConstant.LaunchParam):
    Promise<void> {
5      hilog.info(0x0000, 'testTag', '%{public}s', 'Ability onCreate');
6      // 创建管理访问控制模块的实例
7      let manger = abilityAccessCtrl.createAtManager()
8      // 向用户申请访问授权
9      await manger.requestPermissionsFromUser(this.context,
10         [
11             'ohos.permission.LOCATION',
12             'ohos.permission.APPROXIMATELY_LOCATION',
13         ])
14     }

```

Fence 243

```
{"latitude":40,"longitude":116,"altitude":43.5,"accuracy":5,"speed":0,"timeStamp":1701676043,"direction":45,"timeSinceBoot":123425476384560,"additions":"","additionSize":0,"isFromMock":false}
```

回龙观街道 100m以内 >

金燕龙科研楼 100m以内 >

春野画室 100m以内 >

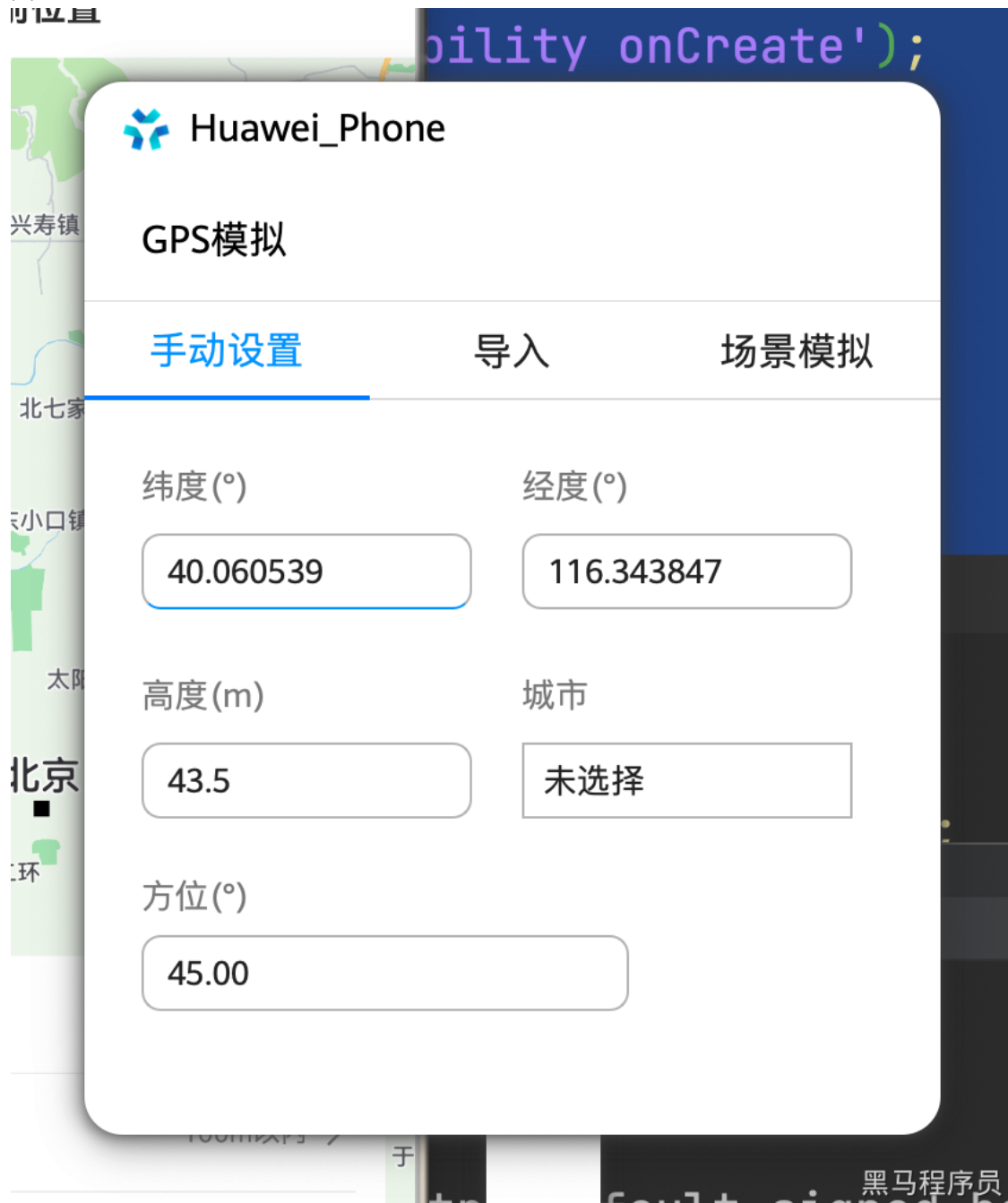
光滑时代(金艳龙科研楼店) 100m以内 >

黑马程序员

因为模拟器没有真实的位置，想要给一个真实的地址的话可以在模拟器位置心里填入一个经纬度如

图

图位置

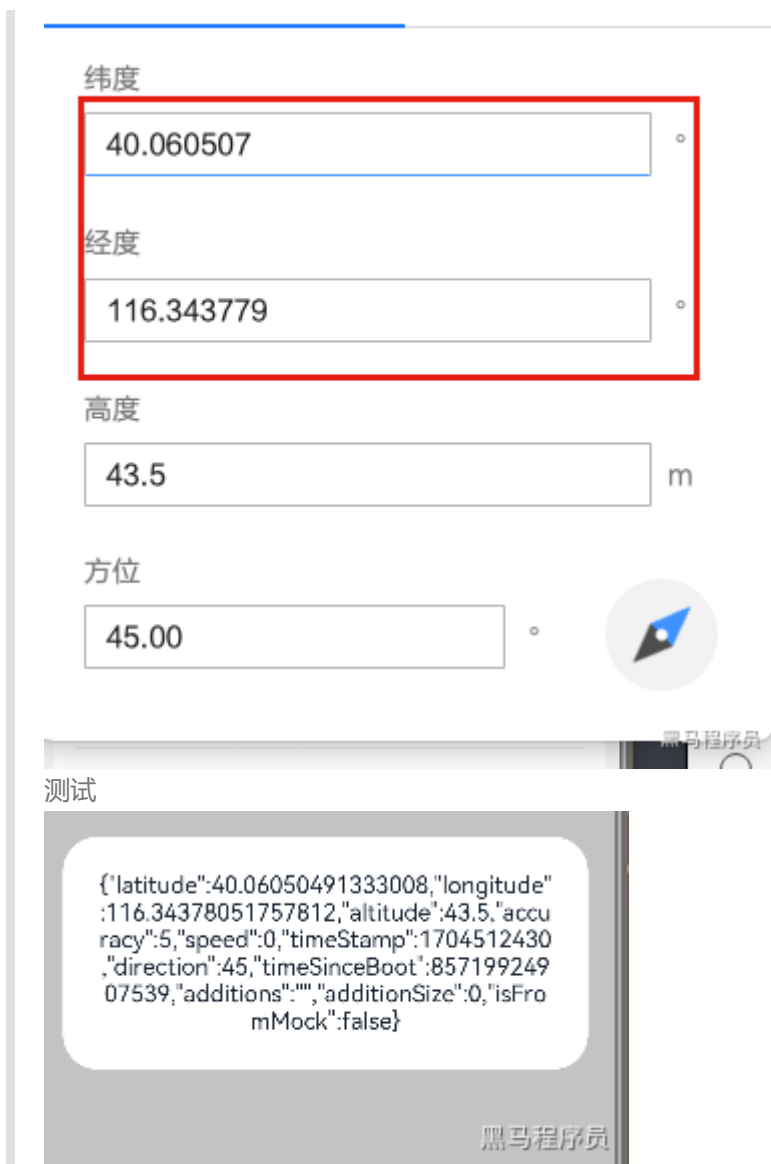


高德地图坐标拾取器



填入

黑马程序员



:::info

提交代码

:::

转化经纬度坐标

坐标系知识介绍

华为地图涉及2种坐标系：

- WGS84：一种大地坐标系，也是目前广泛使用的GPS全球卫星定位系统使用的坐标系。
- GCJ02：由中国国家测绘局制订的地理信息系统的坐标系统，是由WGS84坐标系经过加密后的坐标系。

华为地图使用的坐标类型 [🔗](#)

在国内站点，中国大陆、中国香港和中国澳门使用GCJ02坐标系，中国台湾和海外使用WGS84坐标系。

在海外站点，统一使用WGS84坐标系。

黑马程序员

我们需要将GCJ02转成WGS84

您可以通过[map](#)命名空间下的[convertCoordinate](#)方法进行坐标转换：

- 转化代码

```
1 import { map, mapCommon } from '@kit.MapKit';
2 let wgs84Position: mapCommon.LatLng = {
3   latitude: 30,
4   longitude: 118
5 };
6 let gcj02Posion: mapCommon.LatLng = await
  map.convertCoordinate(mapCommon.CoordinateType.WGS84,
    mapCommon.CoordinateType.GCJ02,wgs84Position);
```

Fence 244

突发!!!

目前获取的地理定位华为改为了GCJ02, 如果我们是显示在花瓣地图上，我们不需要再转化了
高德地图的定位其实需要逆向转化。

获取controller之后设置当前位置

- 声明一个controller

```
1 mapController: map.MapComponentController = new map.MapComponentController()
2
```

Fence 245

- 回调赋值controller

```
1 MapComponent({
2     mapOptions: {
3         position: {
4             target: {
5                 latitude: 39.9,
6                 longitude: 116.4
7             },
8             zoom: 10
9         }
10    },
11    mapCallback: (err, controller) => {
12        if(!err) {
13            this.mapController = controller
14            this.getLocation()
15        }
16    }
17 })
```

Fence 246

- 获取位置转化坐标-调整照相机位置

```
1 async getLocation() {
2     try {
3         const rightPostion = await geoLocationManager.getCurrentLocation()
4
5         // 通过控制器移动焦点到用户的经纬度位置
6         this.mapController.moveCamera(map.newCameraPosition({
7             target: {
8                 longitude: rightPostion.longitude,
9                 latitude: rightPostion.latitude
10            },
11            zoom: 16 // 缩放级别
12        }))
13
14        // 添加一个标记
15        this.mapController.addPointAnnotation({
16            position: {
17                longitude: rightPostion.longitude,
18                latitude: rightPostion.latitude
19            },
20            titles: [{
21                content: '您当前的位置'
22            }],
23        })
24    }
25 }
```



```

23     })
24   } catch (error) {
25     AlertDialog.show({
26       message: error.message
27     })
28   }
29
30 }

```

Fence 247



根据坐标点搜索周围地址

- 声明一个class

```
1 class SiteClass {
2     name: string = ""
3     distance: number = 0
4 }
```

Fence 248

- 获取方法

```
1 // site 是@kit.MapKit提供的一个模块
2
3 const res = await site.nearbySearch({
4     location: {
5         longitude: rightResult.longitude,
6         latitude: rightResult.latitude
7     },
8     pageSize: 4,
9     pageIndex: 1,
10    radius: 50
11 })
12 this.list = res.sites?.slice(0,4) as SiteClass[] // 只拿4条数据
```

Fence 249

- 声明状态变量

```
1 @State
2 list: SiteClass[] = []
```

Fence 250

- 赋值变量

```
1 const res = await site.nearbySearch({
2     location: {
3         longitude: rightResult.longitude,
4         latitude: rightResult.latitude
5     },
6     pageSize: 4,
7     pageIndex: 1,
8     radius: 50
9 })
10 this.list = res.sites as SiteClass[] // 只拿4条数据
```

Fence 251

- 循环地址

```

1      ForEach(this.list, (item: SiteClass) => {
2          HmCardItem({ leftText: item.name, rightText: `${item.distance}m`
3      })
4          .onClick(() => {
5              router.back({
6                  url: "pages/ExceptionReport/ExceptionReport",
7                  params: {
8                      location: item.name
9                  }
10             })
11         })
12     })

```

Fence 252



点击返回地址

- 点击周围地址，返回地址参数

```
1      HmCardItem({ leftText: item.name, rightText:
  `${item.distance}m` })
2      .onClick(() => {
3          router.back({
4              url: "pages/ExceptionReport/ExceptionReport",
5              params: {
6                  location: item.name
7              }
8          })
9      })
```

Fence 253

4. 在上报异常中获取数据

basic模块

- 在公共路由参数中再添加一个address字段

```
1 export interface CommonRouterParams {
2     id?: string
3     oldTime?: string
4     location?: string
5 }
```

Fence 254

entry模块

```
1 - aboutToAppear
2 + onPageShow(): void {
3     const params = router.getParams() as CommonRouterParams
4 +     if (params && params.location) {
5 +         this.exceptionForm.exceptionPlace = params.location
6 +     }
7
8     if (params && params.id) {
9         this.exceptionForm.transportTaskId = params.id
10    }
11 }
12
```

Fence 255

5. 显示位置到CardItem组件上

```
1 HmCardItem({
2   leftText: '上报位置',
3   rightText: this.exceptionForm.exceptionPlace || '请选择',
4   onRightClick: () => {
5     router.pushUrl({
6       url: 'pages/SelectLocation/SelectLocation'
7     })
8   })
9 })
```

Fence 256



:::info

提交代码

:::

处理异常图片的赋值-控制按钮

- 上传图片的赋值

```
1      HmUpload({
2          title: '上传图片(最多6张)',
3          maxNumber: 6,
4          canUpload: true,
5          imageUrl: this.exceptionForm.exceptionImagesList || [],
6          onImageListChange: (list: ImageList[]) => {
7              this.exceptionForm.exceptionImagesList = []
8          }
9      })
```

Fence 257

- 处理描述的双向绑定

```
1      maxNumber: number = 50
```

Fence 258

```
1      TextArea({
2          placeholder: '请输入异常描述',
3          text: this.exceptionForm.exceptionDescribe!
4      })
5
6      .height(130).borderRadius(8).placeholderColor($r('app.color.text_secondary'))
7      .fontSize(14).onChange((value) => {
8          this.exceptionForm.exceptionDescribe = value
9      }).maxLength(this.maxNumber)
10     Text(`${this.exceptionForm.exceptionDescribe?.length ||
11     0}/${this.maxNumber}`)
12     .margin({
13         top: -30
14     })
15     .textAlign(TextAlign.End)
16     .width('100%')
17     .padding({ right: 15 })
18     .fontColor($r('app.color.text_secondary'))
```

Fence 259

- 处理按钮状态

```
1      getBtnEnable () {
2          return !(this.exceptionForm.exceptionDescribe &&
3              this.exceptionForm.exceptionPlace &&
4              this.exceptionForm.exceptionTime &&
5              this.exceptionForm.exceptionType &&
6              this.exceptionForm.exceptionDescribe)
7      }
```

Fence 260

- 绑定按钮

```

1 Row() {
2   Button("提交")
3     .height(50)
4     .width(207)
5     .backgroundColor( $r('app.color.primary') )
6     .enabled(this.getBtnEnable())
7 }

```

Fence 261



:::info
 提交代码
 :::

上报异常提交

1. 封装api

```
1 // 上报异常
2 export const exceptionReportAPI = (data: ExceptionParamsTypeModel) => {
3   return Request.post("/driver/tasks/reportException", data)
4 }
```

Fence 262

2. 按钮点击提交-返回上一个页面

```
1 async btnReport () {
2   await exceptionReportAPI(this.exceptionForm)
3   promptAction.showToast({
4     message: '上报异常成功'
5   })
6   router.back()
7 }
```

Fence 263



:::info

- 提交代码
- ...

根据上报异常数据进行显示



:::info

如果任务详情中，有上报异常数据，则显示控制

:::

- 直接拷贝现成的结构

```
1 // 获取异常信息
2 @Builder
3 getExceptionContent() {
4     ForEach(this.taskDetailData.exceptionList, (item: ExceptionListModel) =>
5     {
6         Row() {
7             Column() {
8                 Row() {
9                     Text("上报时间").fontSize(14).fontColor($r('app.color.text_primary'))
10                     Text(item.exceptionTime).margin({ left: 20
11                     }).fontColor($r('app.color.text_secondary'))
12                     }.height(50).alignItems(VerticalAlign.Center).width('100%')
13                 Row() {
14                     Text("异常类型").fontSize(14).fontColor($r('app.color.text_primary'))
15                     Text(item.exceptionType).margin({ left: 20
16                     }).fontColor($r('app.color.text_secondary'))
17                     }.height(50).alignItems(VerticalAlign.Center).width('100%')
18                 Row() {
19                     Text("处理结果").fontSize(14).fontColor($r('app.color.text_primary'))
20                     Text("继续运输").margin({ left: 20
21                     }).fontColor($r('app.color.text_secondary'))
22                     }.height(50).alignItems(VerticalAlign.Center).width('100%')
23                 }
24                 // 跳转到详情
25                 Image($r("app.media.ic_btn_more")).width(24).height(24)
26             }
27             .width('100%')
```

```

26     .padding({ left: 15, right: 15 })
27     .alignItems(VerticalAlign.Center)
28     .justifyContent(FlexAlign.SpaceBetween)
29
30     })
31 }

```

Fence 264

- 放置到任务详情的内容区

```

1 // ⚠️ 初始数据为空 记得加可选链
2 if (this.taskDetailData.exceptionList?.length > 0) {
3     HmToggleCard({ title: '异常信息' }) {
4         this.getExceptionContent()
5     }
6 }

```

Fence 265



```

:::info
提交代码
:::

```

上报完异常-回到详情加载异常信息

entry模块

1. 在上报异常页面传递一个标记，表示已经加了一条数据

```
1  async btnReport() {
2      await exceptionReport(this.exceptionForm)
3      promptAction.showToast({
4          message: '上报异常成功'
5      })
6      router.back({
7          url: 'pages/TaskDetail/TaskDetail',
8          params: {
9              addExcept: true
10         }
11     })
12 }
```

Fence 266

在路由的公共参数添加一个属性add_except的属性

basic模块

```
1  export interface CommonRouterParams {
2      id?: string
3      oldTime?: string
4      location?: string
5      addExcept?: boolean
6  }
```

Fence 267

2. 在TaskDetail的onPageShow中判断此字段，然后重新加载数据，

注意!!! 只重新赋值异常的信息部分，否则之前上传的图片会被干掉，因为只有异常发生了变化，所以不能执行原来的获取执行逻辑

```
1  async onPageShow() {
2      const params = router.getParams() as CommonRouterParams
3      if(params && params.addExcept) {
4          const result = await getTaskDetailAPI(this.taskDetailData.id)
5          this.taskDetailData.exceptionList = result.exceptionList
6      }
7  }
```

Fence 268

异常信息

上报时间 2023-12-04 07:59:00

异常类型 有异常响动 >

处理结果 继续运输

上报时间 2023-12-04 07:42:00

异常类型 发动机启动困难 >

处理结果 继续运输

黑马程序员

:::info

提交代码

:::

回显上报异常组件



需求

点击右侧的箭头-需要回到异常详情页去显示内容

** 因为没有查询异常详情的接口，我们可以把点击当前行的整行数据传到详情页**

basic模块

- 在公共路由参数添加一个新的参数

```
1
2 export class CommonRouterParams {
3   id?: string = ''
4   oldTime?: string = ''
5   address?: string = ''
6   addExcept?: boolean = false
7   formData?: object
8 }
```

Fence 269

entry模块

- 新建pages/Exception/ExceptionDetail.ets

这里做过多操作已经无意，直接拷贝下方结构即可

- 直接拷贝结构

```
1 import { HmNavBar, HmCardItem, HmCard, CommonRouterParams, ImageListModel }
  from '@hm/basic'
2 import router from '@ohos.router'
3 import { ExceptionListModel } from '../models'
4
5 @Entry
6 @Component
```

```

7 struct ExceptDetail {
8   @State submitForm: ExceptionListModel = {} as ExceptionListModel
9   aboutToAppear() {
10     const params = router.getParams() as CommonRouterParams
11     if (params && params.formData) {
12       // 检查formData
13       this.submitForm = params.formData as ExceptionListModel
14     }
15   }
16   @Builder
17   getCardChildren() {
18     HmCardItem({ leftText: '异常时间', rightText:
19 this.submitForm.exceptionTime, showRightIcon: false, })
20     HmCardItem({ leftText: '上报位置', rightText:
21 this.submitForm.exceptionPlace, showRightIcon: false, })
22     HmCardItem({ leftText: '异常类型', rightText:
23 this.submitForm.exceptionType, showRightIcon: false })
24     HmCardItem({ leftText: '异常描述', showBottomBorder: false,
25 showRightIcon: false, rightText: '' })
26     Row() {
27
28       Text(this.submitForm.exceptionDescribe).fontSize(14).fontColor($r('app.coolor.text_primary'))
29     }.padding(15).justifyContent(FlexAlign.Start).width('100%')
30   }
31   @Builder
32   getUpload() {
33     if (this.submitForm.exceptionImagesList?.length) {
34       Text("异常图片").width('100%').padding(10)
35       Flex({ wrap: FlexWrap.Wrap, direction: FlexDirection.Row }) {
36         ForEach(this.submitForm.exceptionImagesList, (item: ImageListModel)
37 => {
38           Image(item.url)
39             .width(95)
40             .height(95)
41             .borderRadius(4)
42             .margin({ right: 15 })
43         })
44       }
45       .width('100%').margin({ top: 16.5, bottom: 16.5 })
46     }
47   }
48
49   build() {
50     Column() {
51       HmNavBar({ title: '异常详情' })
52       HmCard() {
53         this.getCardChildren()
54       }
55       HmCard() {
56         this.getUpload()
57       }
58     }.height('100%').backgroundColor($r('app.coolor.background_page'))
59   }
60 }

```


55

56 `export default ExceptDetail`

Fence 270

2. 点击TaskDetail的右侧箭头跳转-传递formData

```
1
2      // 跳转到详情
3      Image($r("app.media.ic_btn_more")).width(24).height(24)
4      .onClick(() => {
5          router.pushUrl({
6              url: "pages/ExceptionReport/ExceptionDetail",
7              params: {
8                  formData: item
9              }
10         })
11     })
```

Fence 271



...info
提交代码
...

导航功能



:::info

只在在途的状态下显示导航

:::

```
1 // 在途情况下 才显示导航
2 if (this.taskDetailData.status === TaskTypeEnum.Line) {
3     column() {
4         Image($r("app.media.ic_navigation")).width(22).height(22)
5         Text("开始导航").fontSize(14).margin({ top: 10, bottom: 10 })
6     }.justifyContent(FlexAlign.SpaceBetween)
7     .margin({
8         top: 20
9     })
10 }
11
```

Fence 272

- 点击开始导航

```
1 // 获取基础信息
2 // 开始导航
3 async beginNav() {
4     try {
5         let context = getContext(this) as common.UIAbilityContext
6         context.startAbility({
7             action: 'ohos.want.action.viewData',
8             entities: ['entity.system.browsable'],
9             uri: encodeURI('https://gaode.com/search?query=' +
10 this.taskDetailData.endAddress)
11         })
12     } catch (error) {
13         AlertDialog.show({
14             message: JSON.stringify(error)
15         })
16     }
17 }
```

```
15     }  
16  
17     }
```

Fence 273

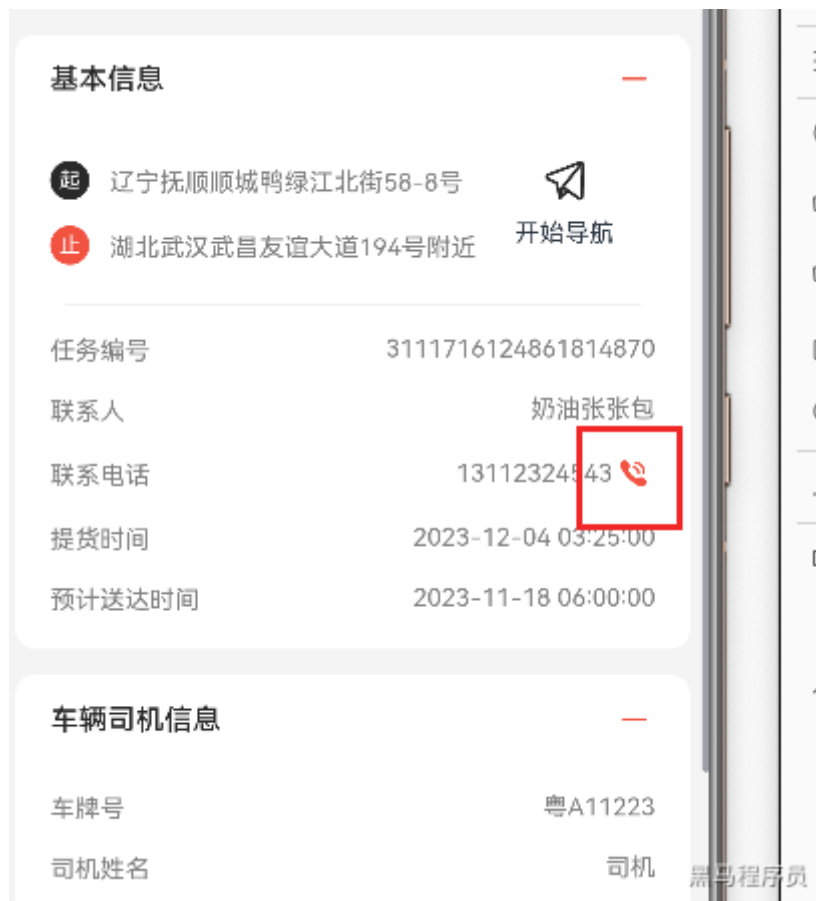
目前高德地图的导航调用方式无法得知，无法打开导航路线，只能用浏览器唤起打开高德页面，传入我们的地址

:::info

提交代码

:::

打电话功能



1. 导入使用包

```
1 import call from '@ohos.telephony.call'
2
```

Fence 274

- 给之前的builder类型添加一个图标点击事件

```
1 interface BaseBuilderClass {
2     title: string
3     value: string
4     icon?: ResourceStr
5     iconClick?: () => void
6 }
```

Fence 275

2. 点击图标打电话

```
1 @Builder
2 getBaseContentItem(item: BaseBuilderClass) {
3     Row() {
4
5         Text(item.title).fontSize(14).fontColor($r('app.color.text_secondary'))
6         .lineHeight(20)
7         Row() {
8
9             Text(item.value).fontSize(14).fontColor($r('app.color.text_secondary'))
```

```

8         if (item.icon) {
9             Image(item.icon).width(24).height(24)
10            .onClick(() => {
11                item.iconClick && item.iconClick()
12            })
13        }
14    }
15    }.justifyContent(FlexAlign.SpaceBetween).width('100%').margin({
16        top: 14
17    })
18 }

```

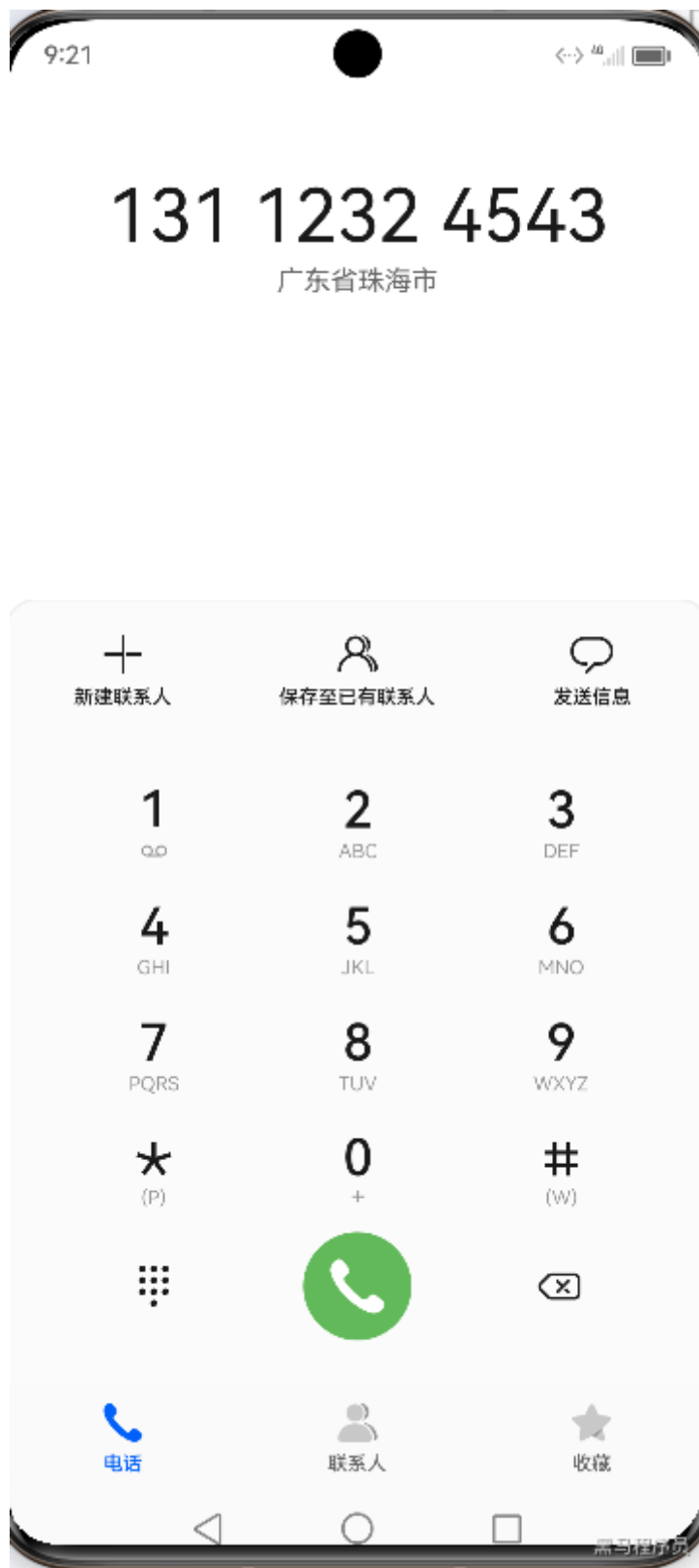
Fence 276

```

1  this.getBaseContentItem({
2      title: '联系电话',
3      value: this.taskDetailData.startHandoverPhone,
4      icon: $r('app.media.ic_phone'),
5      iconClick: () => {
6          call.makeCall(this.taskDetailData.startHandoverPhone);
7      }
8  })

```

Fence 277



:::info
提交代码
:::

已完成任务列表加入搜索条件结构



- 在TaskTabs中复用TaskList

```
1 else if(item.name === "finish") {
2   TaskList({
3     queryParams: {
4       page: 1,
5       pageSize: 5,
6       status: TaskTypeEnum.Finish
7     } as TaskListParamsModel
8   })
9 }
```

Fence 278

- 在TaskList中新增搜索条件

```
1
2 // 添加装饰器
3 @Prop
4 queryParams: TaskListParamsModel = {
5   page: 1, // 表示查询第几页的数据 ++
6   pageSize: 10, // 表示每页查几条数据
7   status: TaskTypeEnum.Waiting,
8 } as TaskListParamsModel
9
10 // 搜索条件builder
11 @Builder
12 getSearchForm() {
13   column() {
14     row() {
15       search({ placeholder: '请输入任务编号'
16     }).backgroundColor($r('app.color.background_page')).height(32)
17     }
18     .justifyContent(FlexAlign.Center)
19     .padding({ left: 15, right: 15, bottom: 5 })
20
21     row() {
22       // 完成搜索页需要测试点击之后键盘和弹层同时弹出的情况
23       button(this.queryParams.startTime || '开始时间')
24         .fontSize(14)
25         .width(106)
26         .height(32)
```



```

26         .padding({ left: 0, right: 0 })
27         .fontColor('#999')
28         .backgroundColor($r('app.color.background_page'))
29         .onClick(() => {
30             DatePickerDialog.show({
31                 selected: new Date(),
32                 onDateAccept: (value) => {
33                     this.queryParams.startTime = dayjs(value).format('YYYY-MM-
DD')
34                 }
35             })
36         })
37
38         Text("至")
39         Button(this.queryParams.endTime || '结束时间')
40             .fontSize(14)
41             .width(110)
42             .height(32)
43             .padding({ left: 0, right: 0 })
44             .fontColor('#999')
45             .backgroundColor($r('app.color.background_page'))
46             .onClick(() => {
47                 DatePickerDialog.show({
48                     selected: new Date(),
49                     onDateAccept: (value) => {
50                         this.queryParams.endTime = dayjs(value).format('YYYY-MM-DD')
51                     }
52                 })
53             })
54
55         Button("筛选")
56             .backgroundColor($r('app.color.primary'))
57             .height(32)
58             .width(60)
59
60     }.width('100%').alignItems(VerticalAlign.Center).justifyContent(FlexAlign.SpaceAround)
61     .backgroundColor($r('app.color.white'))
62     .padding(15)
63     .justifyContent(FlexAlign.Center)
64     .width('100%')
65 }

```

Fence 279

- TaskList中显示搜索表单

```

1 build() {
2     Column() {
3         if(this.queryParams.status === TaskTypeEnum.Finish) {
4             this.getSearchForm()
5         }
6         HmList({
7             onLoad: async () => {
8                 await this.getTaskList(true)

```

```

9      },
10     onRefresh: async () => {
11         await this.onRefresh()
12     },
13     dataSource: $taskListData,
14     renderItem: this.renderItem,
15     finished: this.allPage < this.queryParams.page,
16     finishText: '没啦没啦',
17     loadingText: '拼命加载中'
18 })
19     .layoutweight(1)
20
21 }
22 .height('100%')
23 }

```

Fence 280

- TaskItem-在已完成状态，不显示按钮

```

1  if (this.taskItem.status !== TaskTypeEnum.Finish) {
2      Button(this.getBtnText(), { type: ButtonType.Capsule })
3          .backgroundColor($r('app.color.primary'))
4          .fontColor($r("app.color.white"))
5          .fontSize(14)
6          .height(32)
7          .enabled(this.getBtnEnable())
8          .onClick(() => {
9              this.toPickUp()
10          })
11  }
12

```

Fence 281

- 点击卡片，当已完成时，可以进去查看详情

```

1  .onClick(() => {
2      if (this.taskItem.status === TaskTypeEnum.Finish) {
3          router.pushUrl({
4              url: 'pages/TaskDetail/TaskDetail',
5              params: {
6                  id: this.taskItem.id
7              }
8          })
9      }
10  })

```

Fence 282

- TaskDetail中当已完成时，可以展示图片

```

    }
    if (this.taskDetailData.status === TaskTypeEnum.Waiting
        || this.taskDetailData.status === TaskTypeEnum.Delivered
        || this.taskDetailData.status === TaskTypeEnum.Finish
    ) {
        HmToggleCard({ title: '提货信息' }) {
            this.getPickUpContent()
        }
    }
    // 交货时显示交货照片
    if (this.taskDetailData.status === TaskTypeEnum.Line
        || this.taskDetailData.status === TaskTypeEnum.Delivered
        || this.taskDetailData.status === TaskTypeEnum.Finish
    ) {
        HmToggleCard({ title: '交货信息' }) {
            this.getDeliverContent()
        }
    }
}

```

黑马程序员

- 当已完成时，不显示底部内容

```

1  if(this.taskDetailData.status !== TaskTypeEnum.Finish) {
2      this.getBottomBtn() // 底部按钮结构
3  }

```



:::info

提交代码

:::

已完成搜索-选择日期

entry模块

Q 请输入任务编号

开始时间

至

结束时间

筛选

任务编号: 3111716124861814870

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

任务编号: 3111716124860458266

起 辽宁抚顺顺城鸭绿江北街58-8号

2023年12月4日星期一

2021年 10月 2日

2022年 11月 3日

2023年 12月 4日

2024年 1月 5日

2025年 2月 6日

取消

确定

- 加入loading进度条

```
1 loading: CustomDialogController = new CustomDialogController({
2   builder: HmLoading({
3     title: '搜索查询中'
4   }),
5   customStyle: true,
6   autoCancel: false,
7   alignment: DialogAlignment.Center
8 })
```

Fence 284

控制能否点击，点击事件，查询数据

```
1 Button("筛选")
2   .backgroundColor( $r('app.color.primary'))
3   .height(32)
4   .width(60)
5   .enabled(!(this.queryParams.startTime &&
this.queryParams.endTime))
6   .onClick(async() => {
7     this.loading.open()
8     this.allPage = 1
9     this.queryParams.page = 1
10    await this.getTaskList(false)
11    this.loading.close()
12  })
```

Fence 285

待提货

在途

已完成

Q 请输入任务编号

2023-12-04

至

2023-12-04

筛选

任务编号: 3111716124861814870

起 辽宁抚顺顺城鸭绿江北街58-8号

止 湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

:::info

提交代码

:::

清空状态控制

目前我们发现一个问题，选择完开始时间和结束时间，没有办法取消了，所以我们定义一个状态来控制下

- 定义一个状态

```
1 | @State
2 | reset: boolean = false // 用于控制重置状态
```

Fence 286

- 如果查询完一次之后，将状态设置为true

```
1 | Button(this.reset ? "重置" : "筛选")
2 |     .backgroundColor(this.getSearchEnable() ? $r('app.color.primary')
: $r('app.color.primary_disabled'))
3 |     .height(32)
4 |     .width(60)
5 |     .enabled(this.getSearchEnable())
6 |     .onClick(async() => {
7 |         if(this.reset){
8 |             this.queryParams.startTime = ''
9 |             this.queryParams.endTime = ''
10 |         }
11 |
12 |         this.reset = !this.reset
13 |         this.loading.open()
14 |         this.allPage = 1
15 |         this.queryParams.page = 1
16 |         await this.getTaskList(false)
17 |         this.loading.close()
18 |     })
```

Fence 287

输入单号搜索

开始时间

至

结束时间

筛选

任务编号：3111716124861814870

起

辽宁抚顺顺城鸭绿江北街58-8号

止

湖北武汉武昌友谊大道194号附近

提货时间

2023-12-04 03:25:00

黑马程序员

输入单号不影响分页及其他，只覆盖数据，不影响下拉刷新的逻辑

- 注意参数传递不是id，是transportTaskId。任务编号显示的是id
- 如果有值，那么意味着即使查也只能查出一条，那么不能触发上拉加载
- 有值情况下，如果已经有开始时间和结束时间，要清空，
- 如果有没有值，要清空查询参数中的transportTaskId

```
1  Search({ placeholder: '请输入任务编号', value:
    this.queryParams.transportTaskId || ''
  }).backgroundColor($r('app.color.background_page')).height(32)
2    .onSubmit(async value => {
3      this.loading.open()
4      this.queryParams.page = 1
5      if(value) {
6        this.queryParams.startTime = ''
7        this.queryParams.endTime = ''
8        this.reset = false
```

```

9         this.allPage = 0 // 有值意味着只有一条 因为是根据单号传的，所以不让他
    继续往后查 否则会重复
10         this.queryParams.transportTaskId = value
11     }else {
12         this.allPage = 1 // 没值意味着查不到 重新加载
13         this.queryParams.transportTaskId = ""
14     }
15     const result = await getTaskList(this.queryParams)
16     this.taskListData = result.items || []
17     this.loading.close()
18 }

```

Fence 288

- 这里逻辑捋一捋

整体代码

```

1  import { HmList, HmLoading } from '@hm/basic/Index'
2  import { getTaskList } from '../api'
3  import { TaskInfoItem, TaskInfoItemModel, TaskListParams,
    TaskListParamsModel, TaskTypeEnum } from '../models'
4  import TaskItemCard from './TaskItemCard'
5  import { promptAction } from '@kit.ArkUI'
6
7  // 待提货
8  @Component
9  struct TaskList {
10     loading: CustomDialogController = new CustomDialogController({
11         builder: HmLoading(),
12         customStyle: true
13     })
14     @State
15     queryParams: TaskListParamsModel = new TaskListParamsModel(
16         {
17             status: TaskTypeEnum.waiting, // 待提货的类型
18             page: 1, // 第几页
19             pageSize: 5 // 每页几条数据
20         } as TaskListParams
21     )
22     @State
23     taskListData: TaskInfoItem[] = []
24     @State
25     allPage: number = 1 // 默认只有一页
26     @State
27     reset: boolean = false // 用来控制重置状态
28
29     async getTaskList(append: boolean) {
30         const result = await getTaskList(this.queryParams)
31         // 追加数据
32         // this.taskListData = this.taskListData.concat(result.items) // 拿到返回的数组
33         if (append) {
34             this.taskListData.push(...result.items || []) // 延展运算符的写法
35         } else {
36             this.taskListData = result.items // 直接赋值
37         }

```

```

38
39     this.allPage = result.pages // 总页数
40     this.queryParams.page++ // 下次请求的页码
41 }
42
43 @Builder
44 renderItem(item: object) {
45     TaskItemCard({ taskItem: item as TaskInfoItemModel })
46 }
47
48 // 下拉刷新函数
49 async onRefresh() {
50     // 重新请求第一页数据
51     this.queryParams.page = 1 // 重置第一页
52     await this.getTaskList(false) // 直接赋值
53 }
54
55 // 补零函数
56 addZero(value: number) {
57     return value.toString().padStart(2, "0")
58 }
59
60 // 获取筛选按钮是否可用
61 getSearchEnable() {
62     return !(this.queryParams.startTime && this.queryParams.endTime)
63 }
64
65 @Builder
66 getSearchForm() {
67     Column() {
68         Row() {
69             Search({ placeholder: '请输入任务编号', value:
this.queryParams.transportTaskId || "" })
70             .backgroundColor($r('app.color.background_page'))
71             .height(32)
72             .onSubmit(async (value) => {
73                 // 点击键盘的右下角的提交
74                 // 4042715413936324752
75                 this.loading.open()
76                 this.allPage = 1 // 总页数为1
77                 this.queryParams.page = 1 // 查第一页
78                 if (value) {
79                     // 有单号的情况 如果有只有一条记录
80                     // 后台业务缺陷- 按道理来说 如果传单号的了 应该开始时间和结束时间自动忽略
81                     this.queryParams.startTime = ""
82                     this.queryParams.endTime = ""
83                     this.queryParams.transportTaskId = value
84                 } else {
85                     // 没有单号的情况下恢复之前的查询
86                     this.queryParams.transportTaskId = ""
87                 }
88                 await this.getTaskList(false) // 不追加数据
89                 this.loading.close()
90             })

```

```

91     }
92     .justifyContent(FlexAlign.Center)
93     .padding({ left: 15, right: 15, bottom: 5 })
94
95     Row() {
96         // 完成搜索页需要测试点击之后键盘和弹层同时弹出的情况
97         Button(this.queryParams.startTime || '开始时间')
98             .fontSize(14)
99             .width(106)
100            .height(32)
101            .padding({ left: 0, right: 0 })
102            .fontColor('#999')
103            .backgroundColor($r('app.color.background_page'))
104            .onClick(() => {
105                DatePickerDialog.show({
106                    selected: new Date(),
107                    onDateAccept: (value) => {
108                        this.queryParams.startTime =
109                            `${value.getFullYear()}-${this.addZero(value.getMonth() +
110                                1)}-${this.addZero(value.getDate())}`
111                    })
112                })
113            })
114
115            Text("至")
116            Button(this.queryParams.endTime || '结束时间')
117                .fontSize(14)
118                .width(110)
119                .height(32)
120                .padding({ left: 0, right: 0 })
121                .fontColor('#999')
122                .backgroundColor($r('app.color.background_page'))
123                .onClick(() => {
124                    DatePickerDialog.show({
125                        selected: new Date(),
126                        onDateAccept: (value) => {
127                            this.queryParams.endTime =
128                                `${value.getFullYear()}-${this.addZero(value.getMonth() +
129                                    1)}-${this.addZero(value.getDate())}`
130                        })
131                    })
132                })
133
134                Button(this.reset ? "重置" : "筛选")
135                    .backgroundColor($r('app.color.primary'))
136                    .height(32)
137                    .width(60)
138                    .enabled(this.getSearchEnable())
139                    .onClick(async () => {
140                        if (this.reset) {
141                            // 表示现在需要重置 将开始时间和结束时间清空
142                            this.queryParams.startTime = ""
143                            this.queryParams.endTime = ""
144                        }
145                        this.reset = !this.reset // 状态取反
146                    })
147            })
148    }

```

```

144         this.loading.open()
145         // 搜索
146         // 处理总页数 处理第几页
147         this.allPage = 1 // 默认有一页
148         this.queryParams.page = 1
149         // 获取数据
150         await this.getTaskList(false) // true是追加 false是不追加
151
152         this.loading.close()
153
154     })
155
156     }.width('100%').alignItems(VerticalAlign.Center).justifyContent(FlexAlign.SpaceAround)
157     .backgroundColor($r('app.color.white'))
158     .padding(15)
159     .justifyContent(FlexAlign.Center)
160     .width('100%')
161   }
162
163   build() {
164     column() {
165       // 是否显示搜索条件
166       if (this.queryParams.status === TaskTypeEnum.Finish) {
167         // 显示搜索条件
168         this.getSearchForm()
169       }
170       HmList({
171         dataSource: this.taskListData, // 数据源
172         finished: this.allPage < this.queryParams.page, // 是否还有下一页
173         // 上拉加载的函数
174         onLoad: async () => {
175           // 上拉加载
176           await this.getTaskList(true) // 追加逻辑
177         },
178         onRefresh: async () => {
179           // 下拉刷新
180           await this.onRefresh()
181         },
182         renderItem: (item: object) => {
183           // 如果需要的是builderParams的参数 可以用普通函数包裹一个Builder的函数
184           this.renderItem(item)
185         },
186         loadingText: '拼命加载中',
187         finishText: '没啦没啦'
188       }).height("100%")
189     }
190
191   }
192 }
193
194 export default TaskList

```

:::info

- 提交代码

:::

多线程处理图片压缩

性能优化

- 资源-图片不要全用原图
- 请求-尽可能延后-最好不要把所有请求都放在入口处
- 组件-尽可能采用拆分组件-延展组件-低开门-高入户
- 设计-多层架构-高内聚低耦合-公用har-包共用hsp
- 多任务处理-多线程-耗时任务-IO操作-文件压缩-解压缩-裁剪-位移
- 主线程-子线程处理耗时任务 ([诸多限制](#))

1. 建立多线程 子线程
2. 将主线程拿到的图片给到子线程
3. 子线程进行图片的压缩
4. 压缩完成 将完成的图片信息回传给主线程
5. 子线程自动关闭销毁
6. 主线程继续上传操作

- 新建一个UploadWorker

```
1 import { ImageList } from '@hm/basic/Index';
2 import { ErrorEvent, MessageEvents, ThreadWorkerGlobalScope, util, worker }
  from '@kit.ArKTS';
3 import { fileIo } from '@kit.CoreFileKit';
4 import { image } from '@kit.ImageKit';
5
6 const workerPort: ThreadWorkerGlobalScope = worker.workerPort;
7
8 export class PostParams {
9   files: ImageList[] = []
10   filePath: string = "" // 沙箱目录
11 }
12
13 /**
14  * Defines the event handler to be called when the worker thread receives a
15  * message sent by the host thread.
16  * The event handler is executed in the worker thread.
17  *
18  * @param e message data
19  */
20 workerPort.onmessage = async (e: MessageEvents) => {
21   // 处理耗时任务e
22   const params = e.data as PostParams
23   // 进行图片压缩
24   if (params && params.files) {
25     // 拿到原图
26     // 对原图进行压缩
27     // 需要将原图压缩成新的图片 存到一个位置
28     const imagePackerAPI = image.createImagePacker() // 创建图片压缩API
29     let packOpts: image.PackingOption = { format: "image/jpeg", quality: 20
30   };
31   let arr: ImageList[] = []
32   while (params.files.length) {
33     const obj = params.files.pop() // 每次取一条
```

```

32     const sourceFile = fileIo.openSync(obj?.url,
fileIo.OpenMode.READ_ONLY) // 打开来源文件
33     // 对来源文件进行压缩
34     const newFileName = params.filePath + "/" + util.generateRandomUUID()
+ ".jpg"
35     const targetFile = fileIo.openSync(newFileName, fileIo.OpenMode.CREATE
| fileIo.OpenMode.READ_WRITE)
36     // 目标文件的fd
37     await
imagePackerAPI.packToFile(image.createImageSource(sourceFile.fd),
targetFile.fd, packOpts)
38     fileIo.closeSync(sourceFile.fd) // 关闭来源文件
39     fileIo.closeSync(targetFile.fd) // 关闭目标文件
40     arr.push({
41         url: newFileName
42     })
43 }
44 // 图片压缩完毕 将图片传出去
45 // 这里不需要序列化
46 workerPort.postMessage({ files: arr }) // 子线程往主线程发消息
47 workerPort.close() // 结束子线程
48
49 }
50
51
52 }
53
54 /**
55  * Defines the event handler to be called when the worker receives a message
that cannot be deserialized.
56  * The event handler is executed in the worker thread.
57  *
58  * @param e message data
59  */
60 workerPort.onmessageerror = (e: MessageEvents) => {
61 }
62
63 /**
64  * Defines the event handler to be called when an exception occurs during
worker execution.
65  * The event handler is executed in the worker thread.
66  *
67  * @param e error message
68  */
69 workerPort.onerror = (e: ErrorEvent) => {
70 }

```

Fence 290

- 提货交货时处理图片压缩

```

1
2 // 交货
3 async onDeliver() {
4     this.loading.open()
5     // 建立子线程

```

```

6      const imageCompress1 = new
worker.Threadworker("entry/ets/workers/UploadWorker.ets") // 子线程1
7      const imageCompress2 = new
worker.Threadworker("entry/ets/workers/UploadWorker.ets") // 子线程2
8      let certificatePictureList: ImageList[] = []
9      let deliverPictureList: ImageList[] = []
10
11     // 用来检查是否全部都压缩完毕
12     const checkFile = async () => {
13         if (certificatePictureList.length && deliverPictureList.length) {
14             // 此时此刻才可以进行下一步操作
15             const result = await
Promise.all([UploadFile(certificatePictureList),
UploadFile(deliverPictureList)])
16             await deliver(new DeliverParamsTypeModel({
17                 id: this.taskDetailData.id,
18                 certificatePictureList: result[0],
19                 deliverPictureList: result[1]
20             }))
21             this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
22             this.scroller.scrollToEdge(Edge.Top)
23             promptAction.showToast({ message: '交货成功' })
24             this.loading.close()
25         }
26     }
27
28     imageCompress1.onmessage = (e: MessageEvents) => {
29         const params = e.data as PostParams
30         certificatePictureList = params.files // 已经压缩完毕的图片
31         checkFile() // 检查是否都完成了
32     }
33     imageCompress2.onmessage = (e: MessageEvents) => {
34         const params = e.data as PostParams
35         deliverPictureList = params.files // 已经压缩完毕的图片
36         checkFile()
37     }
38     imageCompress1.postMessage({
39         files: [...this.taskDetailData.certificatePictureList],
40         filePath: getContext().filesDir
41     }) // 给子线程发消息
42     imageCompress2.postMessage({
43         files: [...this.taskDetailData.deliverPictureList],
44         filePath: getContext().filesDir
45     }) // 给子线程发消息
46
47
48     // this.loading.open()
49     // const certificatePictureList = await
UploadFile(this.taskDetailData.certificatePictureList) // 传入提货凭证
50     // const deliverPictureList = await
UploadFile(this.taskDetailData.deliverPictureList) // 货品照片
51     // // 交货操作
52     // await deliver(new DeliverParamsTypeModel({
53     //     id: this.taskDetailData.id,
54     //     certificatePictureList,

```

```
55     // deliverPictureList
56     // })
57     // // 只需要重新获取数据
58     // // 重新获取数据
59     // this.getTaskDetail(this.taskDetailData.id) // 重新拉取数据
60     //
61     // this.scroller.scrollToEdge(Edge.Top)
62     // this.loading.close()
63     // // 滚动到顶部
64     // promptAction.showToast({ message: '交货成功' })
65
66 }
```

集成消息模块

:::info

消息模块内容较为简单

同学们将老师提供的模块导入到项目中集成即可，也可以自己尝试自己去编写

:::

entry模块

1. 新建models/message.ts

```
1  export interface MessageQueryType {
2    /** 消息类型，200：司机端公告，201：司机端系统通知 */
3    contentType: number;
4    page: number;
5    pageSize: number;
6  }
7
8
9  export interface MessageData {
10   /** 总条目数 */
11   counts: number;
12   items: MessageItem[];
13   /** 页码 */
14   page: number;
15   /** 总页数 */
16   pages: number;
17   /** 页尺寸 */
18   pageSize: number;
19 }
20
21 /** 数据列表 */
22 export interface MessageItem {
23   /** 1：用户端，2：司机端，3：快递员端，4：后台管理系统 */
24   bussinessType: number;
25   /** 消息内容 */
26   content: string;
27   /** 消息类型，300：快递员端公告，301：寄件相关消息，302：签收相关消息，303：快件取消
消息，200：司机端公告，201：司机端系统通知 */
28   contentType: number;
29   /** 创建时间 */
30   created: string;
31   /** 创建者 */
32   createUser: number;
33   /** 主键 */
34   id: number;
35   /** 消息是否已读，0：未读，1：已读 */
36   isRead: number;
37   /** 读时间 */
38   readTime: string;
39   /** 相关id */
40   relevantId: number;
41   /** 消息标题 */
42   title: string;
```

```
43  /** 更新时间 */
44  updated: string;
45  /** 更新者 */
46  updateUser: number;
47  /** 消息接受者 */
48  userId: number;
49  }
50  interface DetailType {
51    id: string
52    content: string
53    created: string
54  }
55
56  export class MessageQueryTypeModel implements MessageQueryType {
57    contentType: number = 0
58    page: number = 0
59    pageSize: number = 0
60
61    constructor(model: MessageQueryType) {
62      this.contentType = model.contentType
63      this.page = model.page
64      this.pageSize = model.pageSize
65    }
66  }
67  export class MessageDataModel implements MessageData {
68    counts: number = 0
69    items: MessageItem[] = []
70    page: number = 0
71    pages: number = 0
72    pageSize: number = 0
73
74    constructor(model: MessageData) {
75      this.counts = model.counts
76      this.items = model.items
77      this.page = model.page
78      this.pages = model.pages
79      this.pageSize = model.pageSize
80    }
81  }
82  export class MessageItemModel implements MessageItem {
83    bussinessType: number = 0
84    content: string = ''
85    contentType: number = 0
86    created: string = ''
87    createUser: number = 0
88    id: number = 0
89    isRead: number = 0
90    readTime: string = ''
91    relevantId: number = 0
92    title: string = ''
93    updated: string = ''
94    updateUser: number = 0
95    userId: number = 0
96
97    constructor(model: MessageItem) {
```

```

98     this.bussinesType = model.bussinesType
99     this.content = model.content
100    this.contentType = model.contentType
101    this.created = model.created
102    this.createUser = model.createUser
103    this.id = model.id
104    this.isRead = model.isRead
105    this.readTime = model.readTime
106    this.relevantId = model.relevantId
107    this.title = model.title
108    this.updated = model.updated
109    this.updateUser = model.updateUser
110    this.userId = model.userId
111  }
112 }
113 export class DetailTypeModel implements DetailType {
114   id: string = ''
115   content: string = ''
116   created: string = ''
117
118   constructor(model: DetailType) {
119     this.id = model.id
120     this.content = model.content
121     this.created = model.created
122   }
123 }
124

```

Fence 292

- 在models/index.ts中导出

```

1 | export * from './message'

```

Fence 293

- 新建api/message.ets

```

1 | import { Request } from '@hm/basic'
2 | import { MessageQueryTypeModel, MessageDataModel } from '../models'
3 |
4 | // 获取信息
5 | export const getMessage = (params: MessageQueryTypeModel) => {
6 |   return Request.get<MessageDataModel>("/driver/messages/page", params)
7 | }
8 |
9 | // 标记已读
10 | export const readMessage = (id: string) => {
11 |   return Request.put<null>(`/driver/messages/${id}`)
12 | }
13 |
14 | // 全部已读
15 | export const readAllMessage = (contentType: string) => {
16 |   return Request.put<null>(`/driver/messages/readAll/${contentType}`)
17 | }

```

Fence 294

- 在api/index.ets导出

```
1 export * from './message'
```

Fence 295

- 在pages/Index下新建Message/Message.ets组件

```
1 import { HmNavBar, TabClass } from '@hm/basic'
2 import MessageTemplate from './MessageTemplate'
3 @Component
4 struct Message {
5     @State currentTabName: string = 'information'
6     @State tabBarData: TabClass[] = [{
7         title: '公告',
8         name: 'information'
9     }, {
10        title: '任务通知',
11        name: 'notice'
12    }]
13    @Builder
14    TabBuilder(item: TabClass) {
15        Column() {
16            Text(item.title)
17                .fontSize(16)
18                .fontColor(this.currentTabName === item.name ?
19                $r('app.color.text_primary') : $r('app.color.text_secondary'))
20                .fontWeight(this.currentTabName === item.name ? 500 : 400)
21                .lineHeight(50)
22                .height(50)
23            Divider()
24                .strokeWidth(4)
25                .color($r('app.color.primary'))
26                .opacity(this.currentTabName === item.name ? 1 : 0)
27                .width(this.currentTabName === item.name ? 23 : 0)
28                .animation({
29                    duration: 300,
30                    curve: Curve.EaseOut,
31                    iterations: 1,
32                    playMode: PlayMode.Normal
33                })
34        }
35    }
36    build() {
37        Column() {
38            HmNavBar({ title: '消息', showBackIcon: false })
39            Tabs({
40                barPosition: BarPosition.Start,
41                controller: new TabsController()
42            }) {
43                ForEach(this.tabBarData, (item: TabClass) => {
44                    TabContent() {
45                        Flex() {
46                            MessageTemplate({ type: item.name || '' })
```



```

47     }.height('100%').backgroundColor($r('app.color.background_page'))
48     }.tabBar(this.TabBuilder(item))
49     })
50     }.animationDuration(300).onChange((index) => {
51         this.currentTabName = index === 0 ? 'information' : 'notice'
52     })
53     }.width('100%').height('100%')
54 }
55 }
56
57 export default Message

```

Fence 296

- 在Message.ets旁新建MessageTemplate.ets

```

1  import { getMessage, readAllMessage, readMessage } from '../../api'
2  import { MessageQueryTypeModel, MessageItemModel } from '../../models'
3  import router from '@ohos.router';
4  import { HmList } from '../../components'
5  import promptAction from '@ohos.promptAction';
6
7  @Component
8  struct MessageTemplate {
9      @Prop
10     type: string
11     @State
12     queryParams: MessageQueryTypeModel = new MessageQueryTypeModel({
13         page: 1,
14         pageSize: 10,
15         contentType: this.type === "information" ? 200 : 201
16     })
17     @State
18     allPage: number = 1
19     @State
20     messageList: MessageItemModel[] = []
21
22     async getMessage(append: boolean) {
23         if (this.allPage < this.queryParams.page) {
24             return
25         }
26         const result = await getMessage(this.queryParams)
27         if (append) {
28             this.messageList = this.messageList.concat(result.items) // 追加数据
29         } else {
30             this.messageList = result.items // 覆盖数据
31         }
32         this.queryParams.page++
33         this.allPage = result.pages
34     }
35     async refreshData() {
36         this.allPage = 1 // 只要下拉就默认有一页
37         this.queryParams.page = 1
38         await this.getMessage(false)
39         promptAction.showToast({ message: '刷新成功' })

```

```

40     }
41
42     @Builder
43     renderItem(item: object) {
44         if(this.type === "information") {
45             this.renderInformationItem(item as MessageItemModel)
46         }
47         if(this.type === "notice") {
48             this.renderNoticeItem(item as MessageItemModel)
49         }
50     }
51     // 读取
52     readSuccess() {
53         this.messageList = this.messageList.map(item => {
54             item.isRead = 1
55             return item
56         })
57     }
58     @Builder
59     renderInformationItem (item: MessageItemModel) {
60         Row() {
61             Row() {
62                 if (item.isRead === 0) {
63
64                     Text("").width(8).height(8).backgroundColor($r('app.color.primary')).borderRadius(4)
65
66                     Text(item.content)
67                         .fontSize(14)
68                         .fontColor($r("app.color.text_primary"))
69                         .margin({ left: 6 })
70                         .textOverflow({ overflow: TextOverflow.Ellipsis })
71                         .maxLines(1)
72                     }.width(230)
73
74                     Text(item.created).fontSize(12).fontColor($r("app.color.text_secondary"))
75                 }
76                 .justifyContent(FlexAlign.SpaceBetween)
77                 .alignItems(VerticalAlign.Center)
78                 .padding({ left: 22, right: 22 })
79                 .height(60)
80                 .backgroundColor($r('app.color.white'))
81                 .border({ width: { bottom: 1 }, color: $r("app.color.background_page")
82             })
83             .width('100%')
84             .onClick(() => {
85                 // 先更改数据状态
86                 item.isRead = 1
87                 this.messageList = [...this.messageList]
88                 router.pushUrl({
89                     url: 'pages/Index/Message/MessageDetail',
90                     params: {
91                         content: item.content,
92                         created: item.created,

```

```

91         id: item.id
92     }
93 })
94 })
95 }
96 @Builder
97 renderNoticeItem(message: MessageItemModel) {
98     Column() {
99         Row() {
100             Text("您有新的运输任务")
101             if (message.isRead === 0) {
102                 Text("")
103                     .width(8)
104                     .height(8)
105                     .backgroundColor($r('app.color.primary'))
106                     .borderRadius(4)
107                     .margin({ left: 10 })
108             }
109         }
110
111         Divider().color($r('app.color.background_page')).margin({ top: 13 })
112         Text(message.content.replace(new RegExp("/\n/"), ""))
113             .margin({ top: 11, bottom: 22.5 })
114             .fontSize(13)
115             .fontColor($r('app.color.text_secondary'))
116             .maxLines(2)
117             .textOverflow({ overflow: TextOverflow.Ellipsis })
118             .lineHeight(22)
119         Row() {
120
121             Text(message.updated).fontSize(12).fontColor($r('app.color.text_secondary'
122             ))
123
124             Button("查看详情", { type: ButtonType.Capsule })
125                 .fontColor($r('app.color.primary'))
126                 .border({ width: 1, color: $r('app.color.primary') })
127                 .height(24)
128                 .width(76)
129                 .backgroundColor('#fff')
130                 .onClick(() => {
131                     readMessage(message.id + "")
132                     // 跳转到任务详情
133                     router.pushUrl({
134                         url: 'pages/TaskDetail/TaskDetail',
135                         params: {
136                             id: message.relevantId
137                         }
138                     })
139                 })
140             })
141
142         }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).width('100%')
143     }
144     .width('100%')
145     .backgroundColor($r('app.color.white'))
146     .borderRadius(10)

```

```

142     .padding(16)
143     .margin({ bottom: 15 })
144     .alignItems(HorizontalAlign.Start)
145 }
146
147 build() {
148     Column() {
149         Row() {
150             Image($r("app.media.ic_yidu")).width(16).height(16)
151             Text("全部已
152 读").fontSize(14).fontColor($r('app.color.text_secondary')).margin({
153                 left: 9
154             }).onClick(async () => {
155                 this.queryParams.contentType && await
156                 readAllMessage(this.queryParams.contentType.toString())
157                 promptAction.showToast({ message: '全部已读' })
158                 this.readSuccess && this.readSuccess()
159             })
160             .padding({
161                 left: 17,
162                 right: 17,
163                 top: 14,
164                 bottom: 14
165             }).width('100%')// 全部已读
166             // 公告内容
167             HmList({
168                 finished: this.allPage < this.queryParams.page,
169                 dataSource: $messageList,
170                 onRefresh: async () => {
171                     await this.refreshData()
172                 },
173                 onLoad: async () => {
174                     await this.getMessage(true)
175                 },
176                 renderItem: (item: object) => {
177                     this.renderItem(item)
178                 }
179             })
180         }
181     }
182     export default MessageTemplate

```

Fence 297

- 在Index/Index.ets中引入Message

```

1 import Message from './Message/Message'
2
3
4 build() {
5     Tabs({ barPosition: BarPosition.End }) {
6         ForEach(this.tabsData, (item: TabClass) => {
7             TabContent() {
8                 if(item.name === 'task') {

```

```

9         TaskTabs()
10        }
11        else if(item.name === 'message') {
12            Message()
13        }
14        else {
15            My()
16        }
17        }.tabBar(this.getTabBar(item))
18    })
19 }
20 .onChange(index => {
21     this.currentName = this.tabsData[index].name
22 })
23 .animationDuration(300)
24 }

```

Fence 298

- 在pages/Index/Message下新建MessageDetail.ets(page)

```

1  import { HmNavBar } from '@hm/basic'
2  import router from '@ohos.router';
3  import { readMessage } from '../api/message'
4  import { DetailTypeModel } from '../models'
5  @Entry
6  @Component
7  struct MessageDetail {
8      @State detailForm: DetailTypeModel = new DetailTypeModel({
9          content: '',
10         created: '',
11         id: ''
12     })
13
14     async aboutToAppear() {
15         const params = router.getParams()
16         this.detailForm = params as DetailTypeModel
17         readMessage(this.detailForm.id)
18     }
19
20     build() {
21         Flex({ direction: FlexDirection.Column }) {
22             HmNavBar({ title: '详情' })
23             Column() {
24                 Text("系统公
告").fontSize(16).fontColor($r('app.color.text_primary')).lineHeight(22)
25
26                 Text(this.detailForm.created).fontSize(12).fontColor($r('app.color.text_sec
ondary')).lineHeight(17).margin({
27                     top: 7,
28                     bottom: 17
29                 })
30                 Text(this.detailForm.content).fontSize(14).lineHeight(22)
31             }.alignItems(HorizontalAlign.Start).padding({
32                 top: 20,
33                 left: 16,

```

```
33         right: 16
34     })
35     }.height('100%').backgroundColor($r('app.color.white'))
36 }
37 }
38
39 export default MessageDetail
40
```

集成车辆信息

entry模块

在models/car.ets中加入车辆信息类型

```
1 import { ImageList } from '@hm/basic'
2 /** 响应数据 */
3 export interface UserCarDataType {
4     /** 载重 */
5     allowableLoad: string;
6     /** 所属机构名称 */
7     currentOrganName: string;
8     /** 车辆编号 */
9     id: string;
10    /** 车牌号码 */
11    licensePlate: string;
12    /** 图片 */
13    pictureList: ImageList[];
14    /** 车辆类型名称 */
15    truckType: string;
16 }
17 export class UserCarDataTypeModel implements UserCarDataType {
18     allowableLoad: string = ''
19     currentOrganName: string = ''
20     id: string = ''
21     licensePlate: string = ''
22     pictureList: ImageList[] = []
23     truckType: string = ''
24
25     constructor(model: UserCarDataType) {
26         this.allowableLoad = model.allowableLoad
27         this.currentOrganName = model.currentOrganName
28         this.id = model.id
29         this.licensePlate = model.licensePlate
30         this.pictureList = model.pictureList
31         this.truckType = model.truckType
32     }
33 }
34
```

Fence 300

- 在models/index.ets统一导出

```
1 export * from './car'
```

Fence 301

在api/user.ts中加入封装获取车辆信息api

```
1 // 获取用户车辆信息
2 export const getUserCarInfo = () => {
3     return Request.get<UserCarDataTypeModel>("/driver/users/truck")
4 }
```

```
1 import { HmNavBar } from '@hm/basic'
2 import { getUserCarInfo } from '../api'
3 import { UserCarDataTypeModel, UserCarDataType, ImageList } from
  '../models'
4 @Entry
5 @Component
6 struct Car {
7   @State
8   userCarInfo: UserCarDataTypeModel = new UserCarDataTypeModel({} as
UserCarDataType)
9   aboutToAppear() {
10     this.getUserCarInfo()
11   }
12   async getUserCarInfo() {
13     this.userCarInfo = await getUserCarInfo()
14   }
15   @Builder
16   getContentItem (item: CarItem) {
17     Row() {
18
19       Text(item.leftText).fontSize(14).fontWeight(400).fontColor($r('app.color.te
xt_secondary'))
20
21       Text(item.rightValue).fontSize(14).fontColor($r('app.color.text_primary')).
fontWeight(400)
22     }.justifyContent(FlexAlign.SpaceBetween).alignItems(VerticalAlign.Center).wi
dth('100%').height(40)
23   }
24   build() {
25     Column() {
26       HmNavBar({ title: '车辆信息' })
27       Swiper(new SwiperController()) {
28         ForEach(this.userCarInfo.pictureList, (item: ImageList) => {
29           Row() {
30
31             Image(item.url).width('100%').height('100%').objectFit(ImageFit.Cover).
borderRadius(8)
32             }.height(201).padding({ left: 15, right: 15, top: 15 })
33           })
34         .loop(false)
35         .cachedCount(3)
36         .indicator(true)
37         .curve(Curve.Linear)
38
39         // 信息列表
40         Column() {
41           this.getContentItem({ leftText: '车辆编号', rightValue:
this.userCarInfo.id })
42           this.getContentItem({ leftText: '车辆号牌', rightValue:
this.userCarInfo.licensePlate })
```



```

42         this.getContentItem( { leftText: '车型', rightValue:
this.userCarInfo.truckType })
43         this.getContentItem({ leftText: '所属机构', rightValue:
this.userCarInfo.currentOrgName })
44         this.getContentItem({ leftText: '载重', rightValue:
this.userCarInfo.allowableLoad })
45     }
46     .padding({
47         top: 19.5,
48         bottom: 19.5,
49         left: 20,
50         right: 20
51     })
52     .backgroundColor($r('app.color.white'))
53     .margin(15)
54     .borderRadius(8)
55
    }.height('100%').backgroundColor($r('app.color.background_page')).width('100
%')
56     }
57 }
58
59
60 class CarItem {
61     leftText: string = ""
62     rightValue: string = ""
63 }
64 export default Car

```

Fence 303

- 点击我的-车辆信息跳转过去

```

1  HmCardItem({ leftText: '车辆信息', rightText: '', onRightClick: () => {
2      router.pushUrl({
3          url: 'pages/Car/Car'
4      })
5  } })

```

Fence 304



车辆信息



车辆编号	1575257934065905665
车辆号牌	粤A11223
车型	12.8米箱式货车hjfhjvhj
所属机构	沈阳分拣中心
载重	10.00



黑马程序员

:::info
提交代码
:::

集成任务信息

:::info

获取任务数据之前已经封装过，直接使用即可

:::

- 新建pages/UserTask/UserTask.ets

```
1 import { HmNavBar, HmLoading } from '@hm/basic'
2 import { UserTaskInfoModel, UserTaskInfo, UserTaskInfoParamsModel } from
  '../..models'
3 import { getUserTaskInfo } from '../..api'
4 @Entry
5 @Component
6 struct UserTask {
7   @State TaskInfo: UserTaskInfoModel = new UserTaskInfoModel({} as
    UserTaskInfo)
8   @State list: string[] = []
9   @State queryParams: UserTaskInfoParamsModel = new
    UserTaskInfoParamsModel({
10     year: new Date().getFullYear()+'',
11     month: new Date().getMonth() + 1 +''
12   })
13   @State
14   currentDate: Date = new Date()
15   layer: CustomDialogController = new CustomDialogController({
16     builder: HmLoading(),
17     customStyle: true,
18     alignment: DialogAlignment.Center
19   })
20   async aboutToAppear() {
21     this.getUserTask()
22   }
23   async getUserTask () {
24     this.layer.open()
25     this.TaskInfo = await getUserTaskInfo(this.queryParams)
26     this.layer.close()
27   }
28   build() {
29     column() {
30       HmNavBar({ title: '任务数据' })
31
32       // 本月任务
33       Flex({ direction: FlexDirection.Column, alignItems: ItemAlign.Center,
        justifyContent: FlexAlign.SpaceAround }) {
34         Text("- 本月任务 -
35         ").fontSize(14).fontColor($r('app.color.text_secondary')).lineHeight(20)
36         Row() {
37           Column() {
38             Text(this.TaskInfo.taskAmounts+
39             "").fontSize(18).fontColor($r('app.color.text_primary')).lineHeight(25).margin({
38               bottom: 17
39             })

```

```

40         Text("任务总
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
41     }.justifyContent(FlexAlign.SpaceAround)
42
43     Column() {
44         Text(this.TaskInfo.completedAmounts+"")
45             .fontSize(18)
46             .fontColor($r('app.color.text_primary'))
47             .lineHeight(25)
48             .margin({ bottom: 17 })
49         Text("完成任务
量").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
50     }.justifyContent(FlexAlign.SpaceAround)
51
52     Column() {
53         Text(this.TaskInfo.transportMileage+"")
54             .fontSize(18)
55             .fontColor($r('app.color.text_primary'))
56             .lineHeight(25)
57             .margin({ bottom: 17 })
58         Text("运输里程
(km)").fontSize(12).fontColor($r('app.color.text_primary')).lineHeight(17)
59     }.justifyContent(FlexAlign.SpaceAround)
60     }.justifyContent(FlexAlign.SpaceEvenly).width('100%').flexGrow(1)
61
62     }
63     .backgroundColor($r('app.color.white'))
64     .margin({ left: 14.5, right: 14.5 })
65     .height(100).margin({
66         top: 20,
67         bottom: 20
68     })
69     Row() {
70         Button("切换月份")
71             .backgroundColor($r("app.color.primary"))
72             .onClick(() => {
73                 CalendarPickerDialog.show({
74                     selected: this.currentDate,
75                     onDateAccept: (value) => {
76                         this.queryParams.year = value.getFullYear().toString()
77                         this.queryParams.month = (value.getMonth() + 1).toString()
78                         this.getUserTask()
79                     }
80                 })
81             })
82     }
83     .justifyContent(FlexAlign.Center)
84     .width('100%')
85
86 }
87 .height('100%').backgroundColor($r('app.color.background_page'))
88 }
89 }

```

- 在我的页面跳转过去

```
1  HmCardItem({ leftText: '任务设置', rightText: '',
2      onRightClick: () => {
3          router.pushUrl({
4              url: 'pages/UserTask/UserTask'
5          })
6      }
7  })
```

Fence 306



:::info
提交代码
:::

做一个截图效果

- 实现一个转发按钮

```
1 Row() {  
2     Image($r("app.media.share"))  
3     .width(20)  
4     .height(20)  
5     .fillColor($r("app.color.primary"))  
6     .onClick(async () => {  
7         this.snapImg = await componentSnapshot.get("detail")  
8         this.showSnap = true  
9     })  
10 }  
11 .width("100%")  
12 .justifyContent(FlexAlign.End)  
13 .padding(10)
```



- 定义变量

```
1 @State
2 snapImg: image.PixelMap | null = null
3 @State
4 showSnap: boolean = false
```

Fence 308

- 使用bindContentCover

```
1 .bindContentCover($$this.showSnap, this.getSnapContent(), {
2     modalTransition: ModalTransition.NONE
3 })
```

Fence 309

- 实现弹出内容

```
1  @Builder
2  getSnapContent () {
3    column() {
4      Image(this.snapImg)
5        .width("100%")
6        .height("100%")
7        .objectFit(ImageFit.Auto)
8        .borderRadius(6)
9    }
10   .padding("10%")
11   .width("100%")
12   .height("100%")
13   .backgroundColor("rgba(0,0,0,0.2)")
14   .onClick(() => {
15     this.showSnap = false
16   })
17 }
```




LazyForEach应用

LazyForEach从提供的数据源中按需迭代数据，并在每次迭代过程中创建相应的组件。当在滚动容器中使用了LazyForEach，框架会根据滚动容器可视区域按需创建组件，当组件滑出可视区域外时，框架会进行组件销毁回收以降低内存占用。

- 接口描述

```
1 LazyForEach(  
2     dataSource: IDataSource,           // 需要进行数据迭代的数据源  
3     itemGenerator: (item: any, index: number) => void, // 子组件生成函数  
4     keyGenerator?: (item: any, index: number) => string // 键值生成函数  
5 ): void
```

Fence 311

本质上-LazyForEach和ForEach的用法基本一致，但是ForEach属于全量渲染，而LazyForEach属于只渲染可见区域

- 限制

- LazyForEach必须在容器组件内使用，仅有[List](#)、[Grid](#)、[Swiper](#)以及[WaterFlow](#)组件支持数据懒加载（可配置cachedCount属性，即只加载可视部分以及其前后少量数据用于缓冲），其他组件仍然是一次性加载所有的数据。
- LazyForEach在每次迭代中，必须创建且只允许创建一个子组件。
- 生成的子组件必须是允许包含在LazyForEach父容器组件中的子组件。
- 允许LazyForEach包含在if/else条件渲染语句中，也允许LazyForEach中出现if/else条件渲染语句。
- 键值生成器必须针对每个数据生成唯一的值，如果键值相同，将导致键值相同的UI组件渲染出现问题。
- **LazyForEach必须使用DataChangeListener对象来进行更新，第一个参数dataSource使用状态变量时，状态变量改变不会触发LazyForEach的UI刷新。**
- 为了高性能渲染，通过DataChangeListener对象的onDataChange方法来更新UI时，需要生成不同于原来的键值来触发组件刷新。

- 具体案例

[UI框架-List组件的使用之商品列表 \(ArkTS\) .zip](#)

吸顶效果

- List吸顶

```
1 // xxx.ets
2 @Entry
3 @Component
4 struct ListItemGroupExample {
5     private timeTable: TimeTable[] = [
6         {
7             title: '星期一',
8             projects: ['语文', '数学', '英语']
9         },
10        {
11            title: '星期二',
12            projects: ['物理', '化学', '生物']
13        },
14        {
15            title: '星期三',
16            projects: ['历史', '地理', '政治']
17        },
18        {
19            title: '星期四',
20            projects: ['美术', '音乐', '体育']
21        }
22    ]
23
24    @Builder
25    itemHead(text: string) {
26        Text(text)
27            .fontSize(20)
28            .backgroundColor(0xAABBCC)
29            .width("100%")
30            .padding(10)
31    }
32
33    @Builder
34    itemFoot(num: number) {
35        Text('共' + num + "节课")
36            .fontSize(16)
37            .backgroundColor(0xAABBCC)
38            .width("100%")
39            .padding(5)
40    }
41
42    build() {
43        Column() {
44            List({ space: 20 }) {
45                ForEach(this.timeTable, (item: TimeTable) => {
46                    ListItemGroup({ header: this.itemHead(item.title), footer:
47                        this.itemFoot(item.projects.length) }) {
48                        ForEach(item.projects, (project: string) => {
49                            ListItem() {
                                Text(project)
```

```

50         .width("100%")
51         .height(100)
52         .fontSize(20)
53         .textAlign(TextAlign.Center)
54         .backgroundColor(0xFFFFF)
55     }
56     }, (item: string) => item)
57 }
58 .divider({ strokeWidth: 1, color: color.Blue }) // 每行之间的分界线
59 })
60 }
61 .width('90%')
62 .sticky(StickStyle.Header | StickStyle.Footer)
63 .scrollBar(BarState.Off)
64 }.width('100%').height('100%').backgroundColor(0xDCDCDC).padding({ top:
5 5 })
65 }
66 }
67
68 interface TimeTable {
69     title: string;
70     projects: string[];
71 }

```